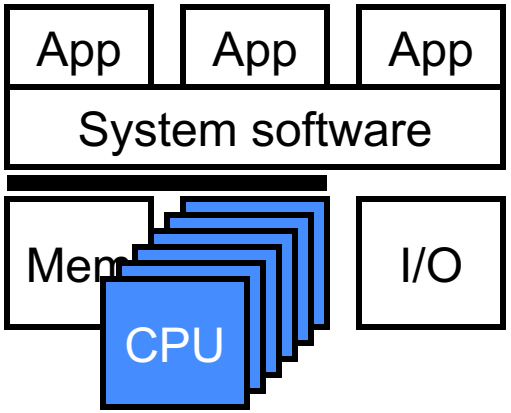
Multicore

CIS 5710

Computer Organization & Design

Shared Memory Multiprocessors



- Thread-level parallelism (TLP)
- Shared memory model
  - Multiplexed uniprocessor
  - Hardware multihreading
  - Multiprocessing
- Cache coherence
  - Valid/Invalid, MSI, MESI
- Parallel programming
- Synchronization
  - Lock implementation
  - Locking gotchas
- Memory consistency models

Readings

- P&H
  - Chapter 7.1-7.3, 7.5
  - Chapter 5.8, 5.10
- Suggested reading
  - "A Primer on Memory Consistency and Cache Coherence" (Synthesis Lectures on Computer Architecture) by Daniel Sorin, Mark Hill, and David Wood, November 2011
  - "Why On-Chip Cache Coherence is Here to Stay" by Milo Martin, Mark Hill, and Daniel Sorin, Communications of the ACM (CACM), July 2012.

Beyond Implicit Parallelism

- Consider "daxpy":

```
double a, x[SIZE], y[SIZE], z[SIZE];
void daxpy():
  for (i = 0; i < SIZE; i++)
    z[i] = a*x[i] + y[i];
```

- Lots of instruction-level parallelism (ILP)
  - Great!
  - But how much can we really exploit? 4 wide? 8 wide?
    - Limits to (efficient) super-scalar execution
- But, if SIZE is 10,000 the loop has 10,000-way parallelism!
  - How do we exploit it?

Explicit Parallelism

- Consider "daxpy":

```
double a, x[SIZE], y[SIZE], z[SIZE];
void daxpy():
  for (i = 0; i < SIZE; i++)
    z[i] = a*x[i] + y[i];
```

- Break it up into N "chunks" on N cores!
  - Done by the programmer (or maybe a *really* smart compiler)

```
void daxpy(int chunk_id):
  chuck_size = SIZE / N
  my_start = chuck_id * chuck_size
  my_end = my_start + chuck_size
  for (i = my_start; i < my_end; i++)
    z[i] = a*x[i] + y[i]
```

SIZE = 400, N=4

Chunk ID	Start	End
0	0	99
1	100	199
2	200	299
3	300	399

- Assumptions
  - Local variables are "private" and x, y, and z are "shared"
  - Assumes SIZE is a multiple of N (that is, SIZE % N == 0)

Explicit Parallelism

- Consider "daxpy":

```
double a, x[SIZE], y[SIZE], z[SIZE];
void daxpy(int chunk_id):
  chuck_size = SIZE / N
  my_start = chuck_id * chuck_size
  my_end = my_start + chuck_size
  for (i = my_start; i < my_end; i++)
    z[i] = a*x[i] + y[i]
```

- Main code then looks like:

```
parallel_daxpy():
  for (tid = 0; tid < CORES; tid++) {
    spawn_task(daxpy, tid);
  }
  wait_for_tasks(CORES);
```

Explicit (Loop-Level) Parallelism

- Another way: "OpenMP" annotations to inform the compiler

```
double a, x[SIZE], y[SIZE], z[SIZE];
void daxpy() {
  #pragma omp parallel for
  for (i = 0; i < SIZE; i++) {
    z[i] = a*x[i] + y[i];
  }
```

- But only works if loop is actually parallel
  - If not parallel, unpredictable incorrect behavior may result

Multicore & Multiprocessor Hardware

Multiplying Performance

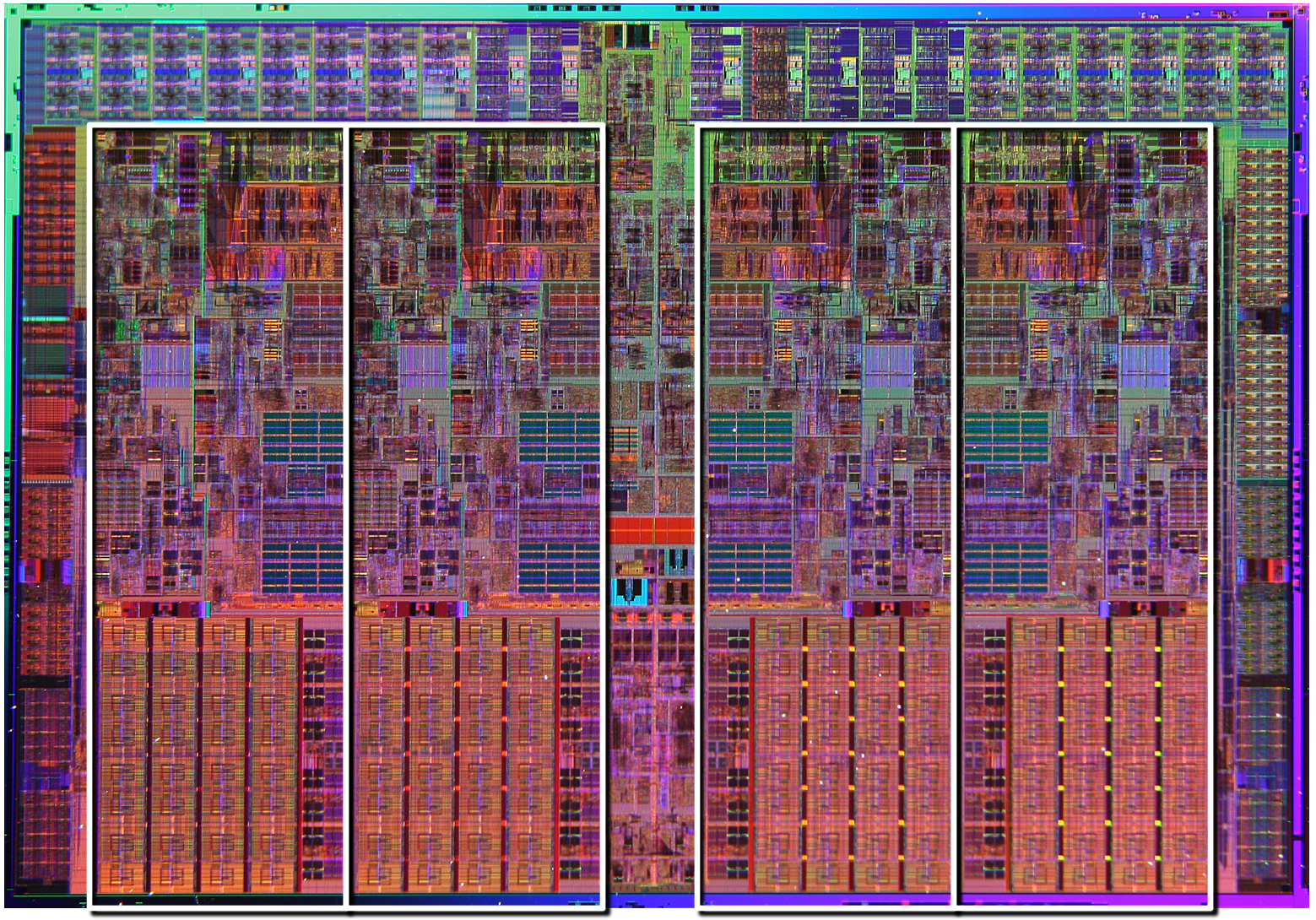
- A single core can only be so fast
  - Limited clock frequency
  - Limited instruction-level parallelism
- What if we need even more computing pow
  - Use multiple cores! But how?
- Old-school (2000s): Ultra Enterprise 25k
  - 72 dual-core UltraSPARC IV+ processors
  - Up to 1TB of memory
  - Niche: large database servers
  - $$$, weighs more than 1 ton



- Today: multicore is everywhere
  - Can't buy a single-core smartphone…
    - …or even smart watch!



Intel Quad-Core "Core i7"



Application Domains for Multicore

- Scientific computing/supercomputing
  - Examples: weather simulation, aerodynamics, protein folding
  - Large grids, integrating changes over time
  - Each core computes for a part of the grid
- Server workloads
  - Example: airline reservation database
  - Many concurrent updates, searches, lookups, queries
  - Each core handle different requests
- Video games
  - Each core renders part of a frame

But software must be written to expose parallelism

Multicore is Energy Efficient

- Explicit parallelism (multicore) is highly energy efficient
- Recall: dynamic voltage and frequency scaling
  - Performance vs power is NOT linear
  - Example: Intel's Xscale
    - 1 GHz → 200 MHz reduces energy used by 30x
- Consider the impact of parallel execution
  - What if we used 5 Xscales at 200Mhz?
  - Similar performance as a 1Ghz Xscale, but **1/6th the energy**
    - 5 cores * 1/30th = 1/6th
- And, amortizes background "uncore" energy among cores
- Assumes parallel speedup (a difficult task)
  - Subject to Ahmdal's law

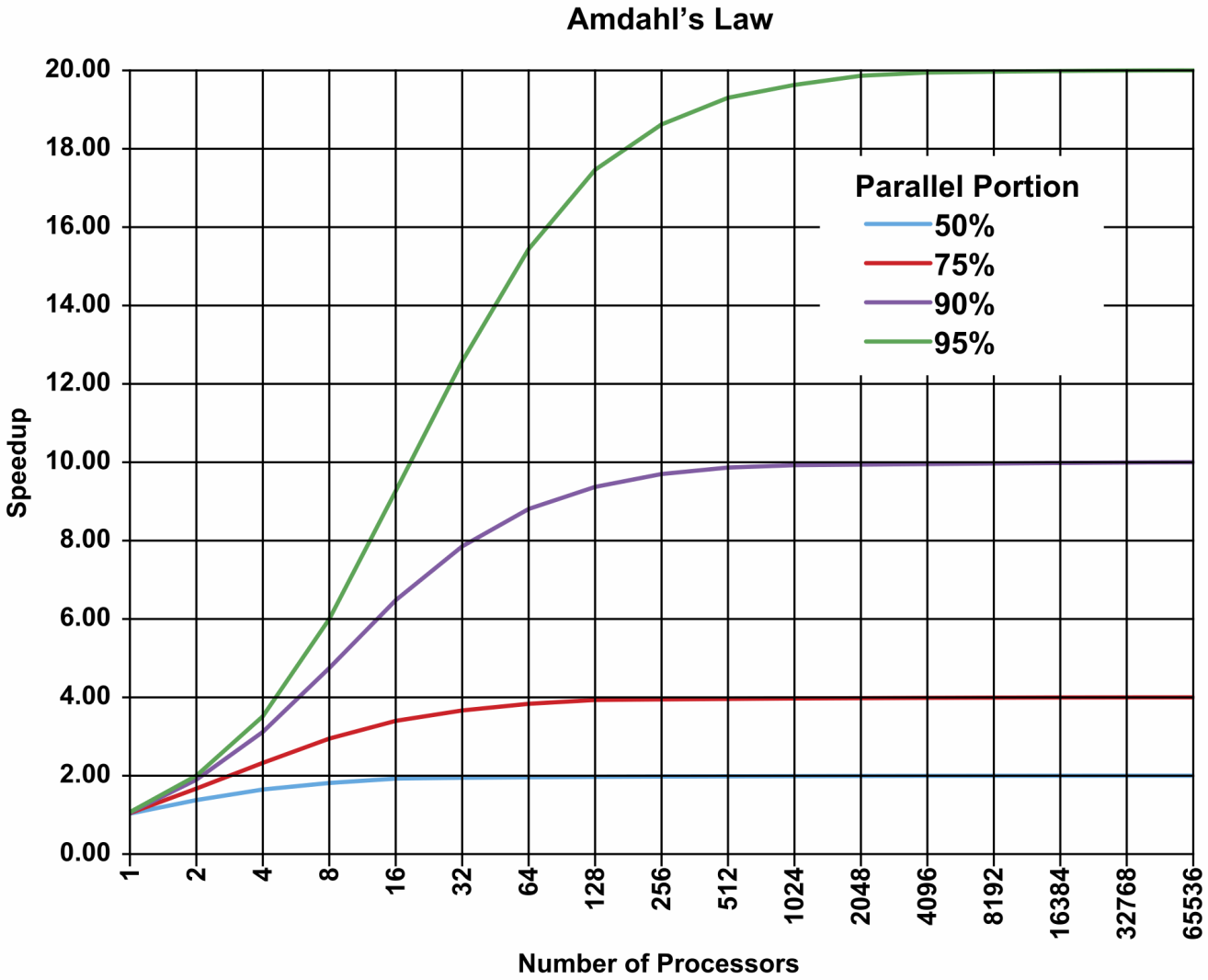
Amdahl's Law

- Restatement of the law of diminishing returns
  - Total speedup limited by non-accelerated piece
  - Analogy: drive to work & park car, walk to building

- Consider a task with a "parallel" and "serial" portion
  - What is the speedup with N cores?
  - Speedup(n, p, s) = (s+p) / (s + (p/n))
    - p is "parallel percentage", s is "serial percentage"
  - What about infinite cores?
    - Speedup(p, s) = (s+p) / s = 1 / s

- Example: can optimize 50% of program A
  - Even a "magic" optimization that makes this 50% disappear…
  - …only yields a 2X speedup

Amdahl's Law Graph



Source: Wikipedia

Threading & The Shared Memory Programming Model

First, Uniprocessor Concurrency

- **A software thread**
  - Private per-thread PC, registers, stack
  - Shared state: memory (globals, heap)
    - actually stack lives in memory so it can be shared, too
  - Threads share the same virtual memory address space
    - A process is like a thread, but with its own address space
  - Languages (Java, C, C++, …) have built-in threading support
- the Operating System (OS) manages threads
  - In single-core system, all threads share one processor
    - Hardware timer interrupt occasionally activates OS
    - Quickly swapping threads gives illusion of concurrent execution
  - Much more in an OS course!

Shared Memory Programming Model

- Programmer explicitly creates multiple threads
- Loads & stores access a single **shared memory**
  - All memory is shared & accessible by all threads
- A thread switch can occur at any time
  - Pre-emptive multithreading by OS
- Common uses:
  - Handling user interaction (GUI programming)
  - Handling I/O latency (send/receive network messages)
  - **Express parallel work via Thread-Level Parallelism**
    - This is our focus!

Shared Memory Model: Interleaving

- Initially: all variables zero (that is, x=0, y=0)

thread 1	thread 2
store 1 → y load x	store 1 → x load y

- What value pairs can be read by the two loads?

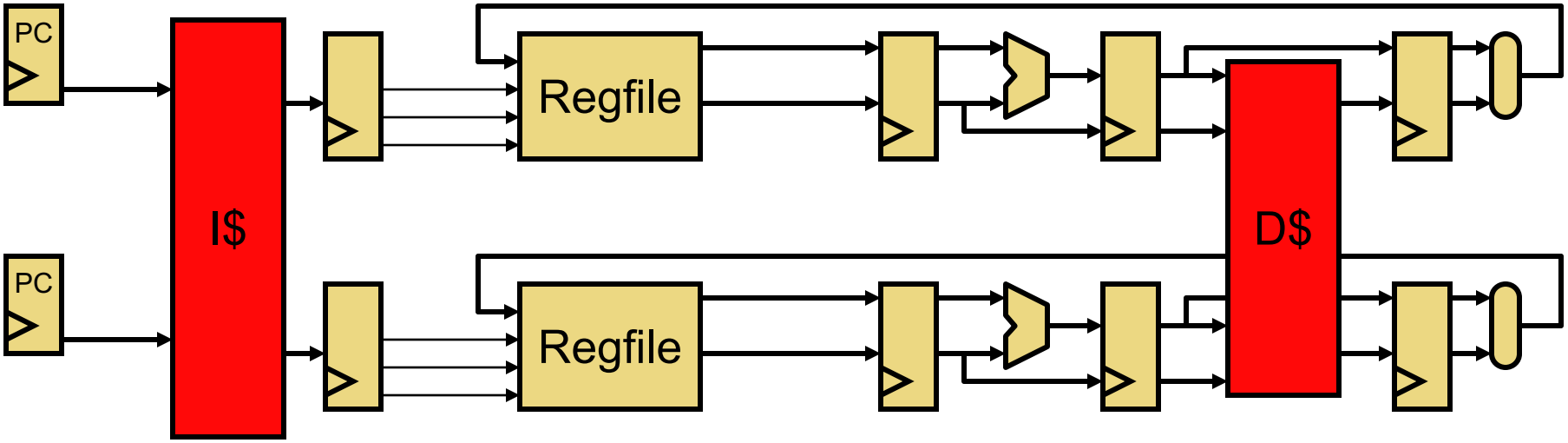
store 1 → y load x store 1 → x load y (x=0, y=1)	store 1 → y store 1 → x load x load y (x=1, y=1)	store 1 → y store 1 → x load y load x (x=1, y=1)
store 1 → x load y store 1 → y load x (x=1, y=0)	store 1 → x store 1 → y load y load x (x=1, y=1)	store 1 → x store 1 → y load x load y (x=1, y=1)

- What about (x=0, y=0)?

Shared Memory Implementations

- **Multiplexed uniprocessor**
  - Runtime system and/or OS pre-empt & swap threads
  - Interleaved, but no parallelism
- **Multiprocessors**
  - Allow private caches, much better performance
  - Same interleaved shared-memory model
- **Hardware multithreading**
  - Tolerate pipeline latencies, higher efficiency
  - Same interleaved shared-memory model
- **All have the same programming model**

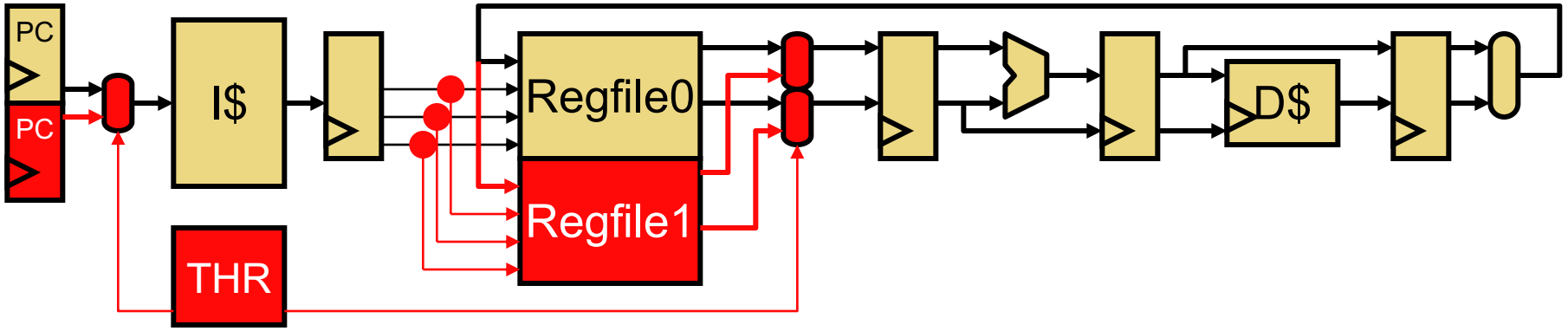
Simplest Multiprocessor



- Replicate entire processor pipeline!
  - Instead of replicating just register file & PC
  - Share the caches for now (but we'll fix this soon)
- Can execute multiple threads
  - Valid implementation of shared memory programming model
  - Operations (loads and stores) are interleaved "at random"
  - Loads returns the value written by most recent store

Hardware Multithreading

- **Not** the same as software multithreading!
- Each **hardware thread** can run one sw thread
  - threads can come from same or different processes



- **Hardware Multithreading (MT)**
  - Multiple hardware threads dynamically share a single pipeline
  - Replicate only per-thread structures: program counter & registers
  - Hardware interleaves instructions

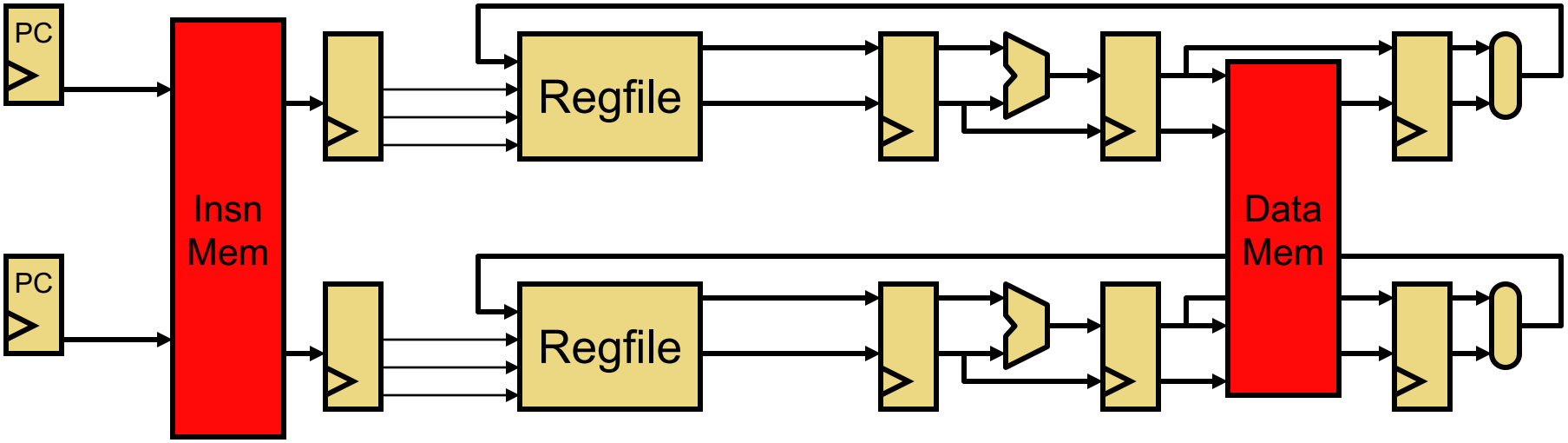
Hardware Multithreading

- Why use hw multithreading?
  - + **Multithreading improves utilization and throughput**
    - Single programs underutilize pipeline (e.g., cache miss)
    - allow insns from different hw threads in pipeline at once
  - **Multithreading does not improve single-thread performance**
    - Individual threads may run slower
  - **Coarse-grain MT**: switch on cache misses: why?
  - **Simultaneous MT**: fine-grained switching
    - Intel calls this *hyperthreading*



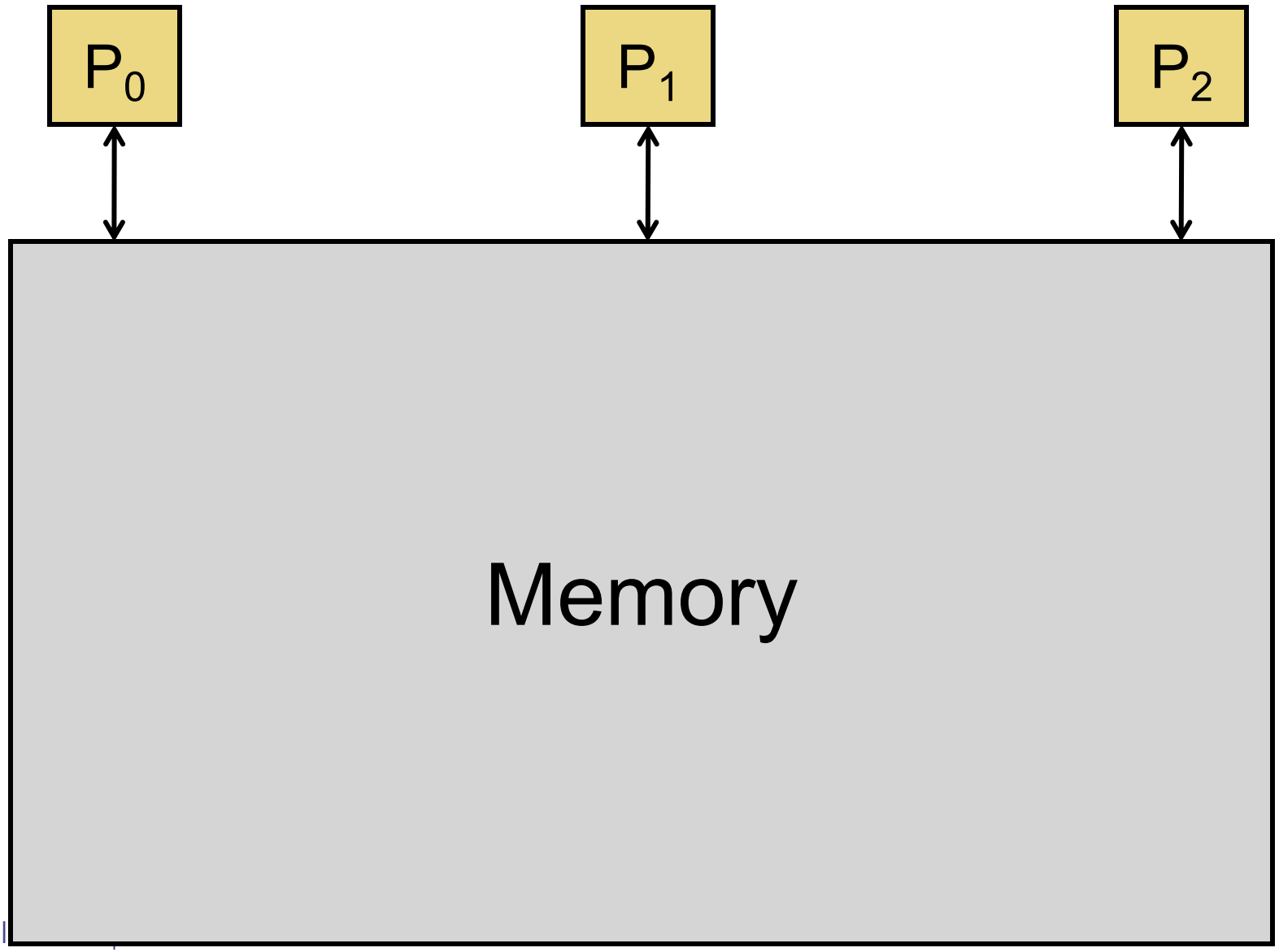
Cache Coherence

Recall: Simplest Multiprocessor

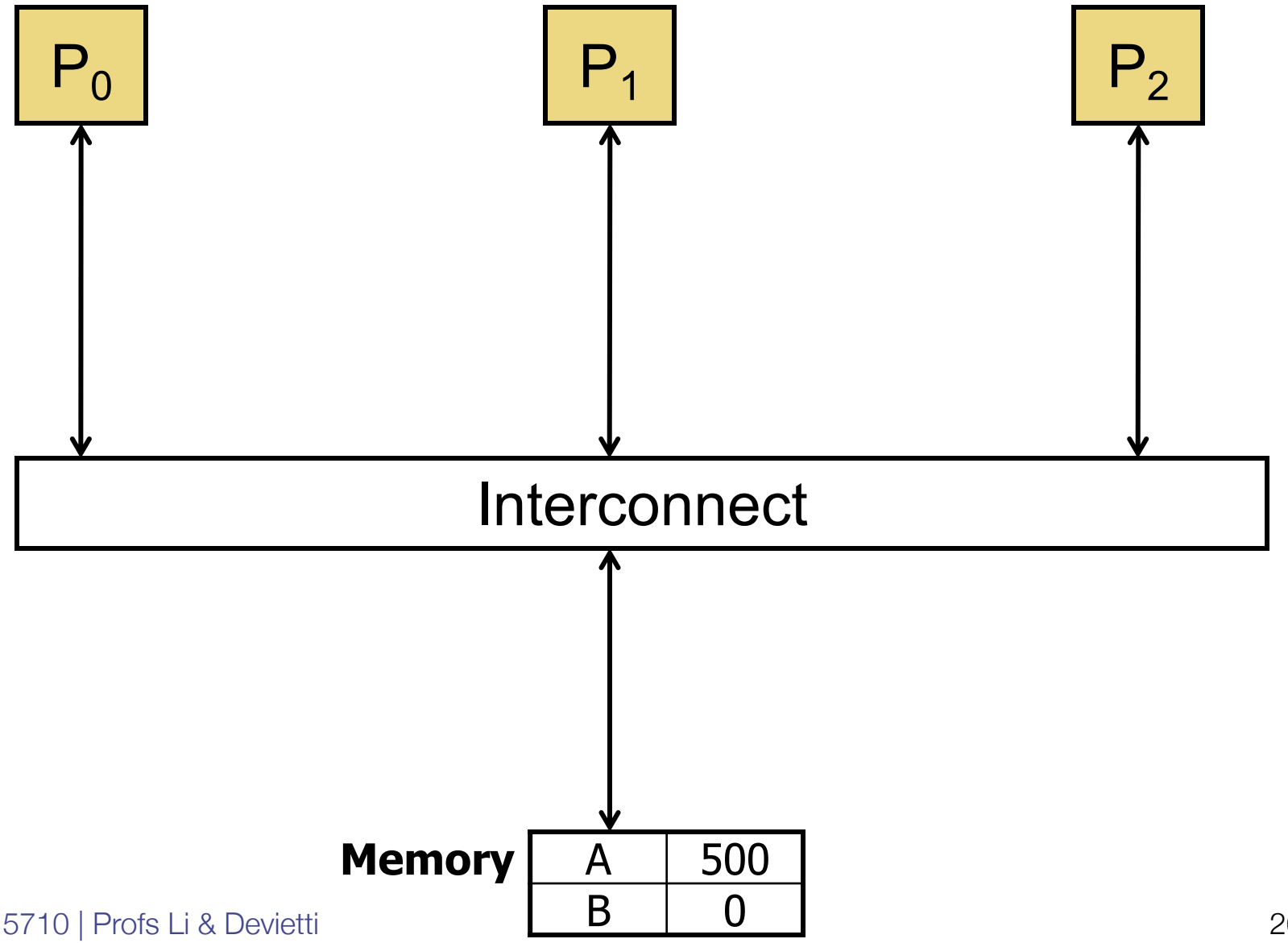


- Sharing L1 caches is slow
  - Hurts bandwidth and latency
- Solution: use per-processor ("private") caches
  - Coordinate them with a **Cache Coherence Protocol**
- Must still provide shared-memory invariant:
  - **Loads read the value written by the most recent store**

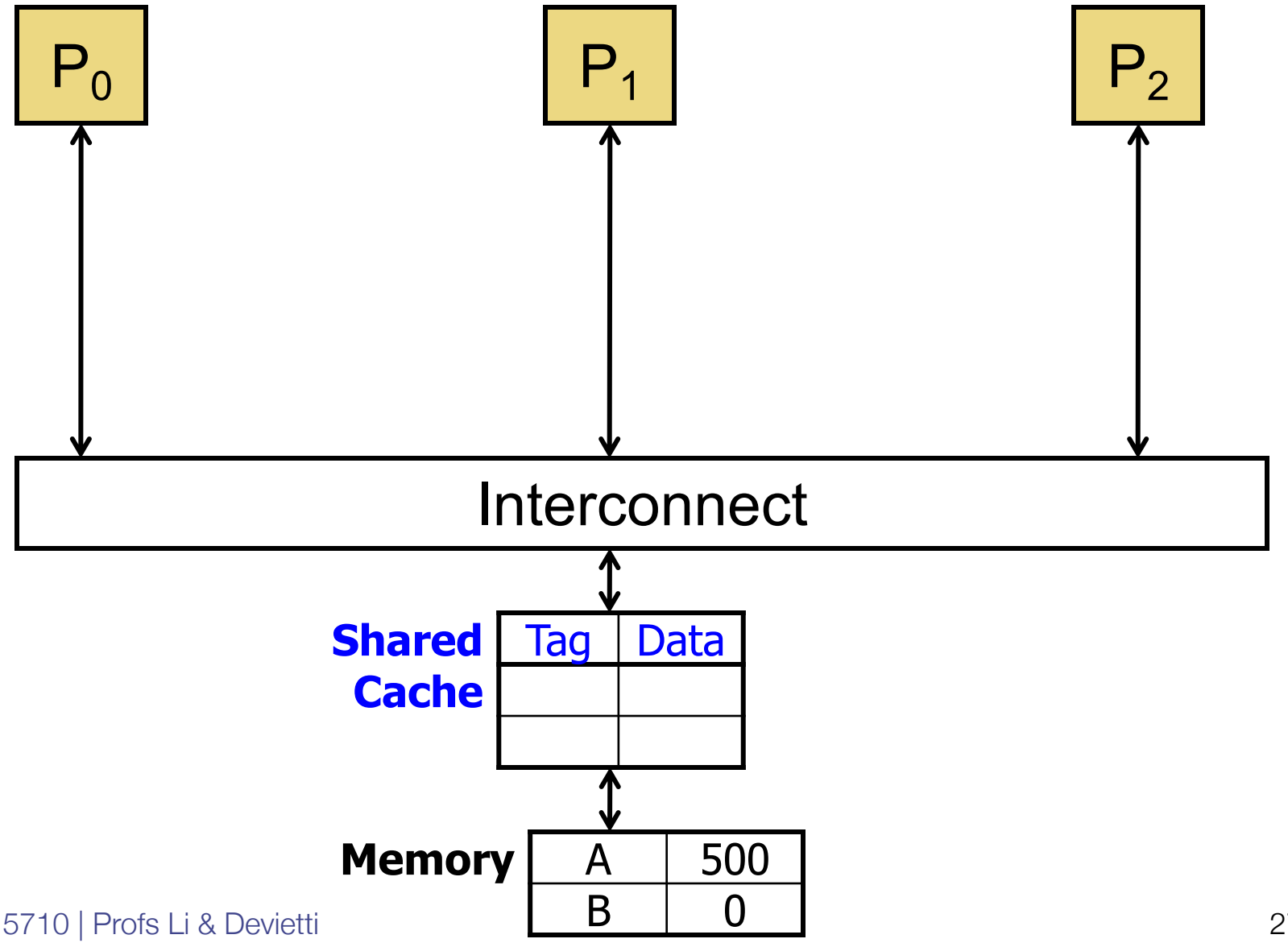
Starting point: no caches



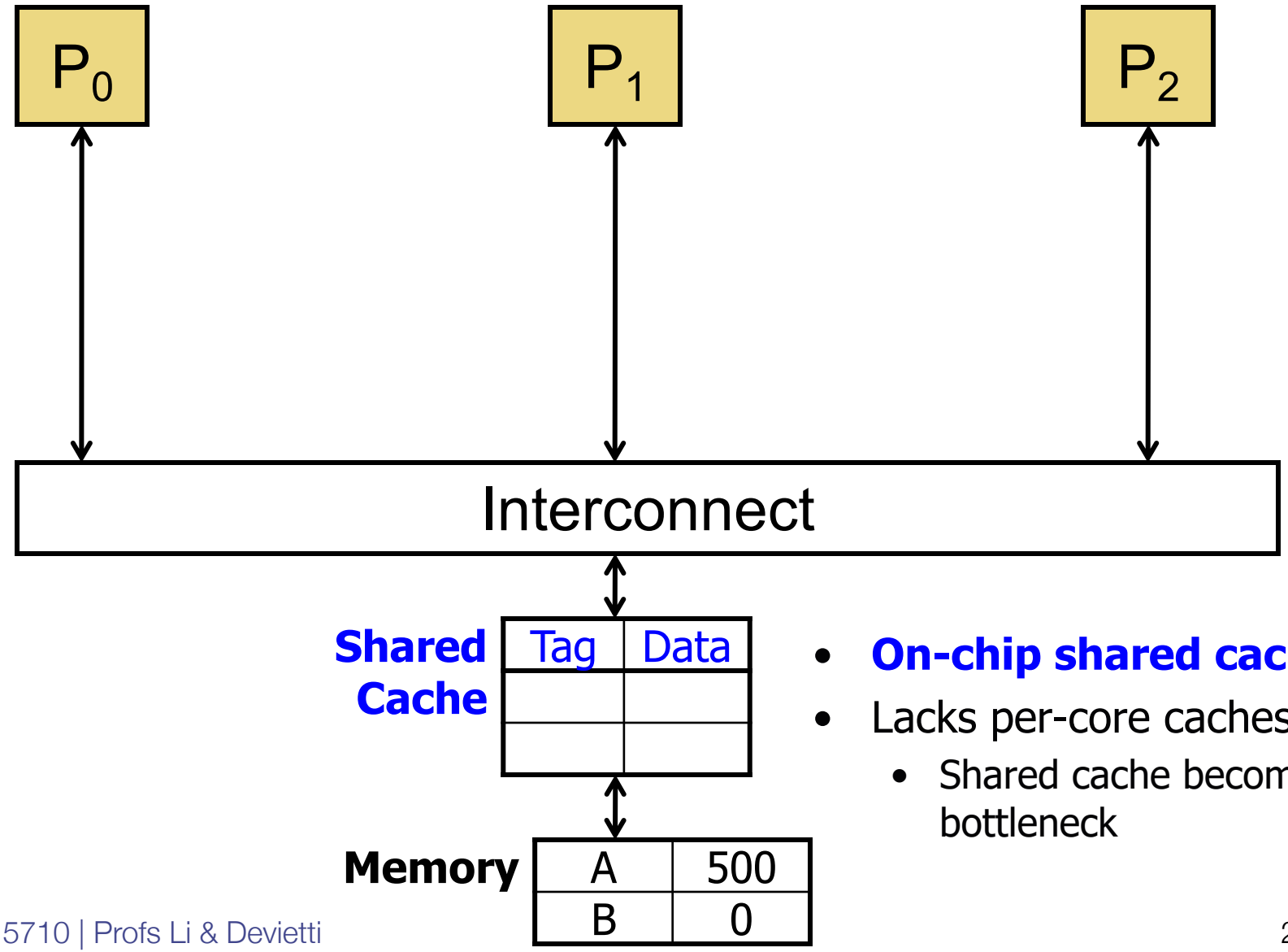
Starting point: no caches



Add a Shared Cache

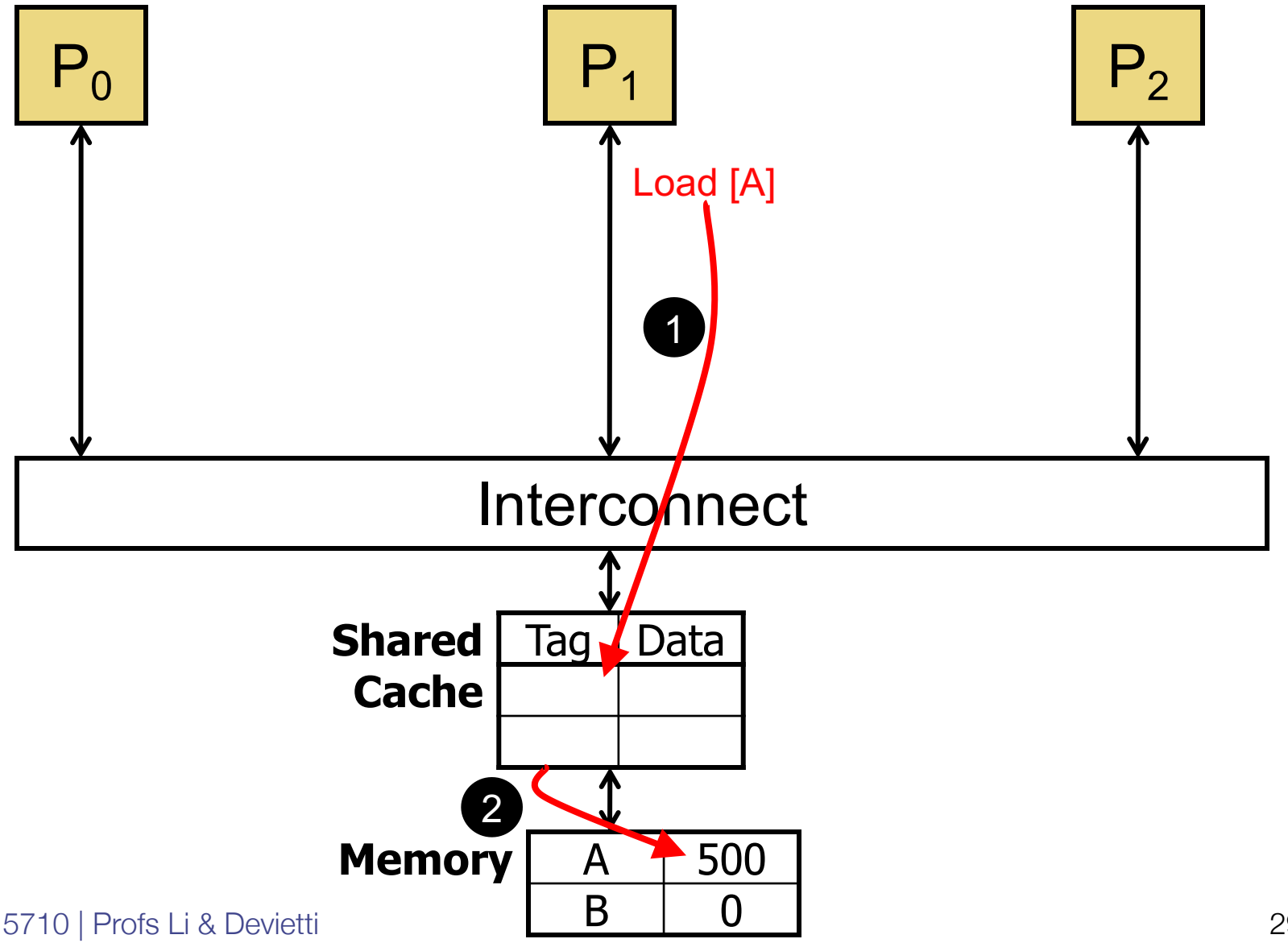


Shared Cache Implementation

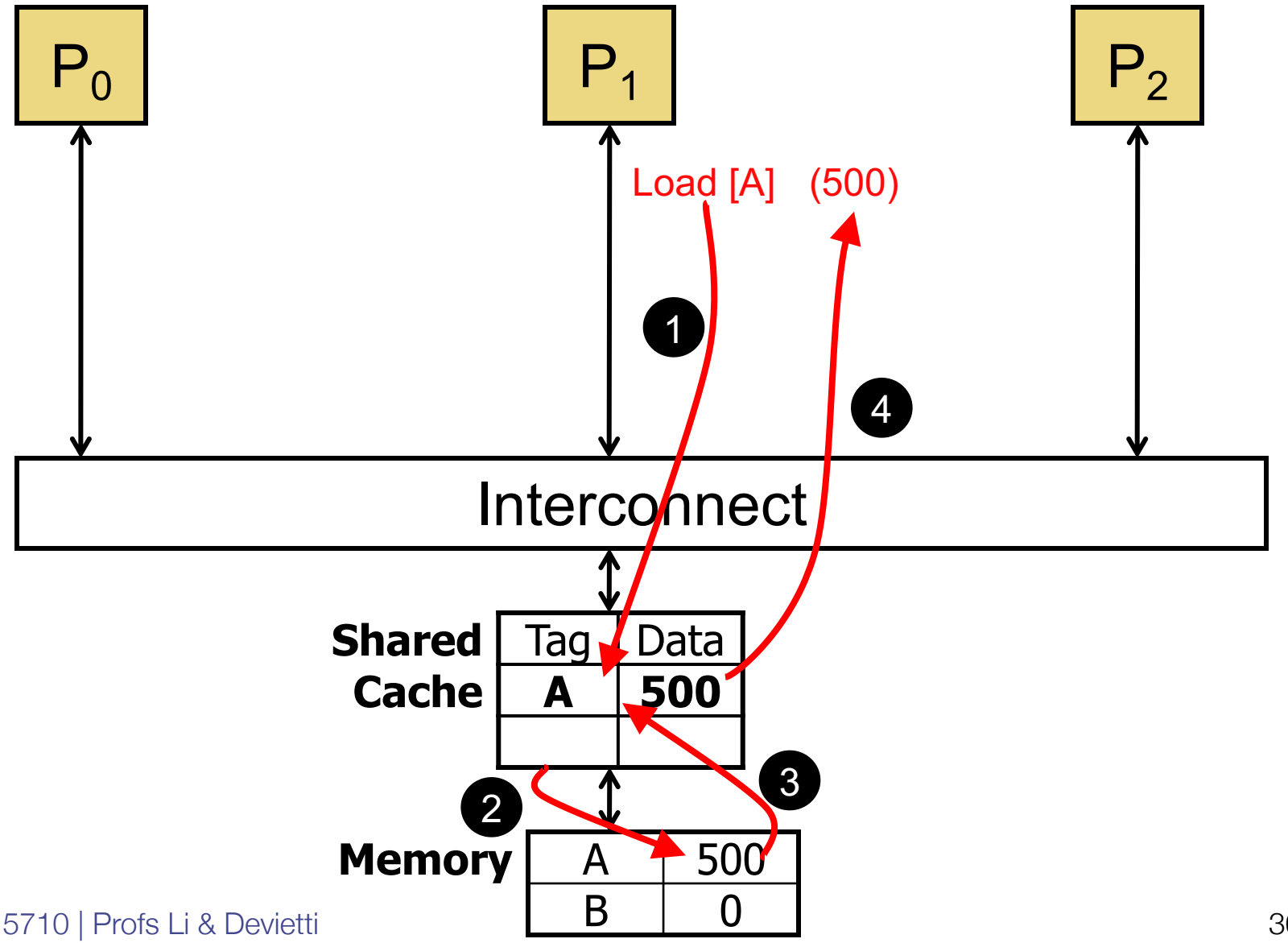


- **On-chip shared cache**
- Lacks per-core caches
  - Shared cache becomes bottleneck

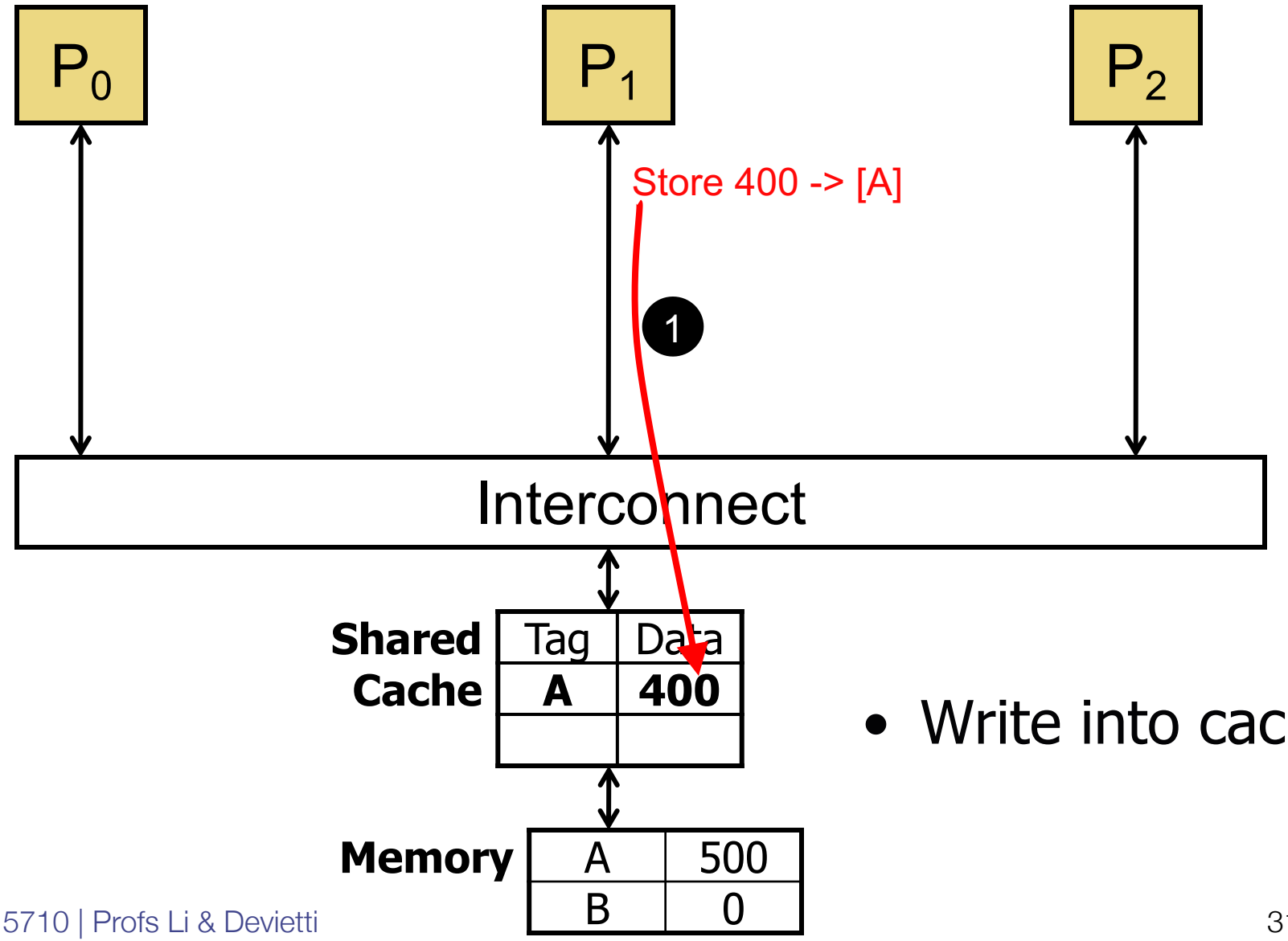
Shared Cache Implementation



Shared Cache Implementation

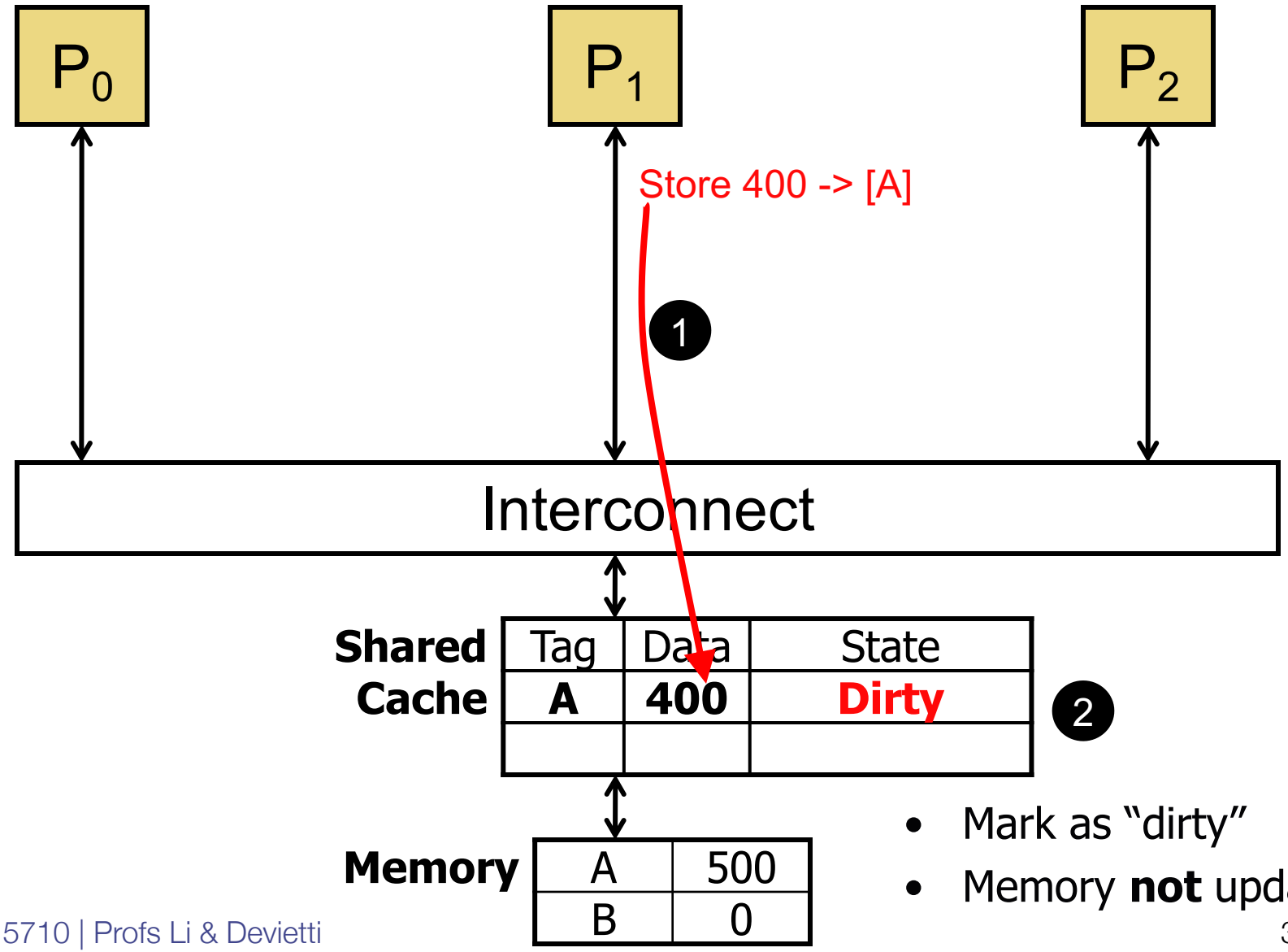


Shared Cache Implementation



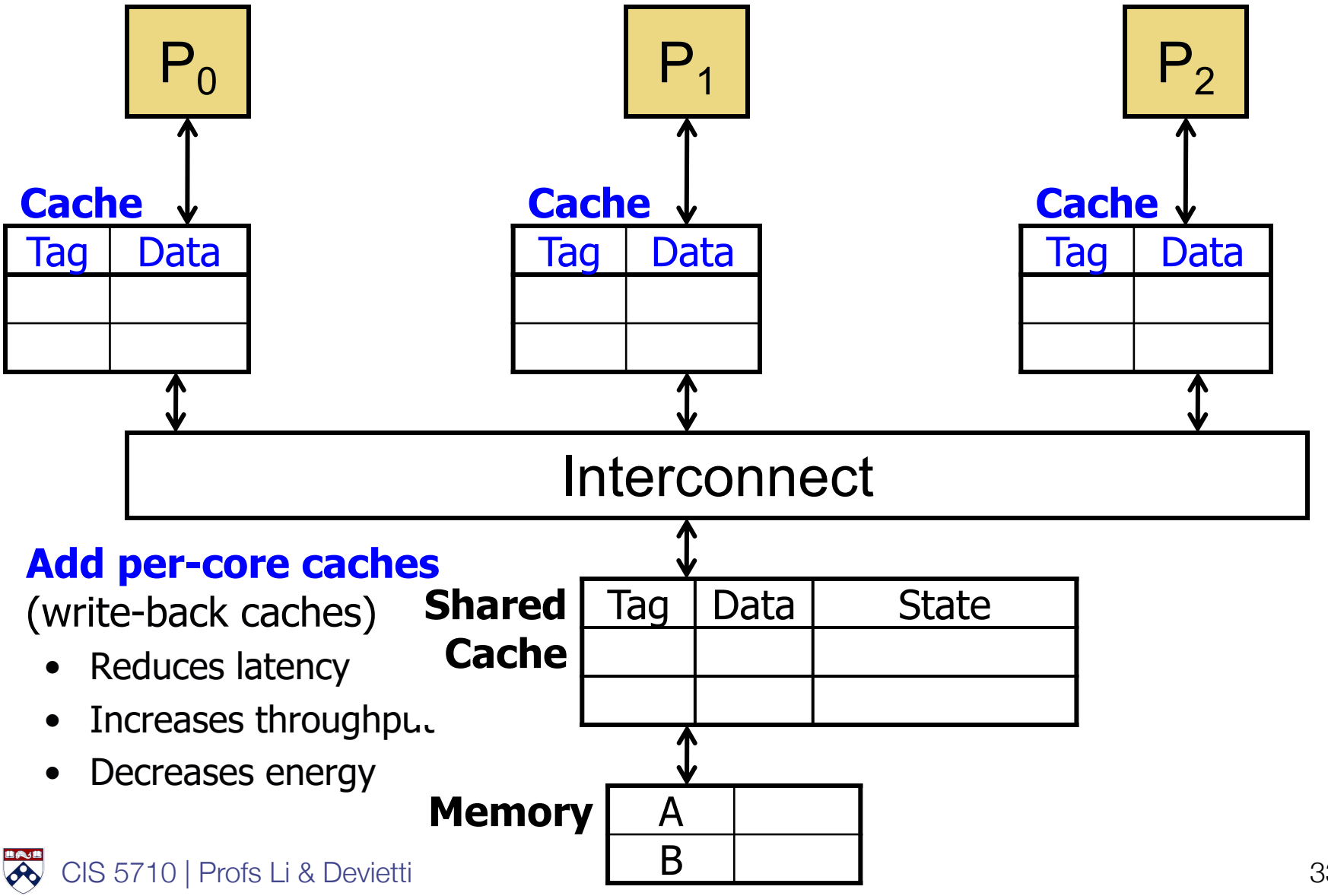
- Write into cache

Shared Cache Implementation



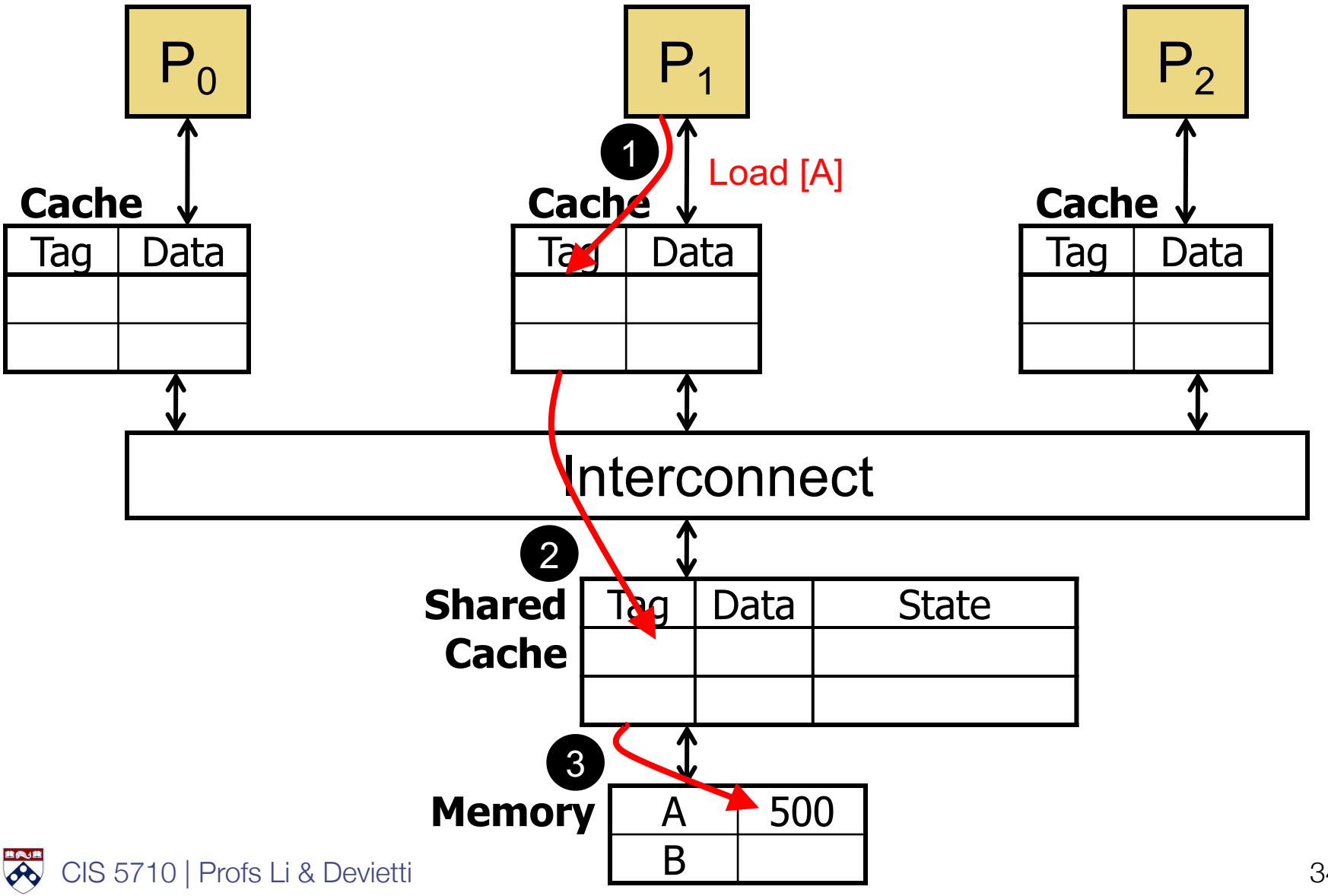
- Mark as "dirty"
- Memory **not** updated

Adding Private Caches

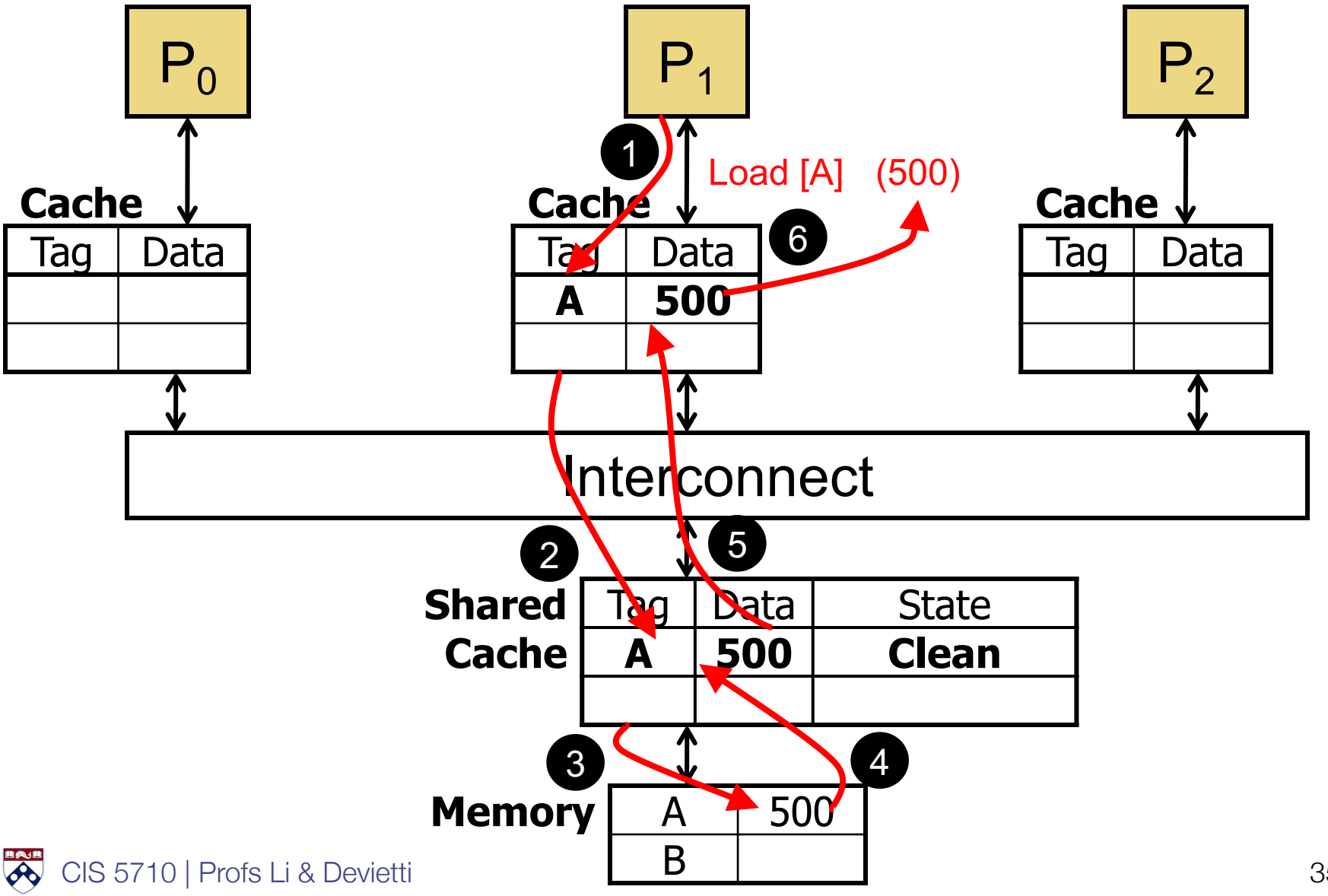


- **Add per-core caches** (write-back caches)
  - Reduces latency
  - Increases throughput
  - Decreases energy

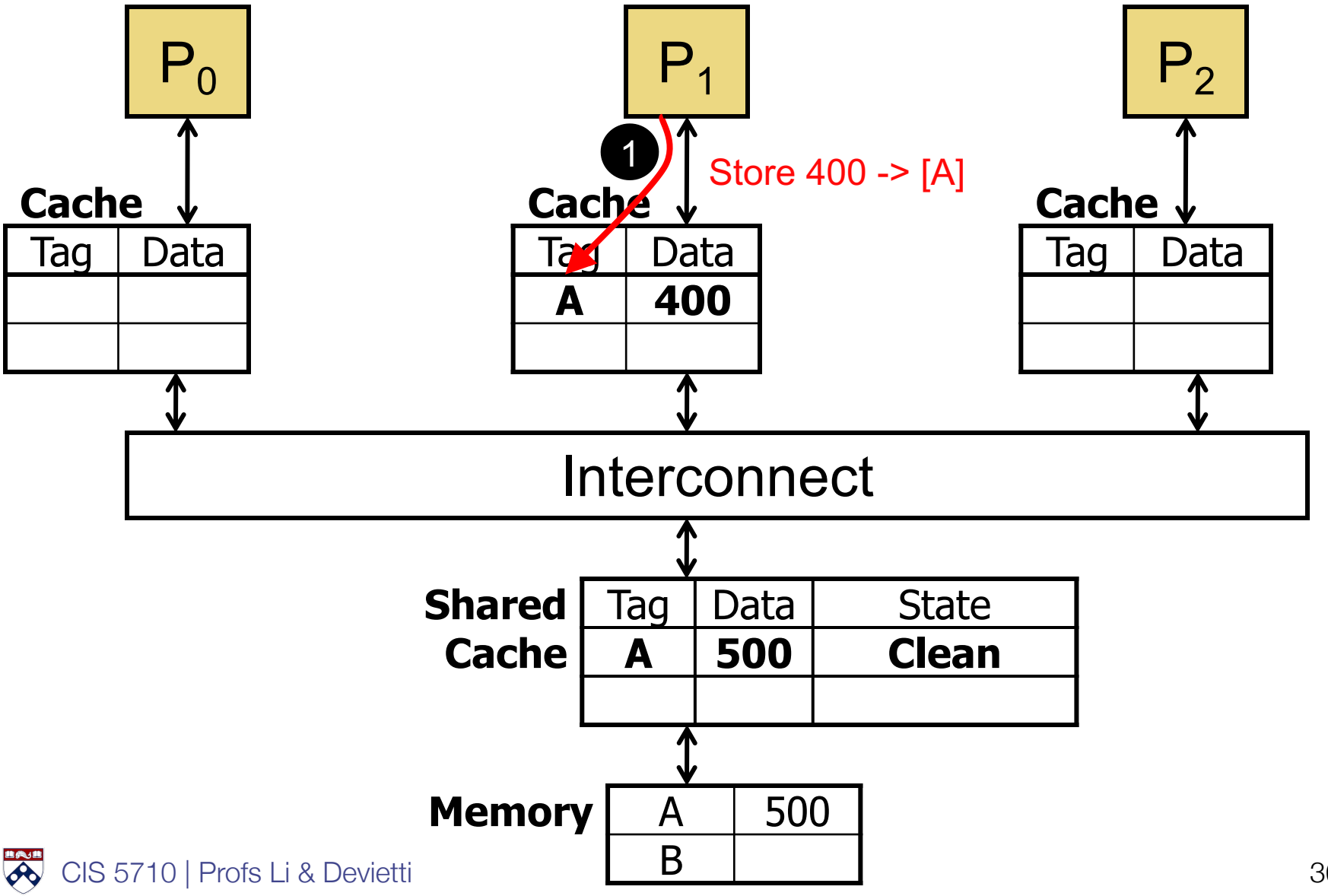
Adding Private Caches



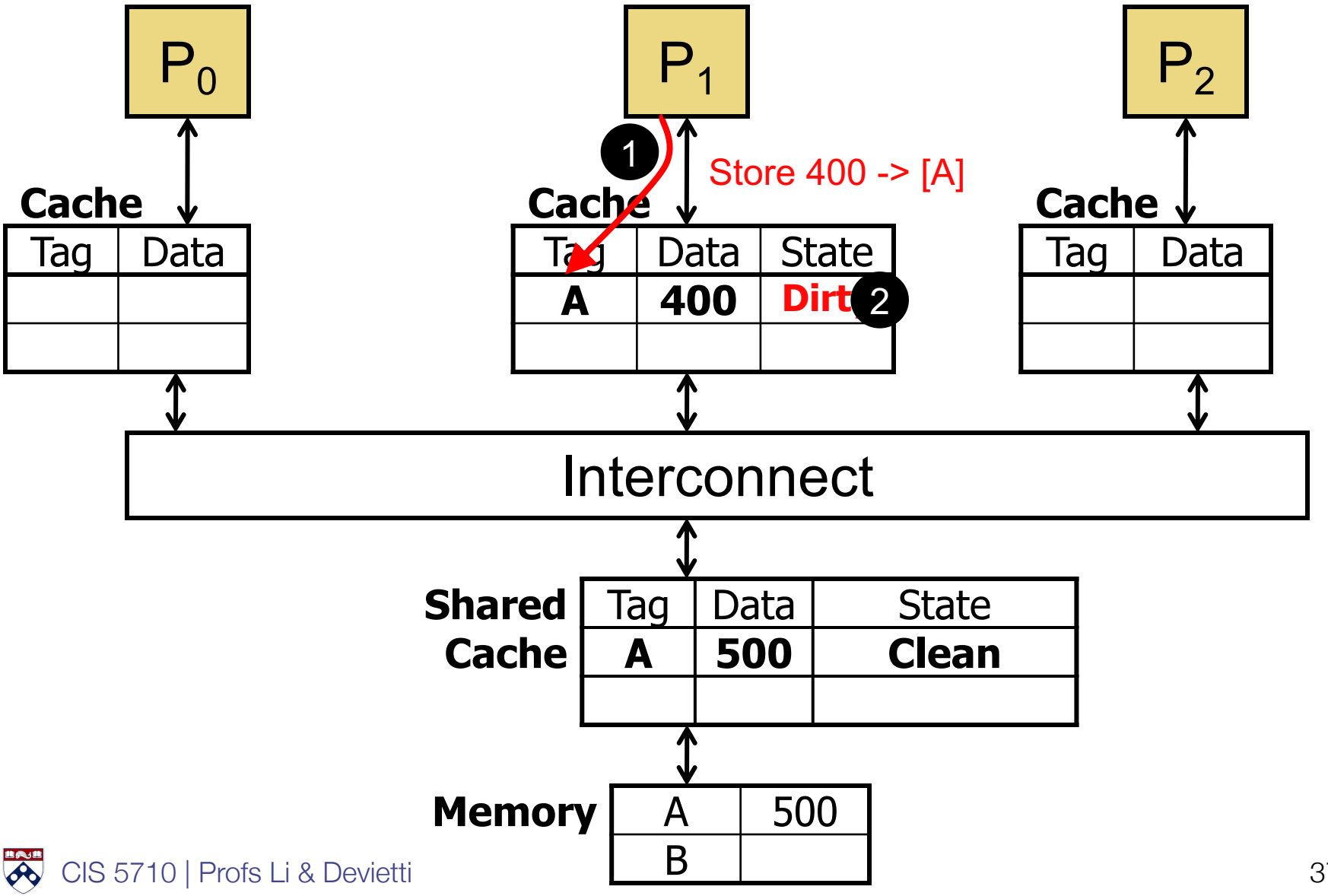
Adding Private Caches



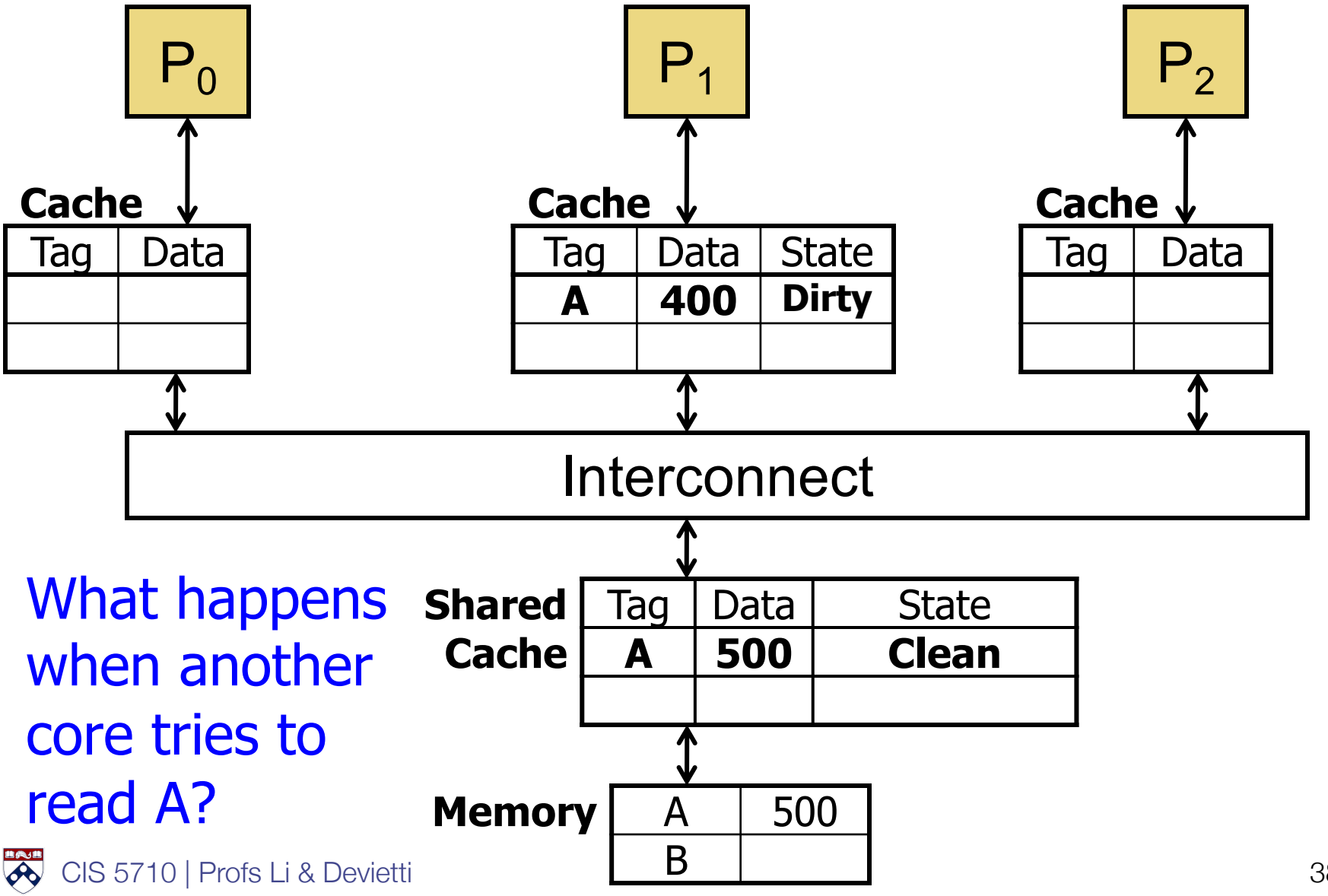
Adding Private Caches



Adding Private Caches

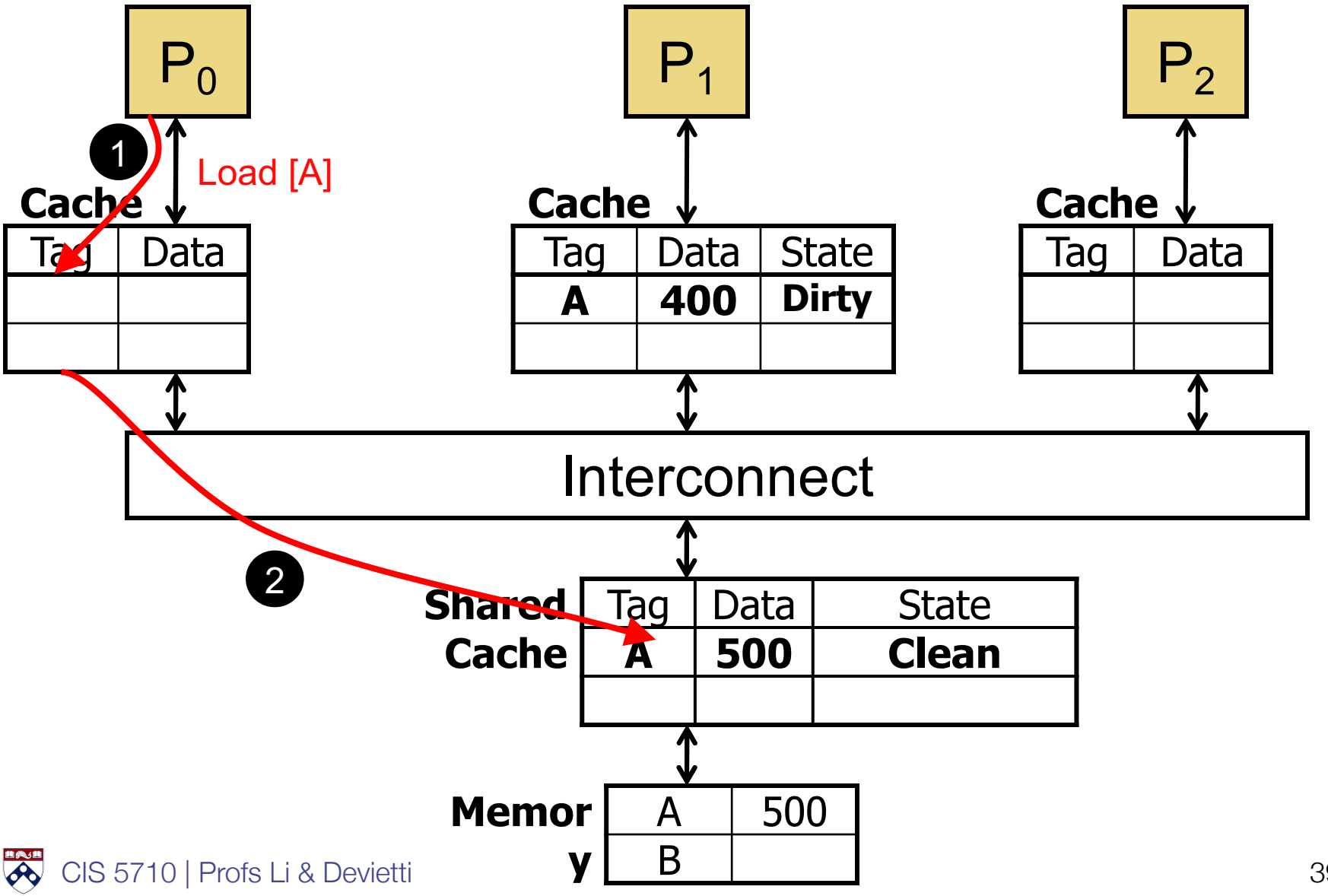


Private Cache Problem

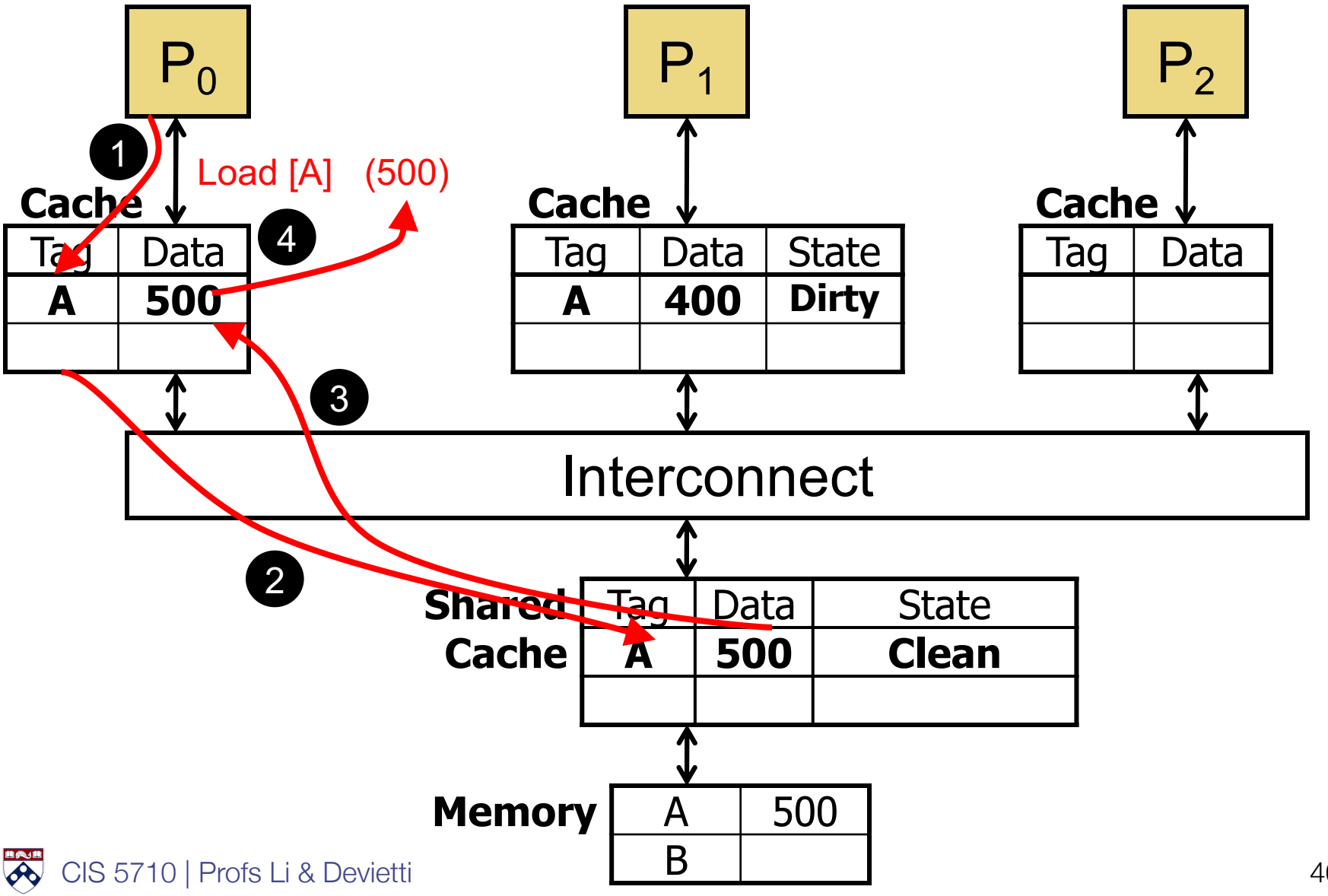


- What happens when another core tries to read A?

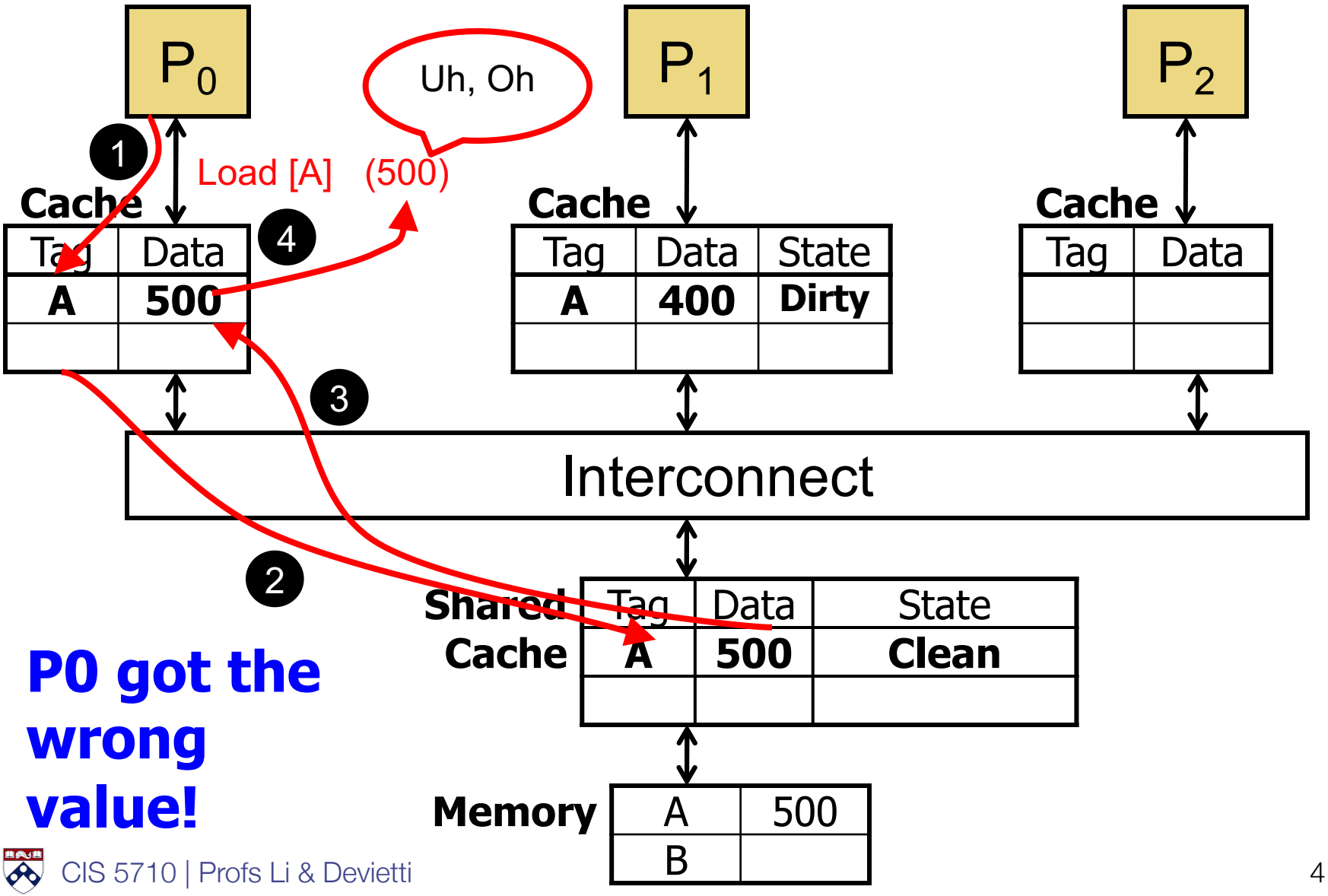
Private Cache Problem



Private Cache Problem



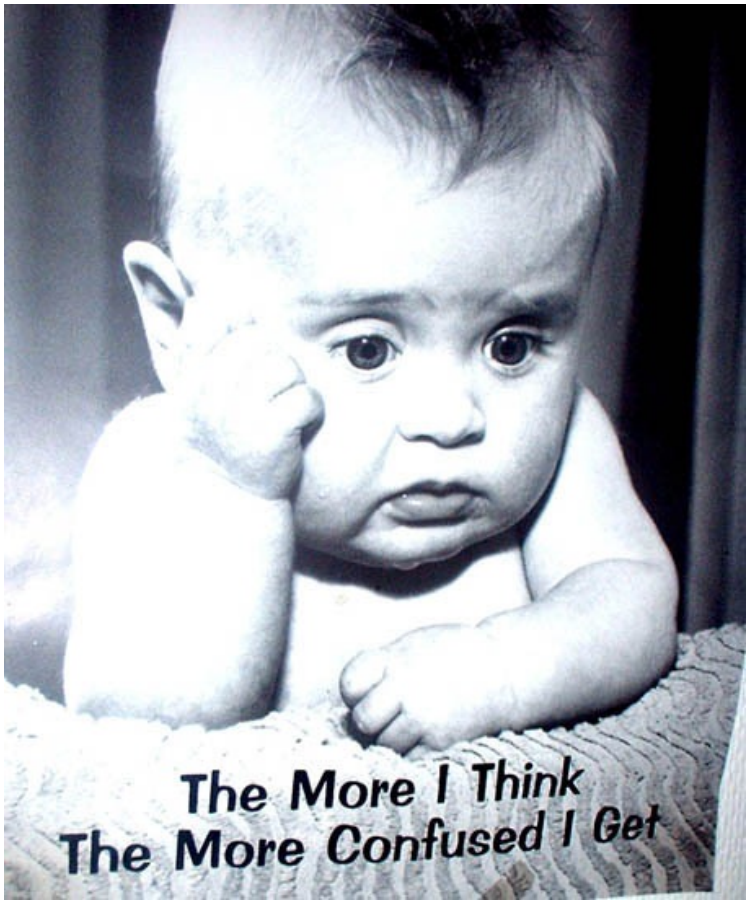
Private Cache Problem



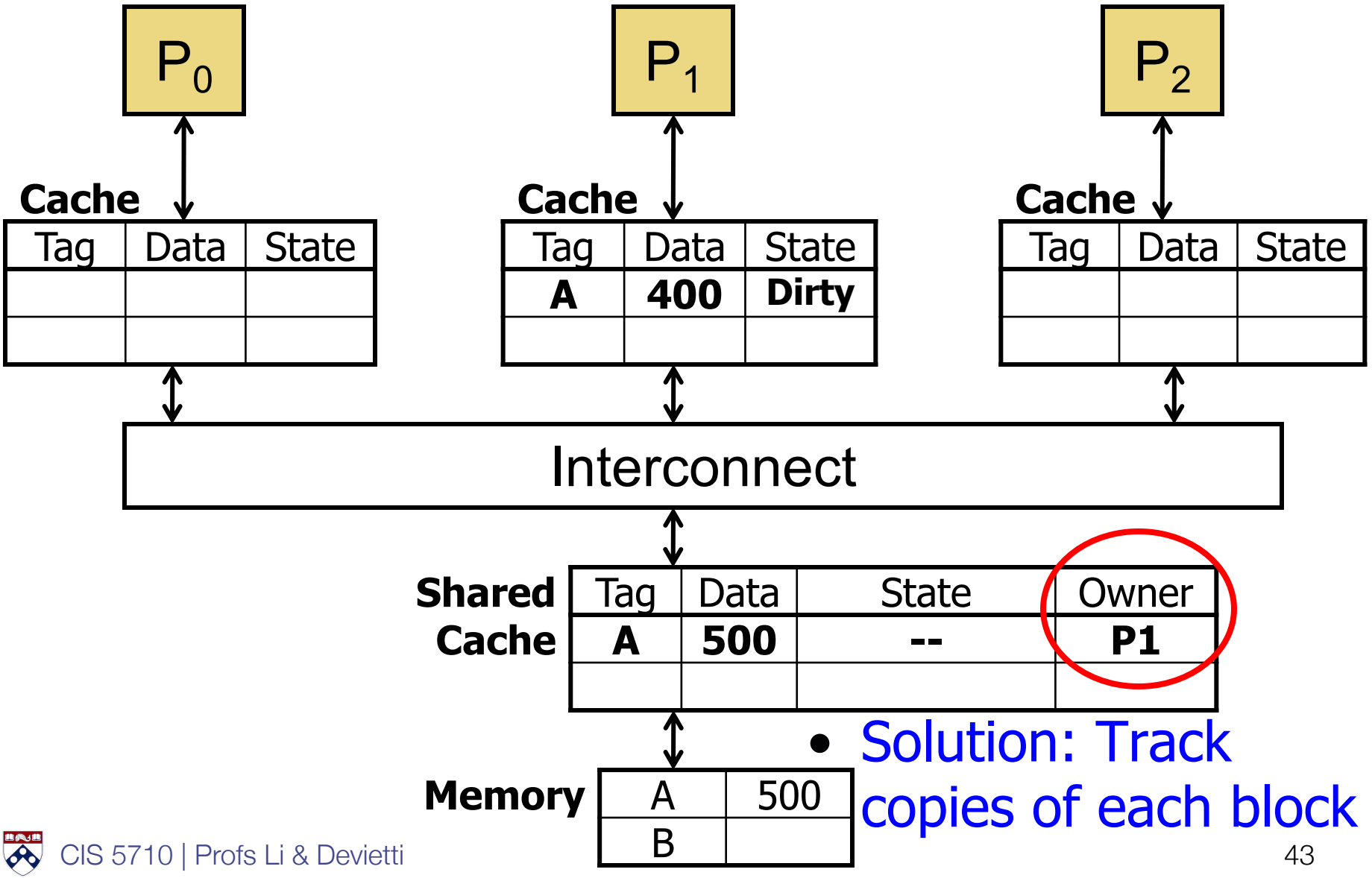
- **P0 got the wrong value!**

Cache Coherence: Who bears the brunt?

- Caches are supposed to be invisible to the programmer!

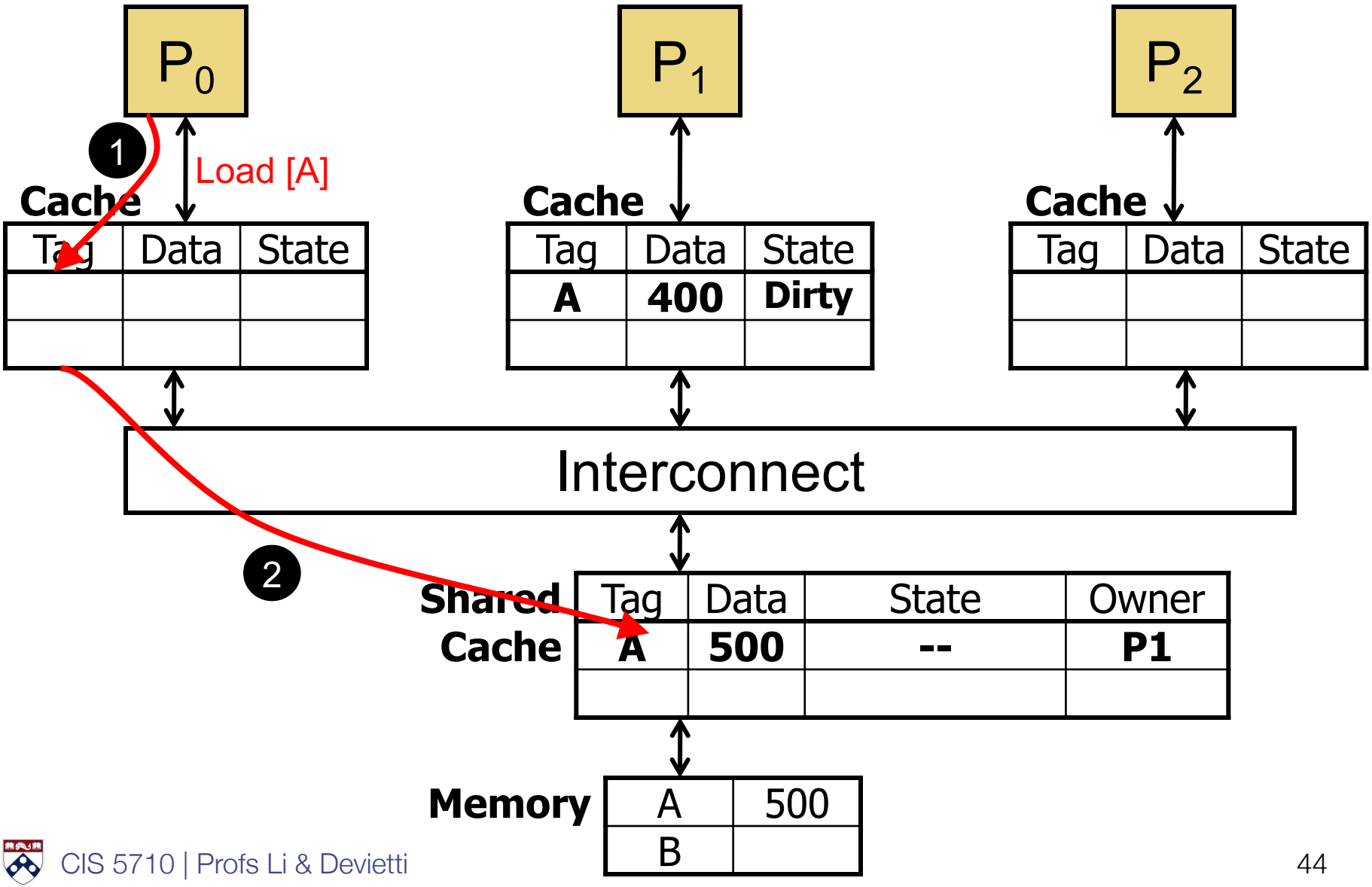


Fix Problem by Tracking Sharers

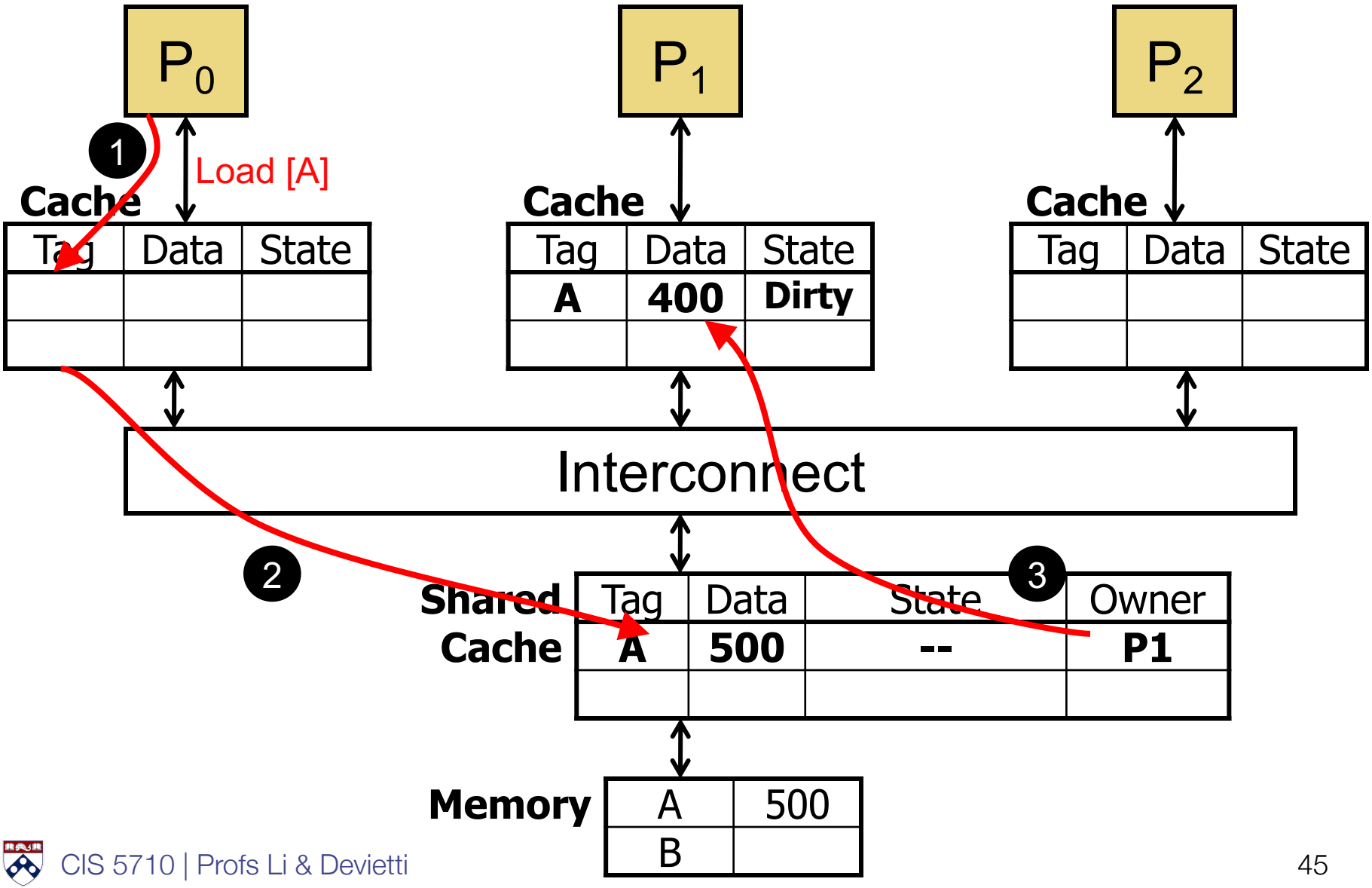


- Solution: Track copies of each block

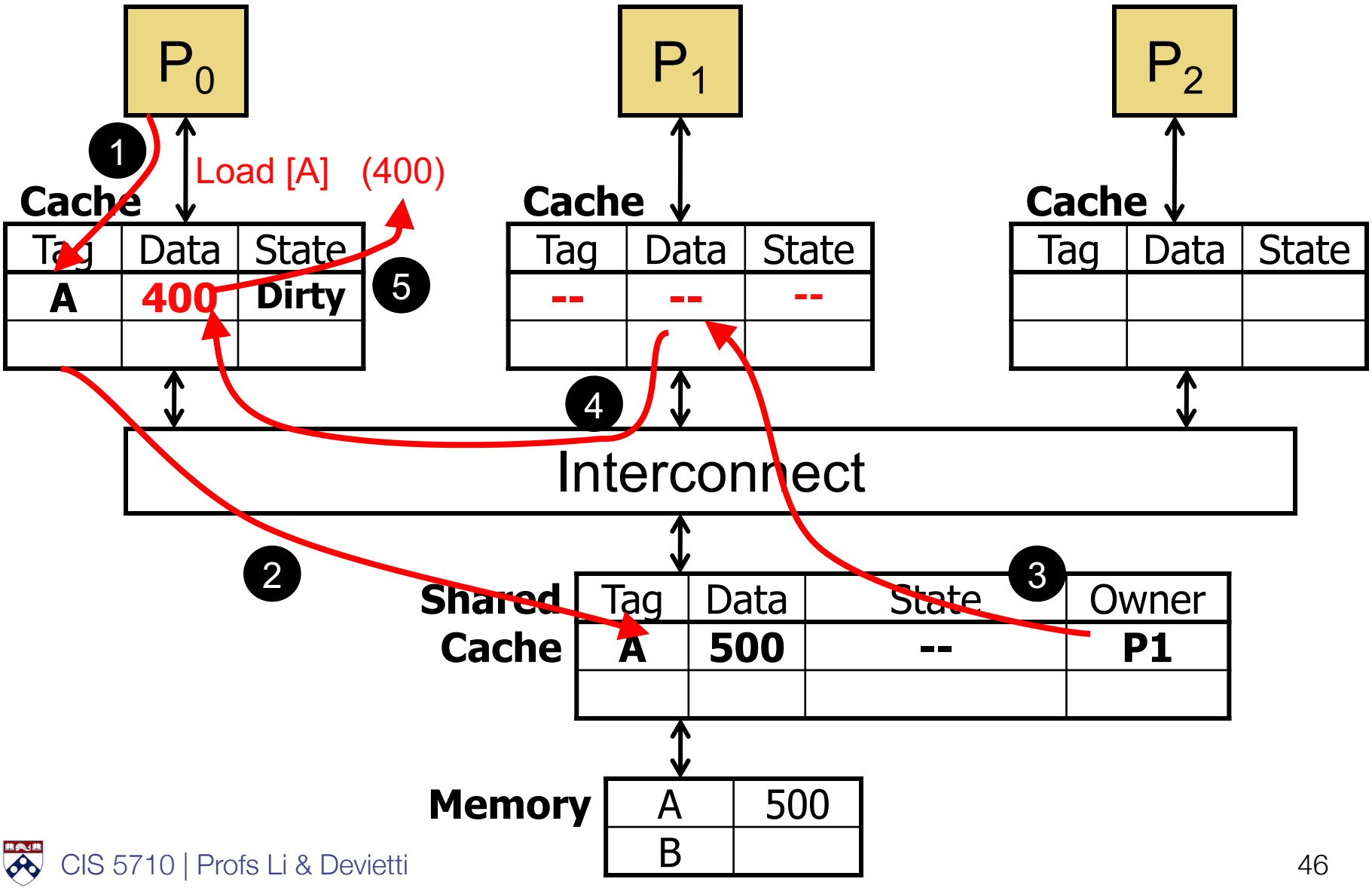
Use tracking info to invalidate



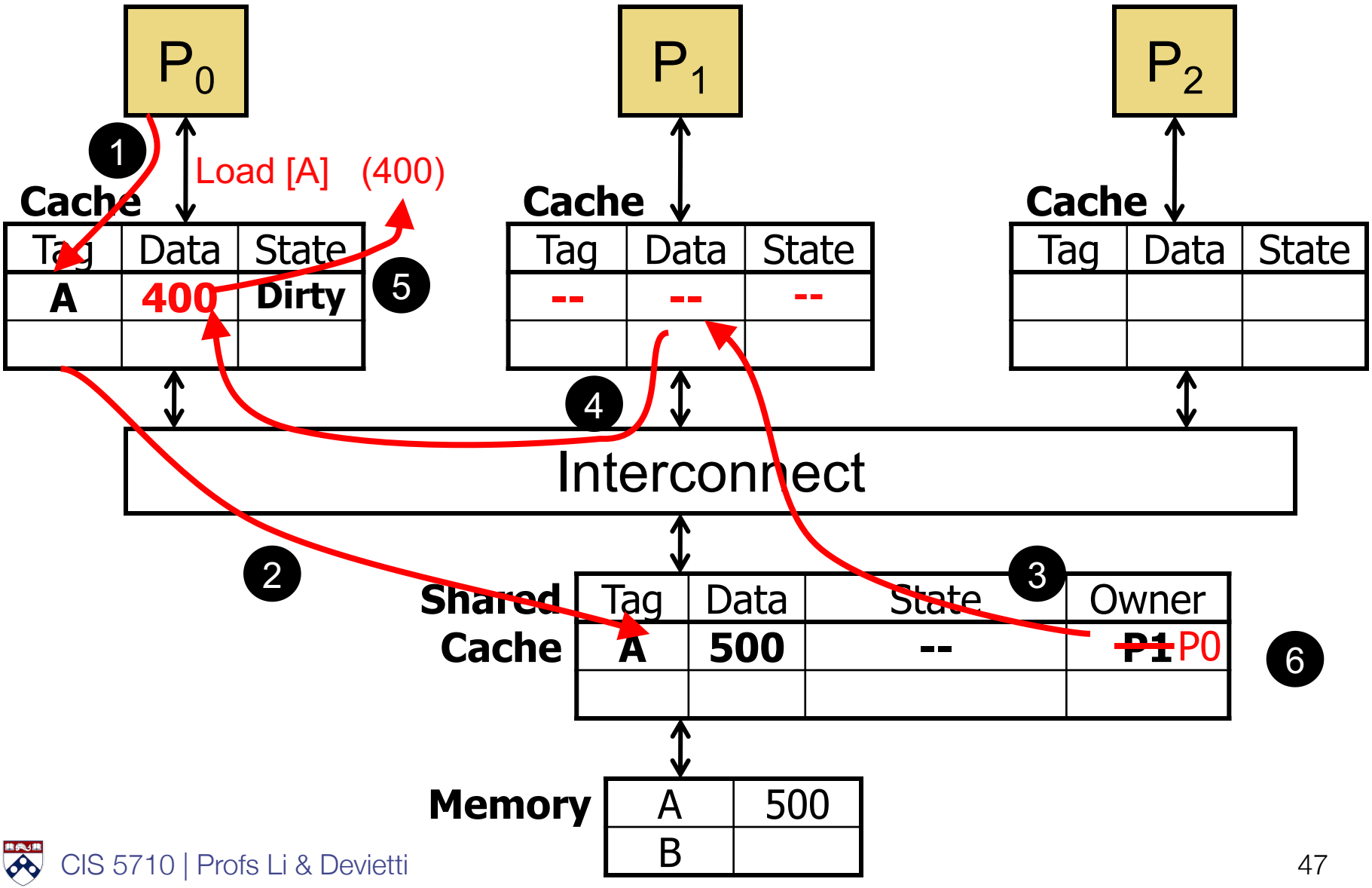
Use tracking info to invalidate



Use tracking info to invalidate



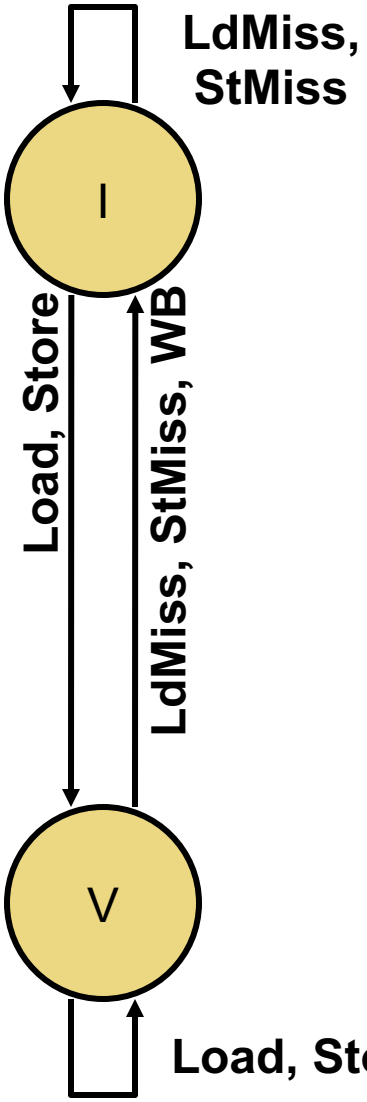
Use tracking info to invalidate



Valid/Invalid cache coherence

- To enforce the shared memory invariant…
  - *Loads read the value written by the most recent store*
- …we enforce the invariant
  - **At most one valid copy of a block**
  - Simplest form is a **two-state Valid/Invalid protocol**
  - If a core wants a copy, must find and invalidate it
- On a cache miss, how is the valid copy found?
  - **Snooping**: broadcast to all, whoever has it responds
  - **Directory**: track sharers explicitly
- **Problem**: multiple read-only copies not allowed
  - Makes read-only data, code, etc. slow

VI (MI) Coherence Protocol



- **VI (valid-invalid) protocol**
  - Two states (per block in cache)
    - **V (valid)**: have block
    - **I (invalid)**: don't have block

    + Can implement with valid bit
- Protocol diagram (at left)
  - Summary
    - If anyone wants to read/write block
    - Give it up: transition to **I** state
    - Write-back if your own copy is dirty

VI Protocol State Transition Table

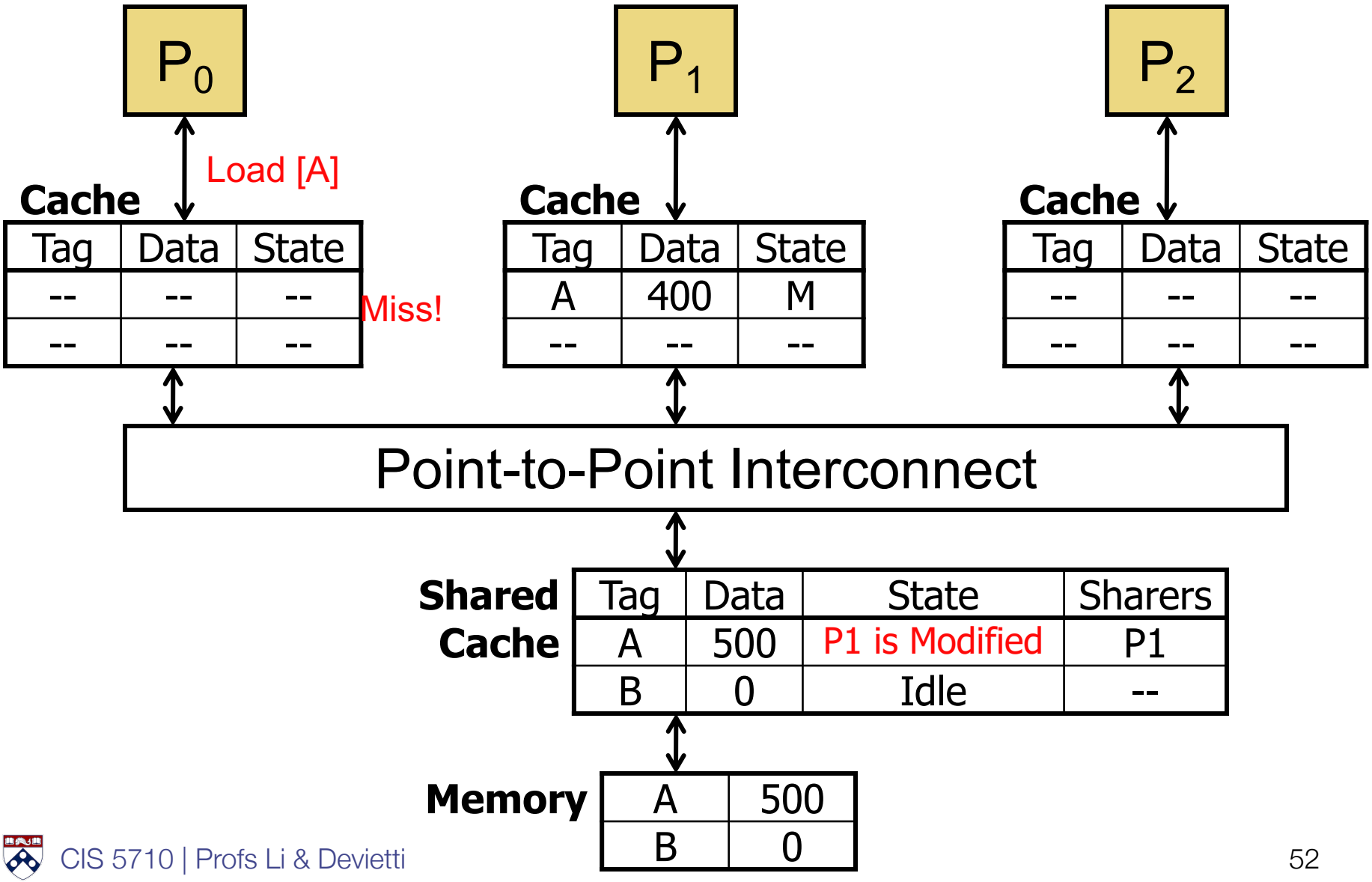
State	*This Processor*		*Other Processor*	
	Load	Store	Load Miss	Store Miss
Invalid (I)	Load Miss ➔ V	Store Miss ➔ V	---	---
Valid (V)	Hit	Hit	Send Data ➔ I	Send Data ➔ I

- Rows are "states"
  - I vs V
- Columns are "events"
  - Writeback events not shown
- Memory controller not shown
  - **Memory sends data when no processor responds**

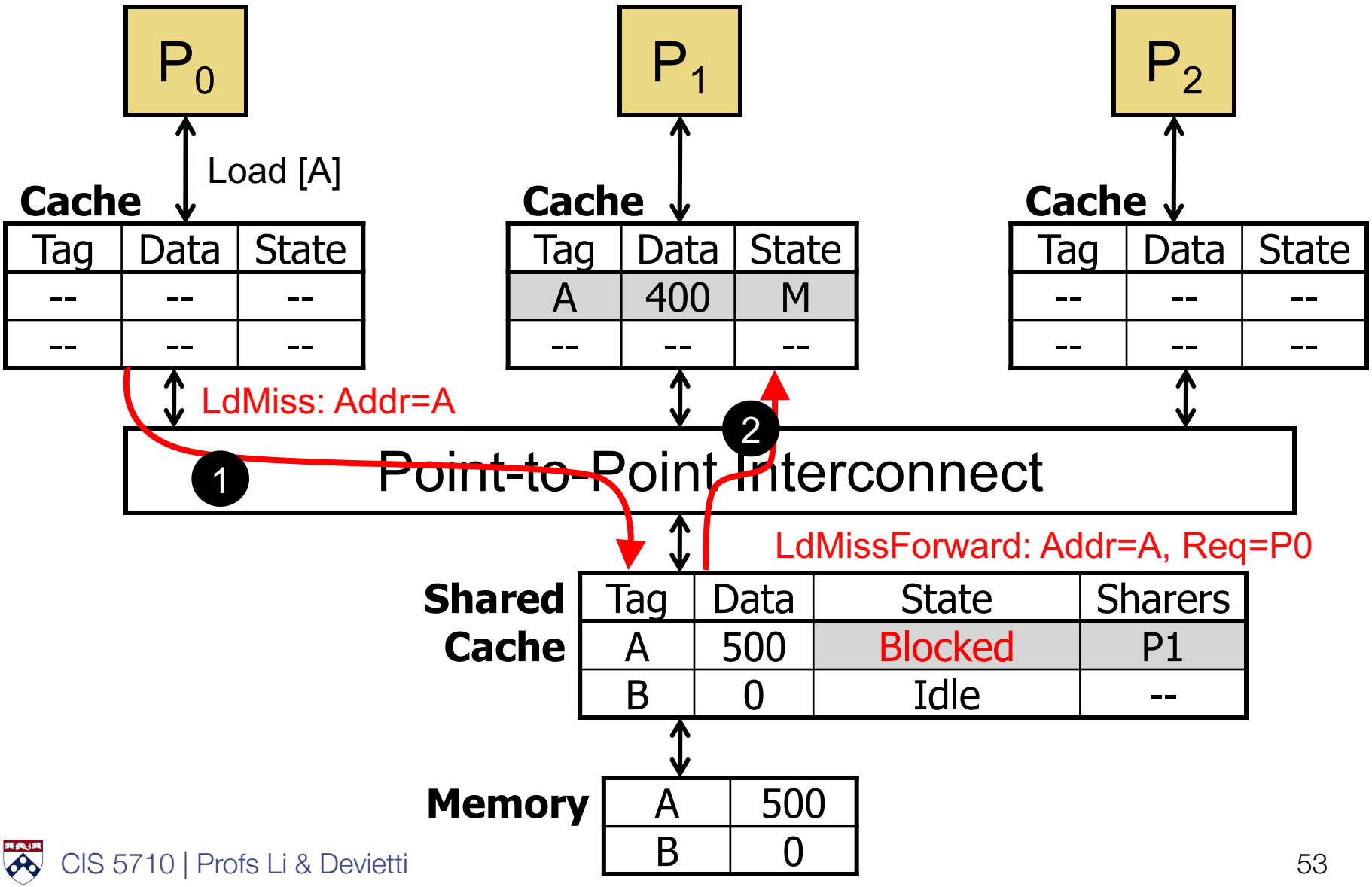
MSI Cache Coherence Protocol

- Add an extra state to track read-only lines
  - **Multiple read-only copies, or a single read/write copy**
  - **Modified (M): read/write permission**
  - **Shared (S): read-only permission**
  - **Invalid (I): no permission**
- Also track a **Sharer bit vector** in shared cache
  - One bit per core; tracks all shared copies of a block
  - When a write occurs, must invalidate all readers
- Allows for many readers…
  - …while still enforcing shared memory invariant (*Loads read the value written by the most recent store*)

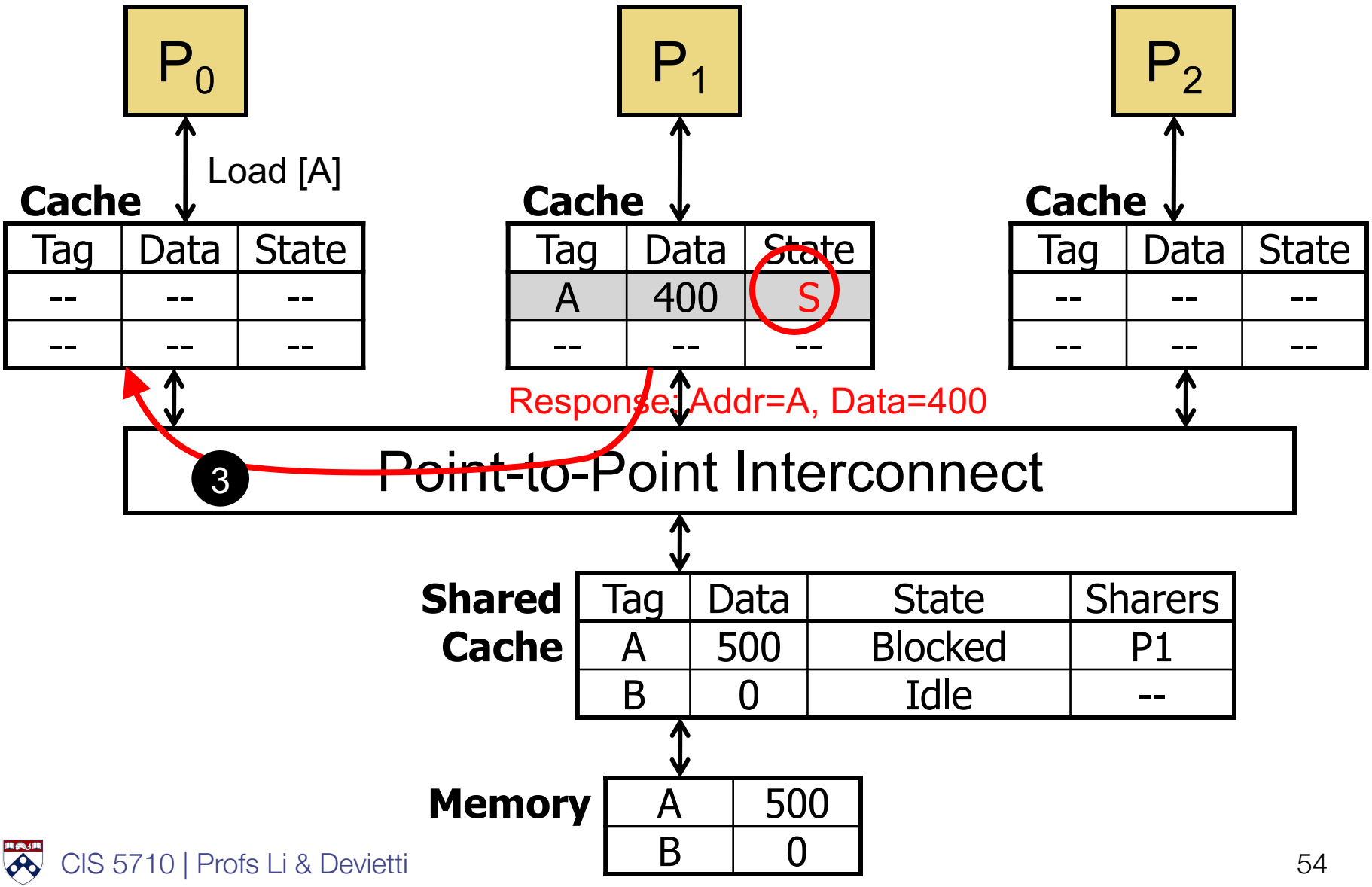
MSI Coherence Example: Step #1



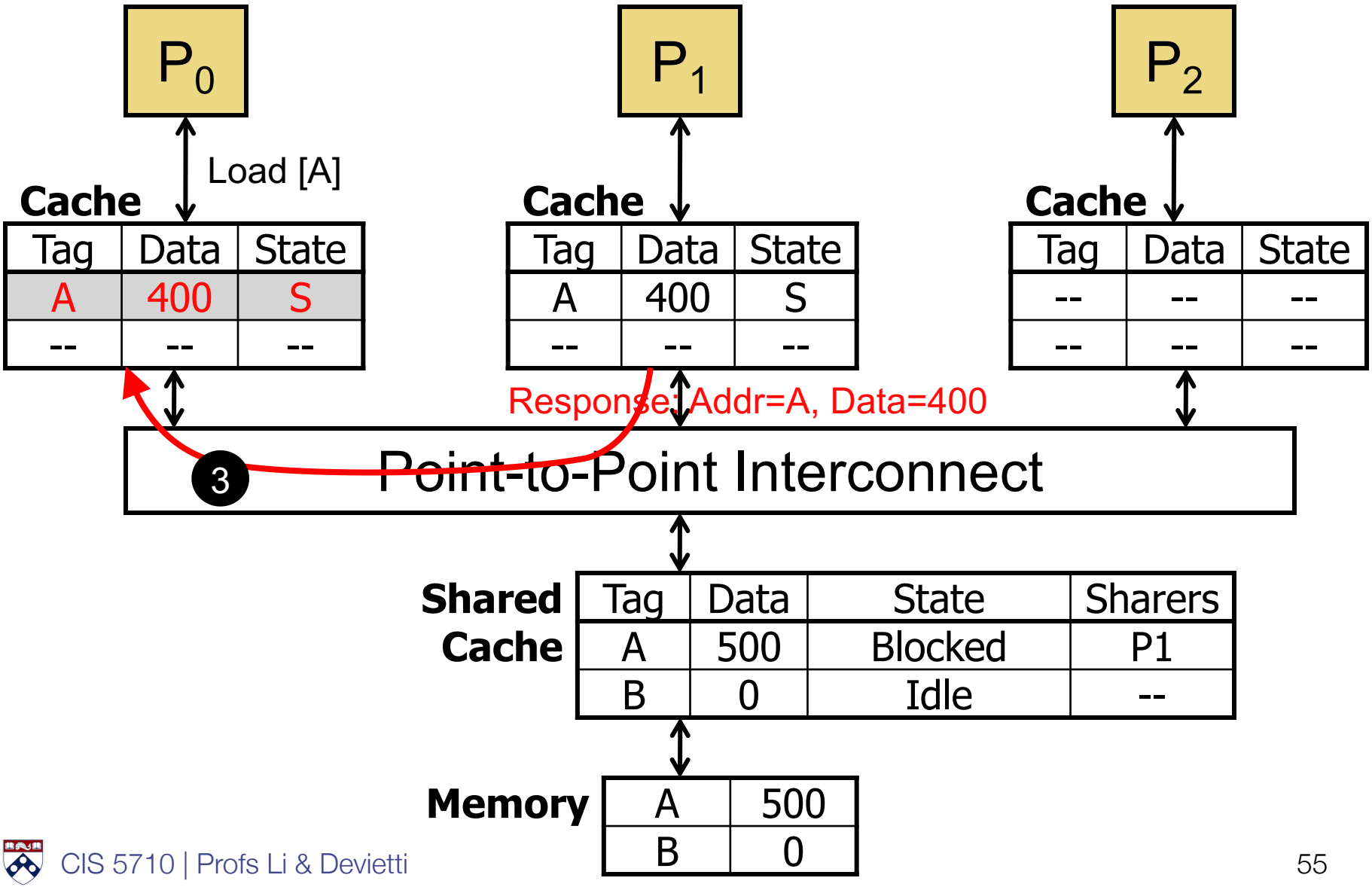
MSI Coherence Example: Step #2



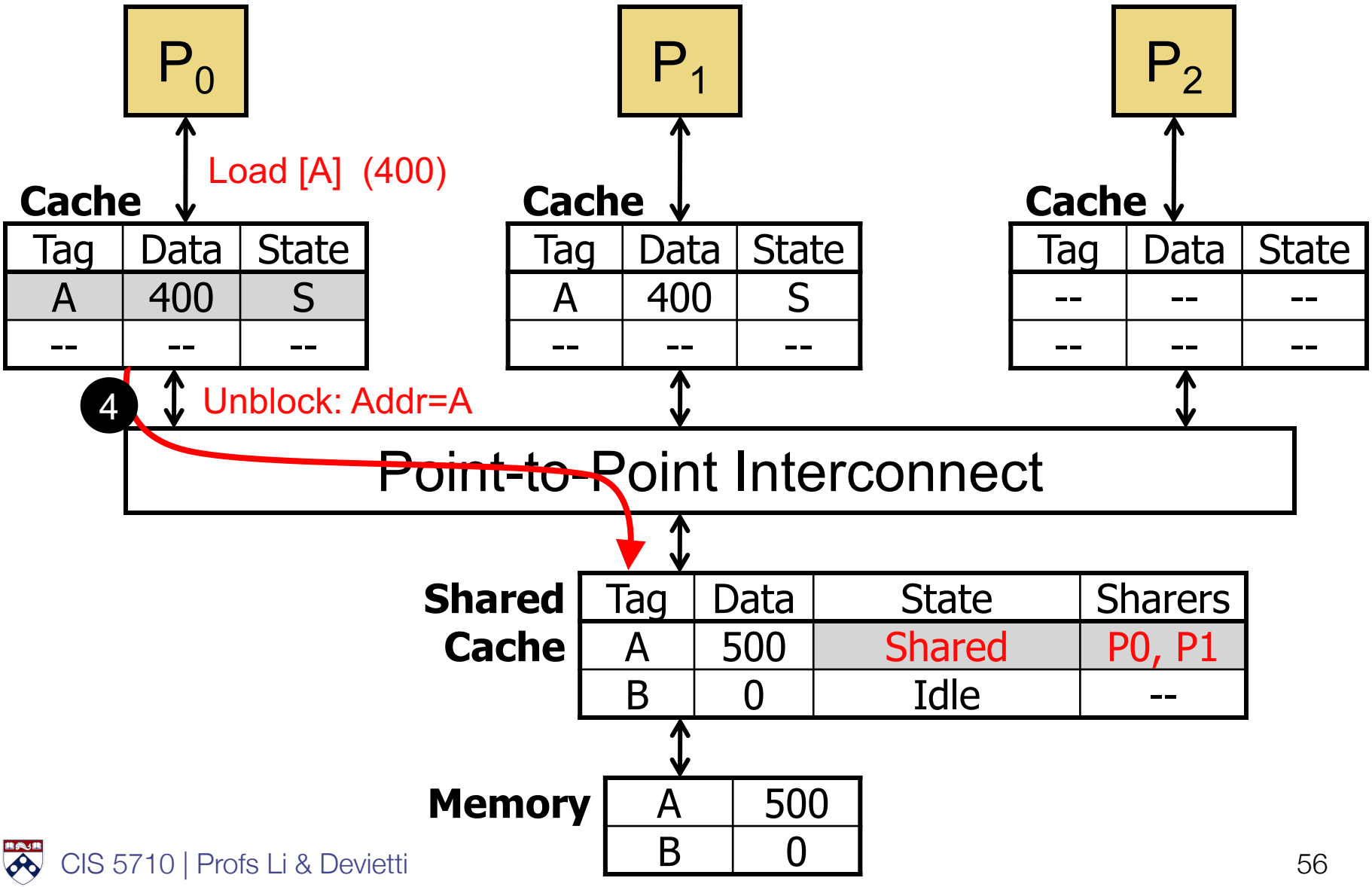
MSI Coherence Example: Step #3



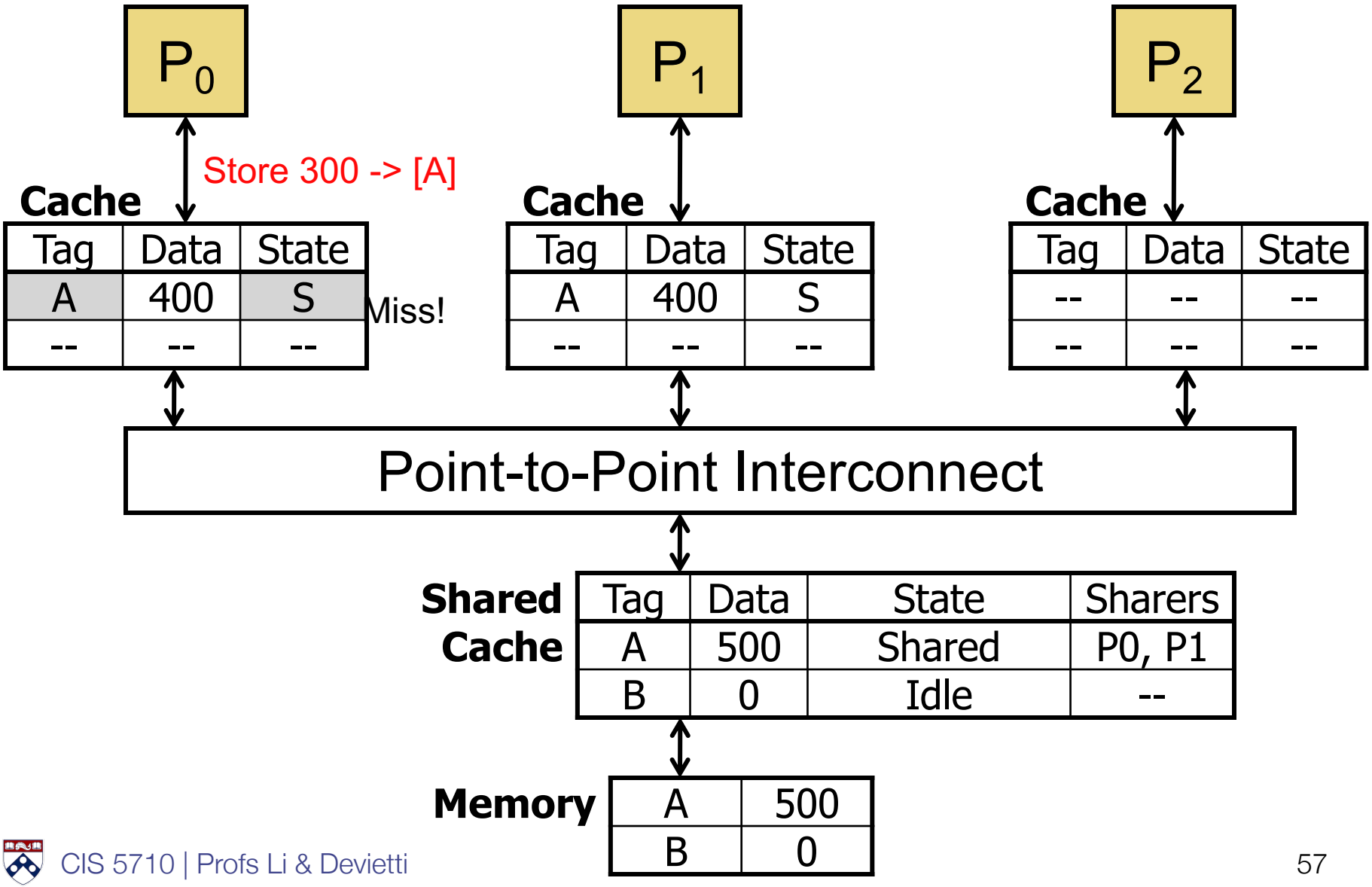
MSI Coherence Example: Step #4



MSI Coherence Example: Step #5



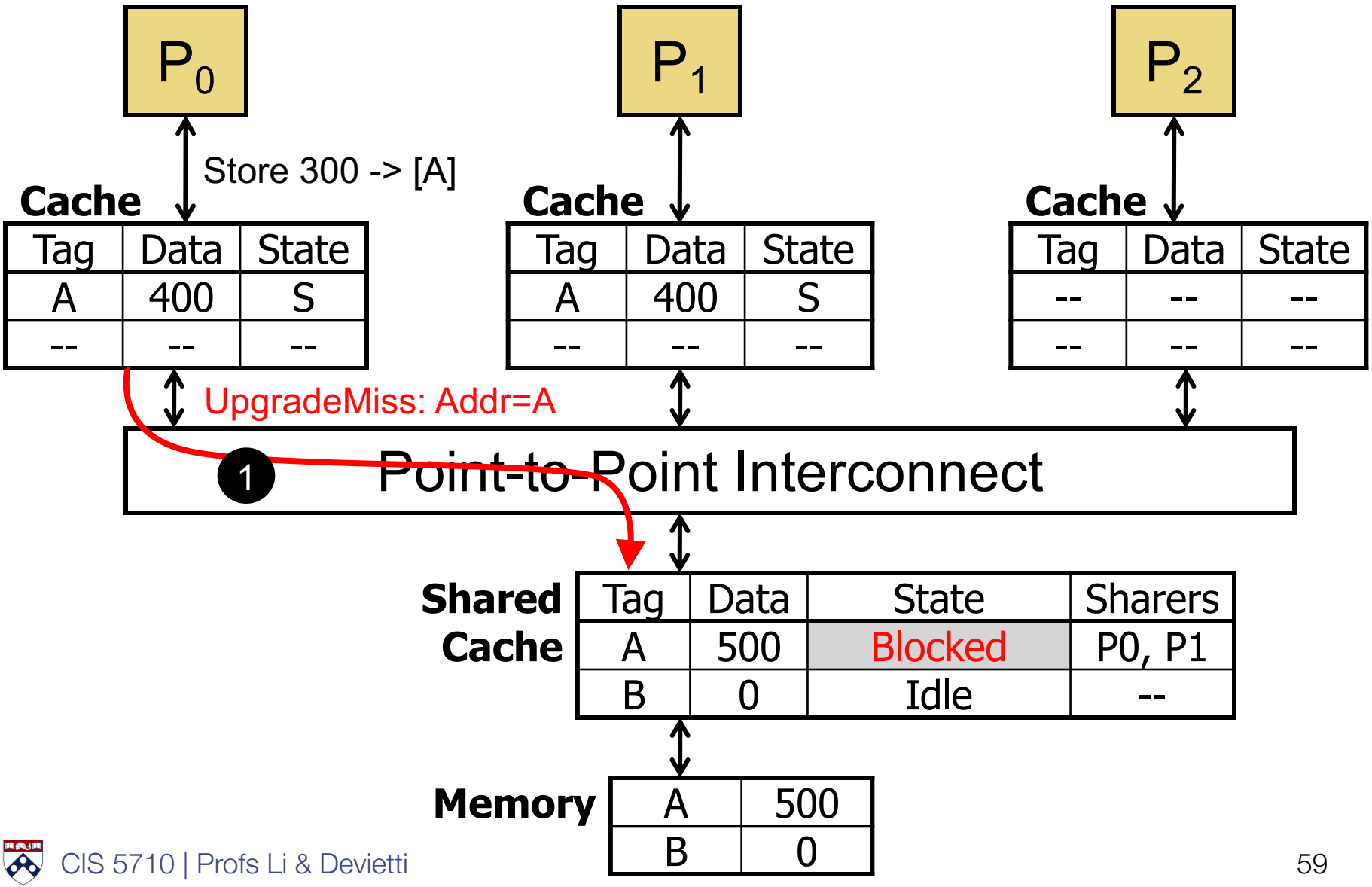
MSI Coherence Example: Step #6



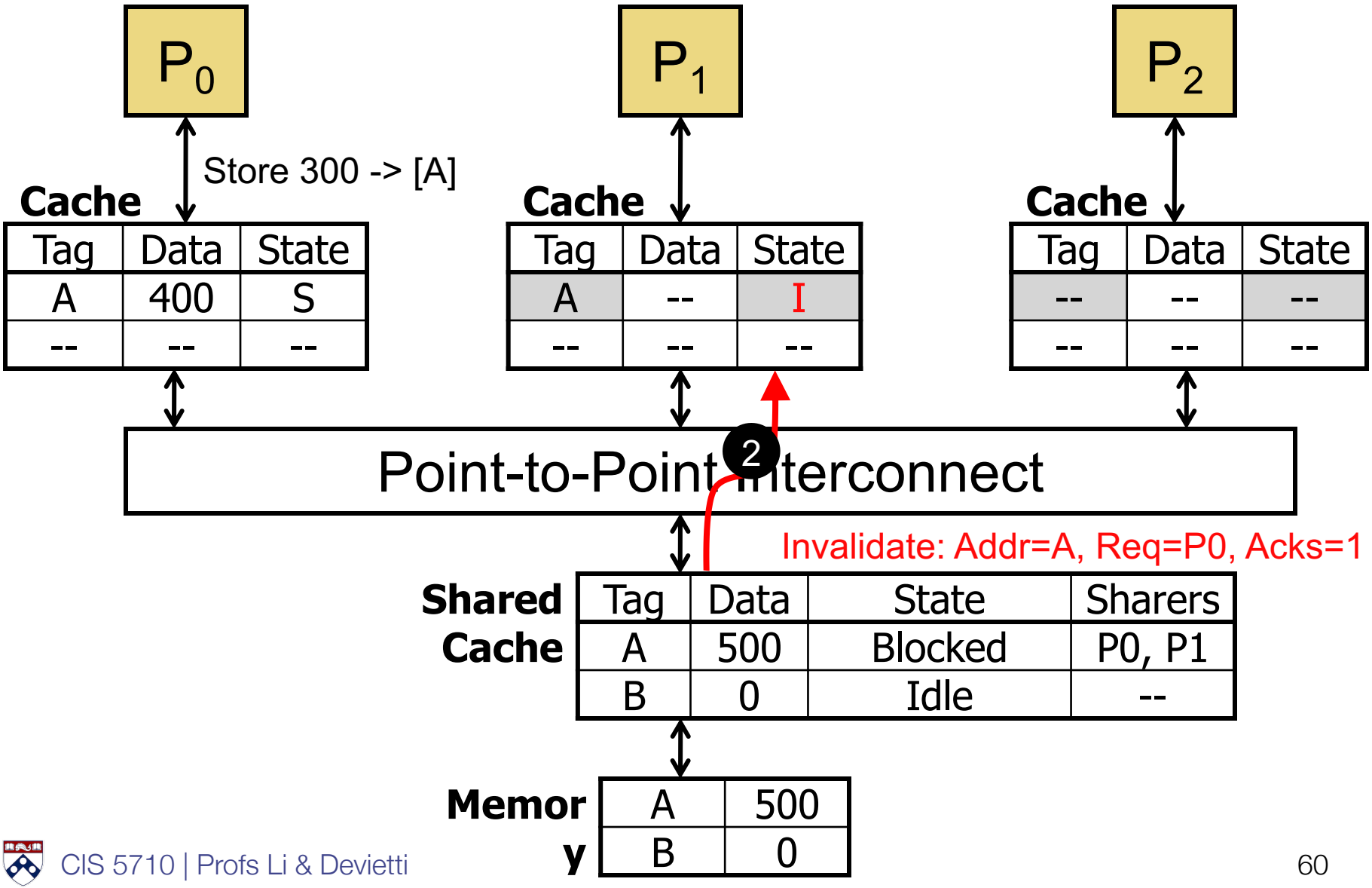
Classifying Misses: 3C Model

- Divide cache misses into three categories
  - **Compulsory (cold)**: never seen this address before
  - **Capacity**: miss caused because cache is too small
    - Would miss even in fully associative cache
  - **Conflict**: miss caused because cache associativity is too low
    - All misses that aren't compulsory or capacity
  - **COHERENCE: miss due to external invalidations**

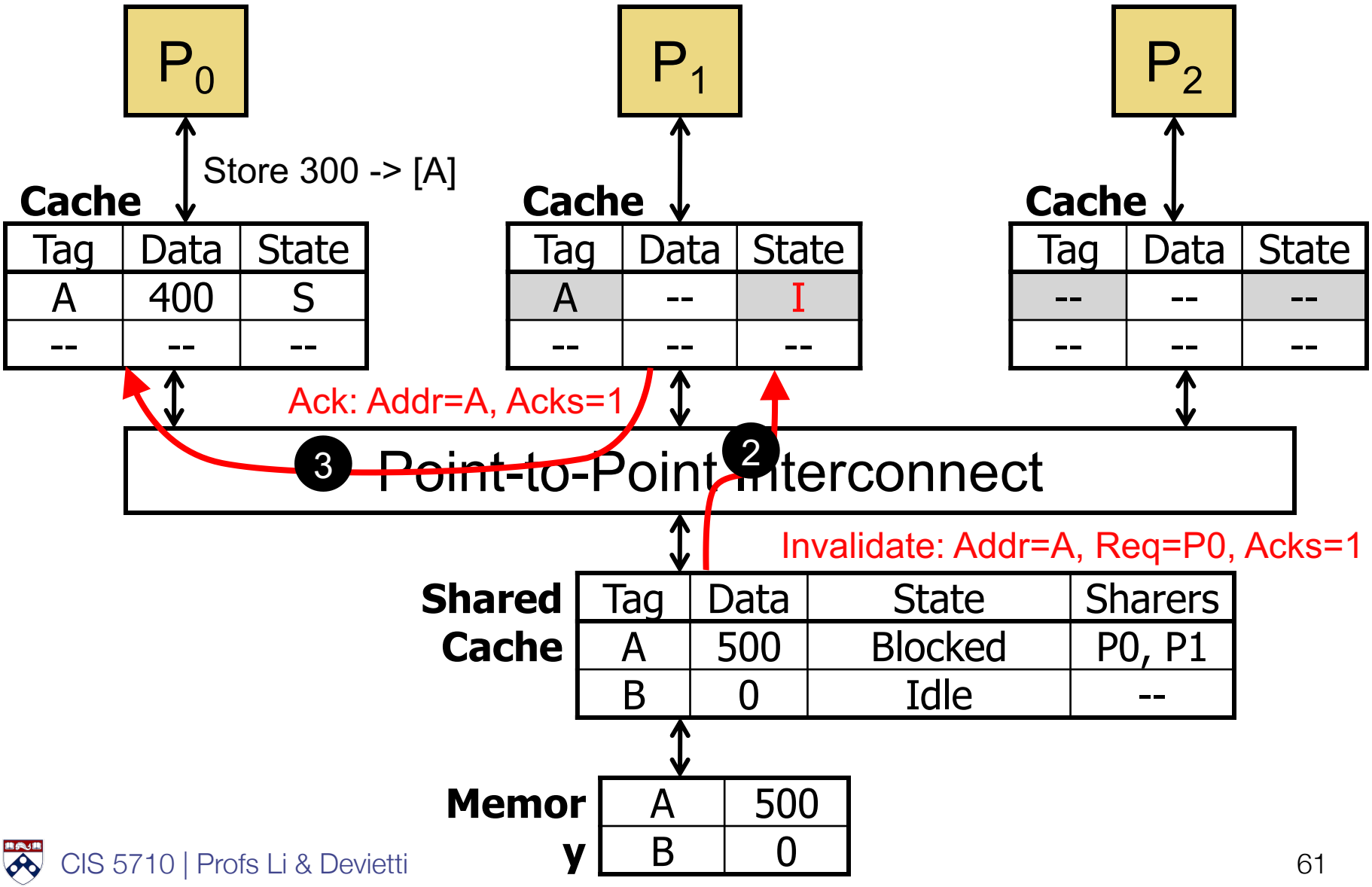
MSI Coherence Example: Step #7



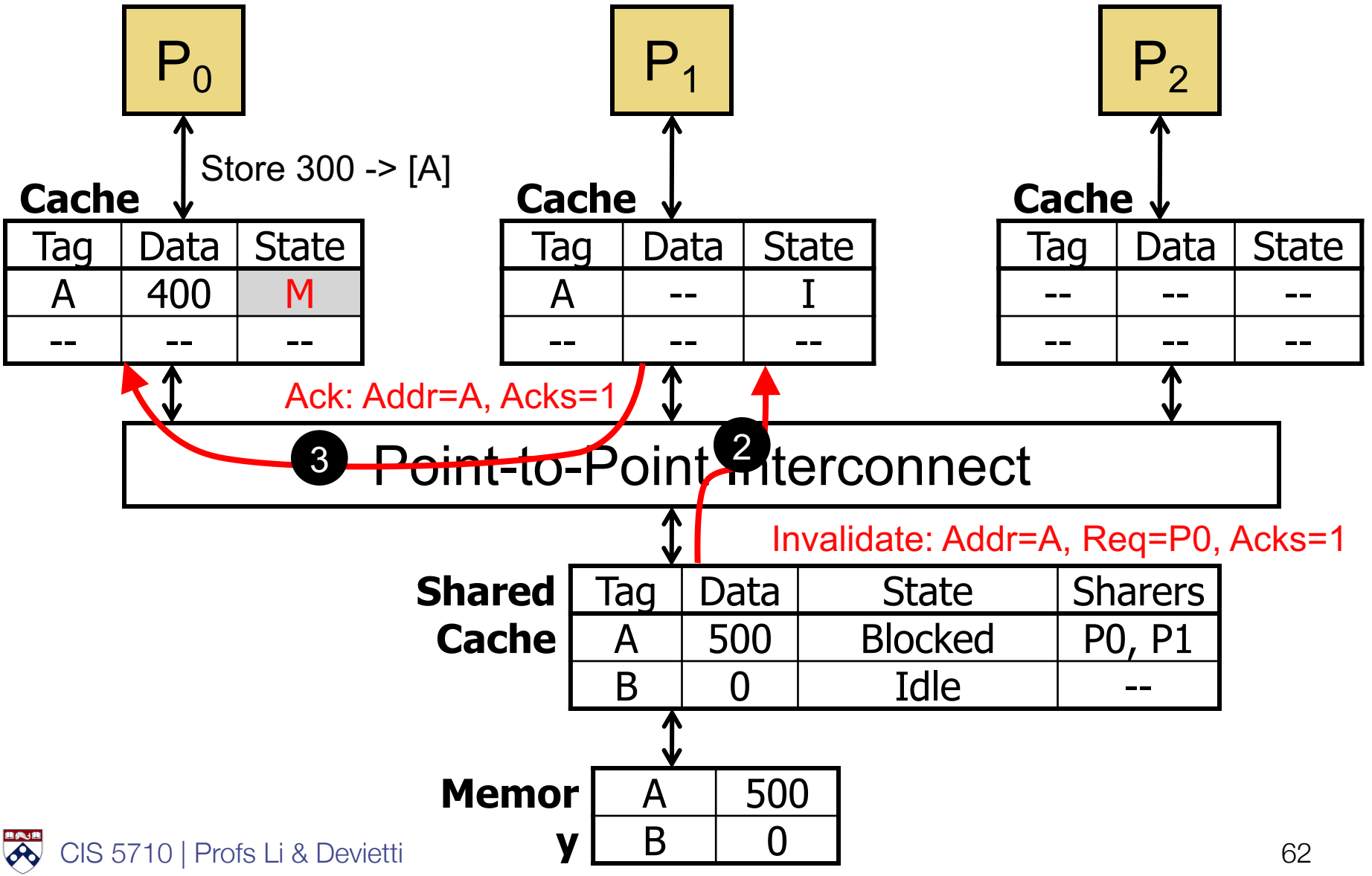
MSI Coherence Example: Step #8



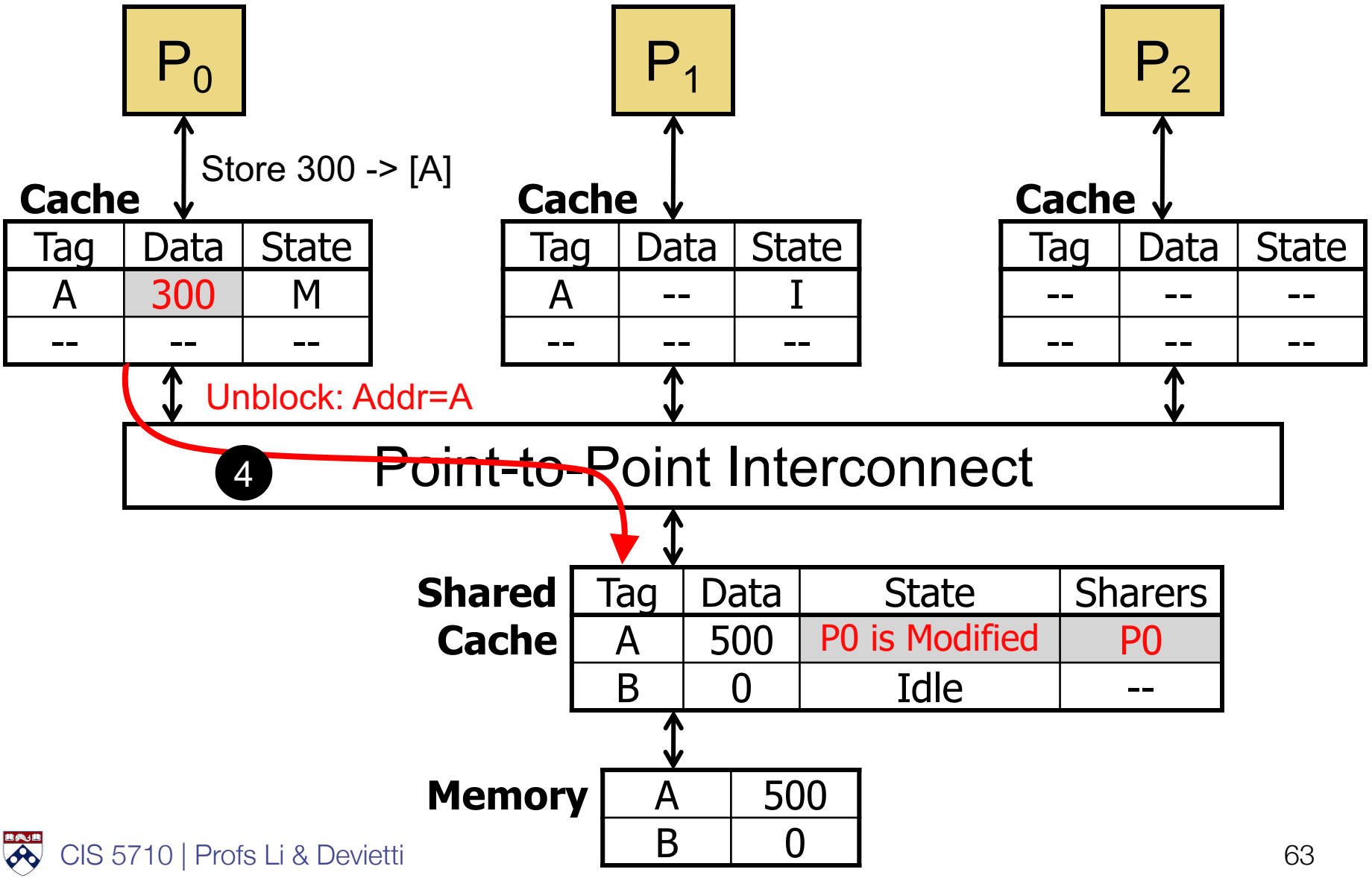
MSI Coherence Example: Step #9



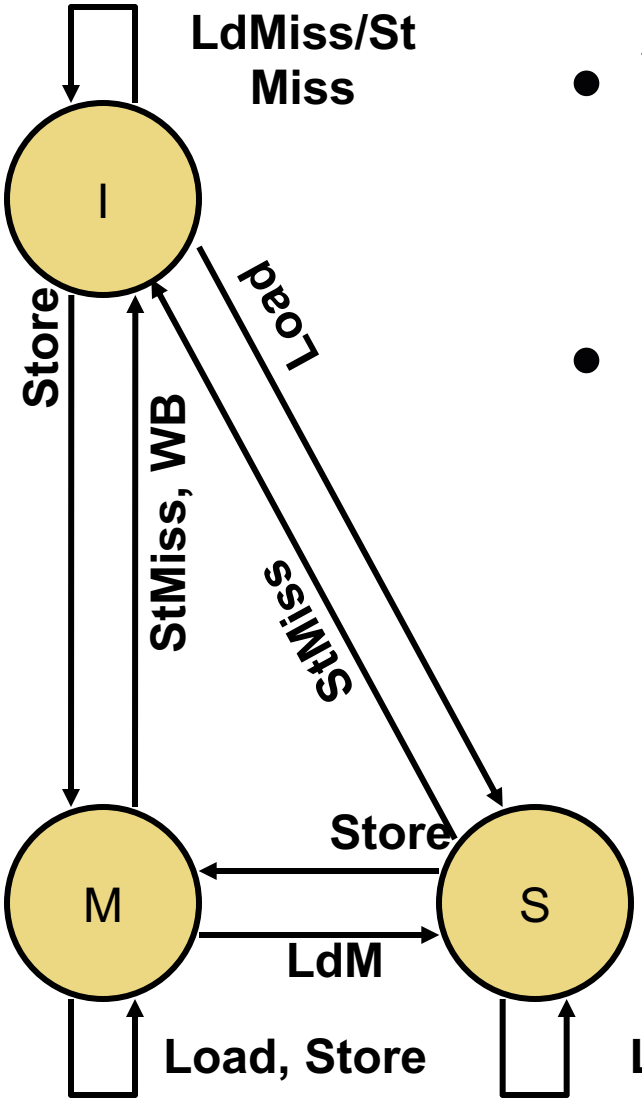
MSI Coherence Example: Step #10



MSI Coherence Example: Step #11



VI → MSI



- VI protocol is inefficient
  - Only one cached copy allowed
  - Disallows multiple read-only copies
- **MSI (modified-shared-invalid)**
  - Splits V state into two states
    - **M (modified)**: local dirty copy
    - **S (shared)**: local clean copy
  - Allows **either**
    - Multiple read-only copies (S-state), or
    - Single read/write copy (M-state)

MSI Protocol State Transition Table

State	*This Processor*		*Other Processor*	
	Load	Store	Load Miss	Store Miss
Invalid (I)	Load Miss ➔ S	Store Miss ➔ M	---	---
Shared (S)	Hit	Upgrade Miss ➔ M	Send Data ➔ S	Send Data ➔ I
Modified (M)	Hit	Hit	Send Data ➔ S	Send Data ➔ I

Cache Coherence and Cache Misses

- Coherence introduces two new kinds of misses
  - **Upgrade miss:** stores to read-only blocks
    - Delay to acquire write permission to read-only block
  - **Coherence miss**
    - Miss to a block evicted by another processor's requests
- Making the cache larger…
  - Doesn't reduce these types of misses
  - So, as cache grows large, these sorts of misses dominate
- **False sharing**
  - Two or more processors sharing parts of the same block
  - But *not* the same bytes within that block (no actual sharing)
  - Creates pathological "ping-pong" behavior
  - Careful data placement may help, but is difficult

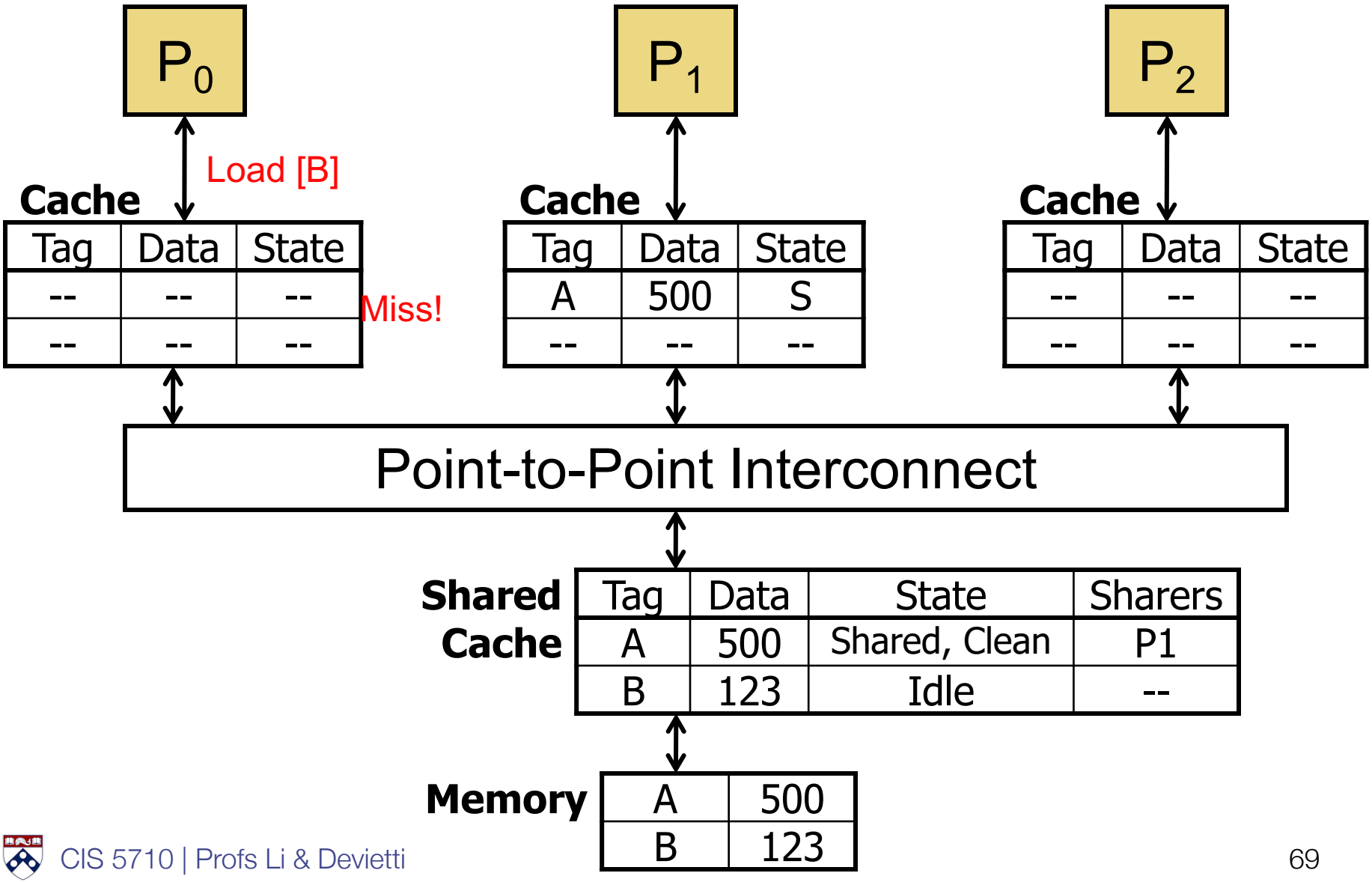
MESI Cache Coherence

- Consider load followed by store by same core
  - **two misses**: load miss plus an upgrade miss…
  - … even if the block is never shared!
  - Some programs have lots of private data
    - E.g., single-threaded programs
    - Potentially **doubling** number of cache misses
- Solution:
  - Add an **Exclusive** state
  - "I have the **only** cached copy, and it's a **clean** copy"
    - Has read/write permissions
    - Just like Modified but clean instead of dirty

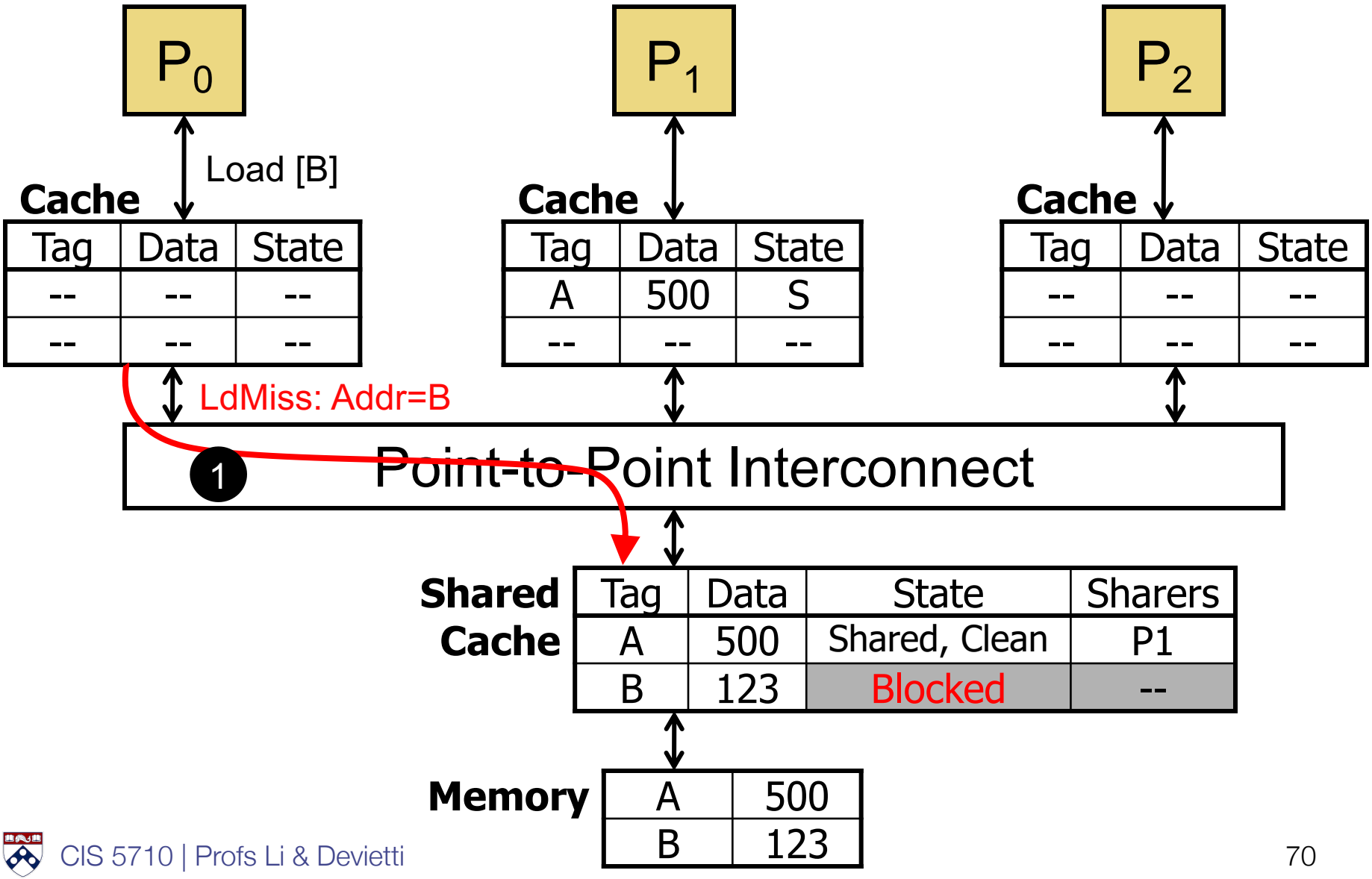
MESI Operation

- Goals:
  - Avoid “upgrade” misses for non-shared blocks
  - While not increasing eviction (aka writeback or replacement) traffic
- Two cases on a load miss to a block…
  - **Case #1**: … with no current sharers
    (that is, no sharers in the set of sharers)
    - Grant requester Exclusive copy with read/write permission
  - **Case #2**: … with other sharers
    - As before, grant just a Shared copy with read-only permission
- A store to a block in Exclusive changes it to Modified
  - **Instantaneously & silently** (no latency or traffic)
- On block eviction (aka writeback or replacement)…
  - If Modified, block is dirty, must be written back to next level
  - If Exclusive, writing back the data is not necessary

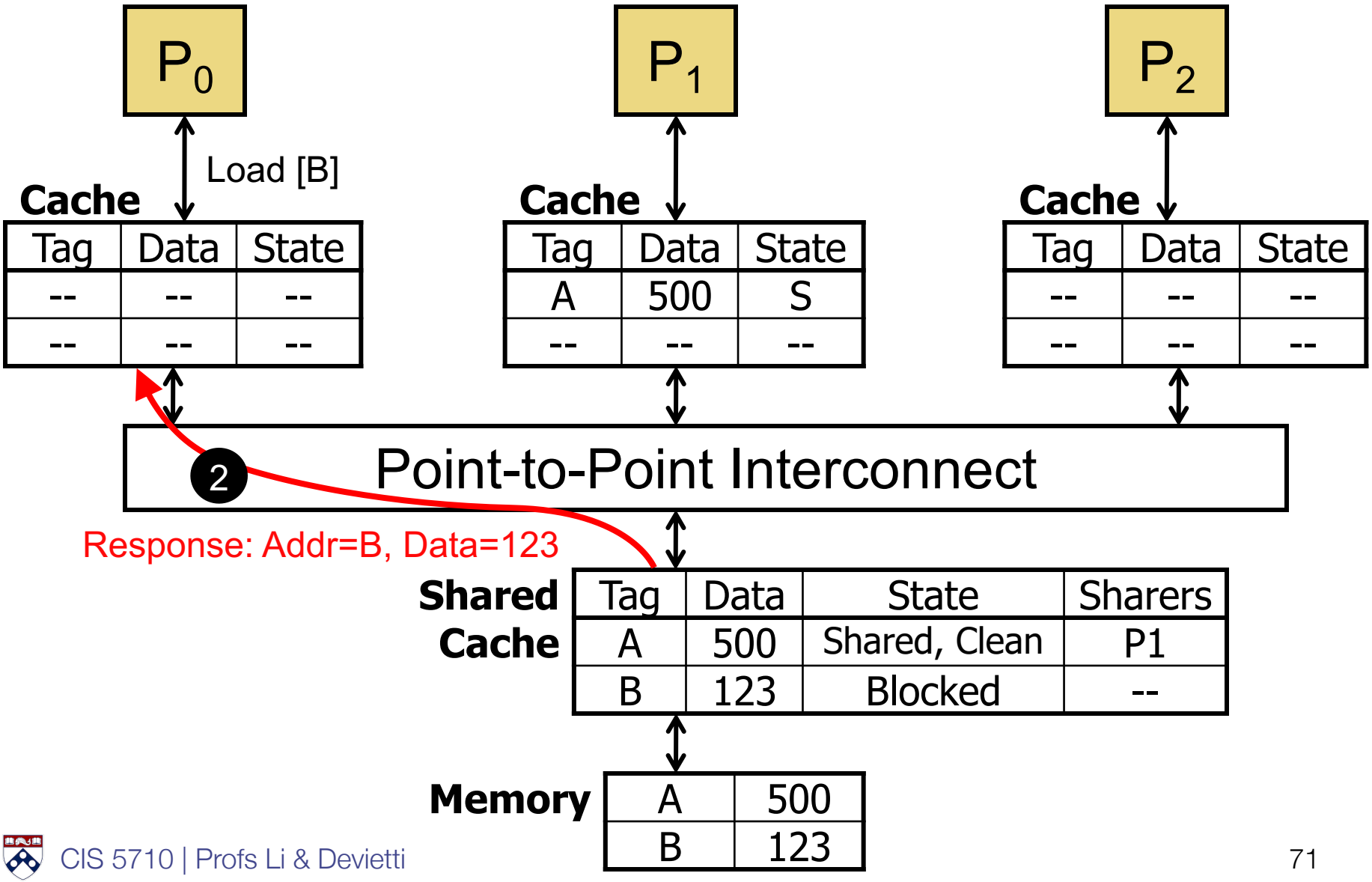
MESI Coherence Example: Step #1



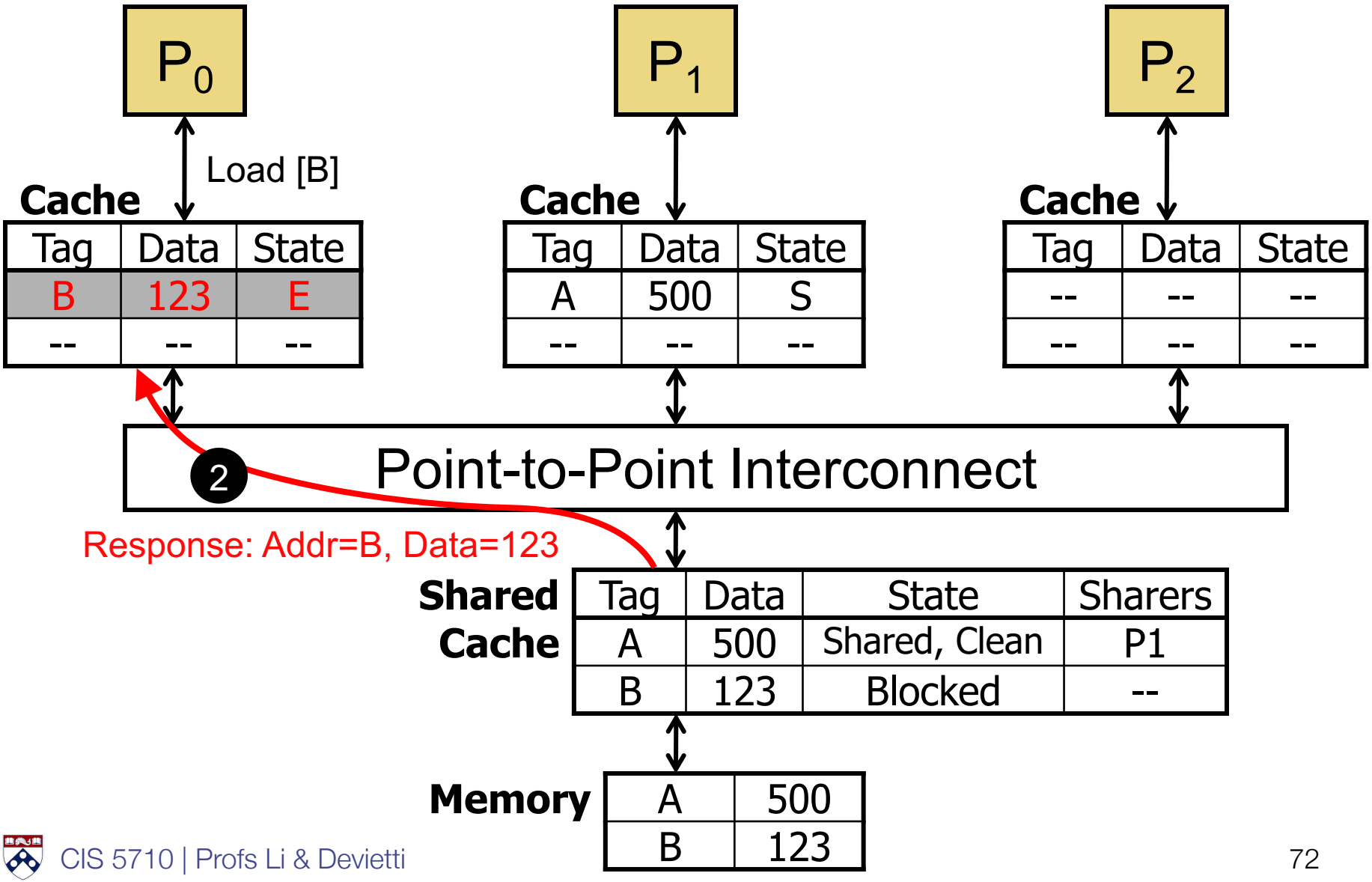
MESI Coherence Example: Step #2



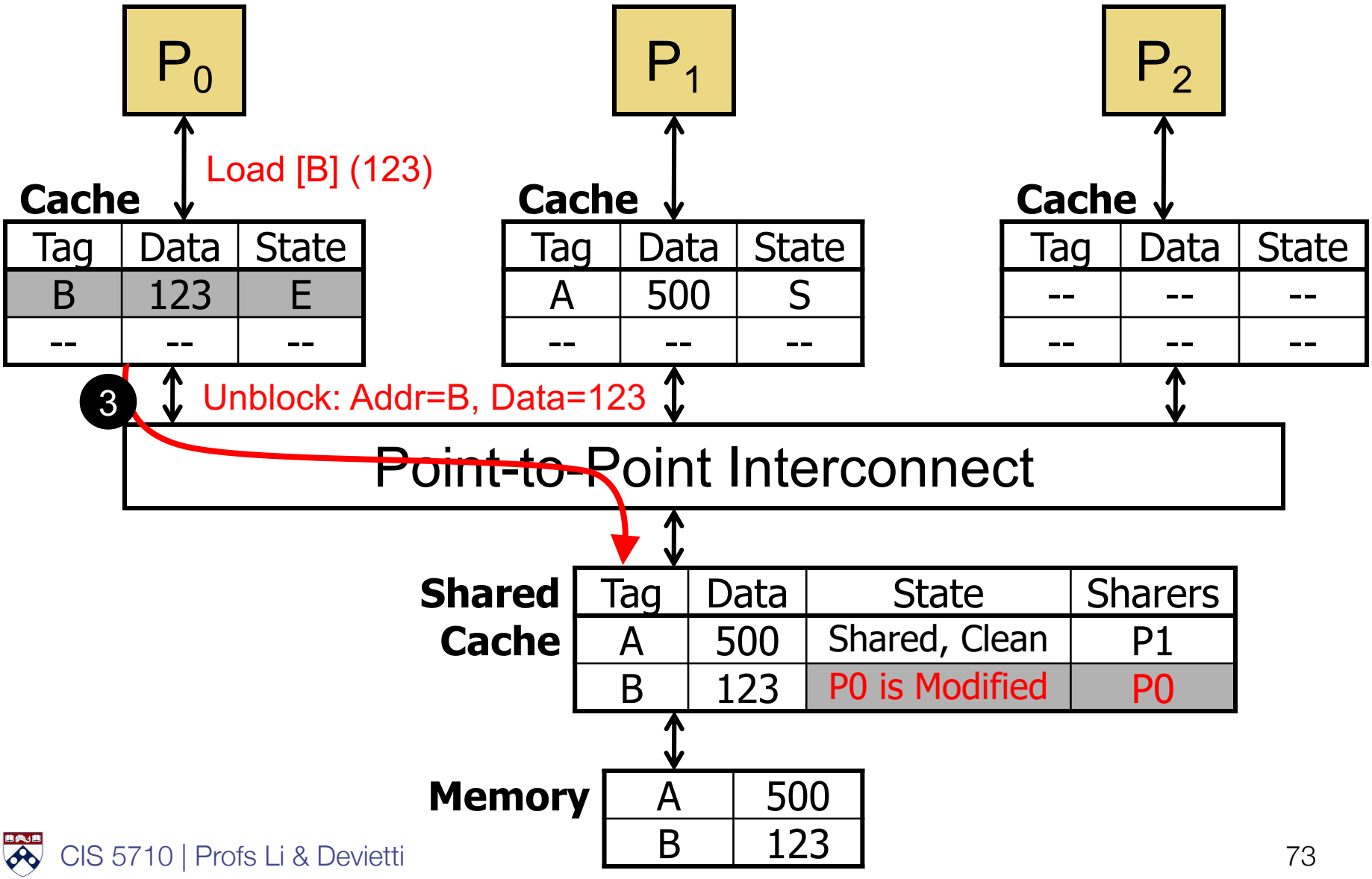
MESI Coherence Example: Step #3



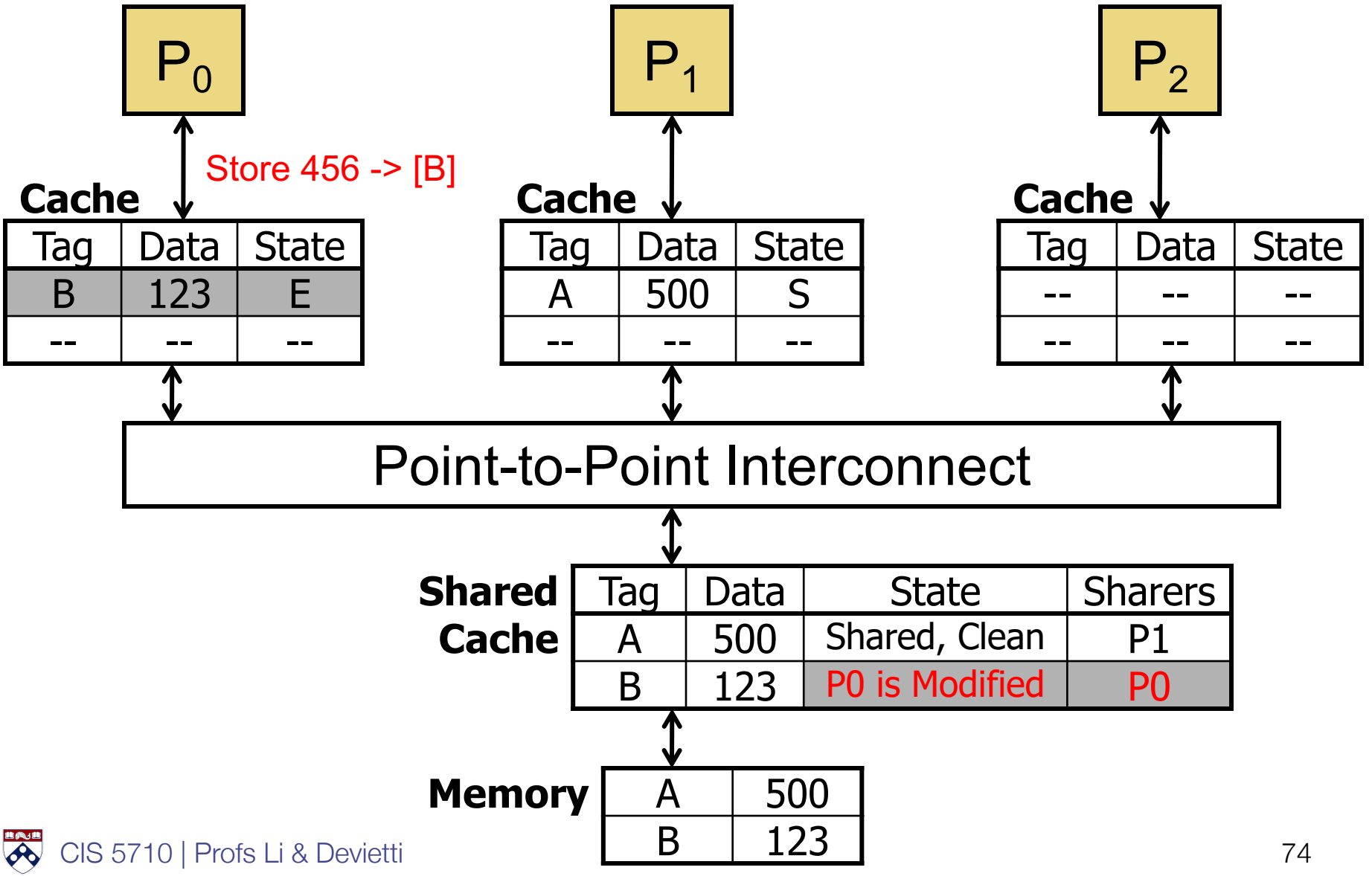
MESI Coherence Example: Step #4



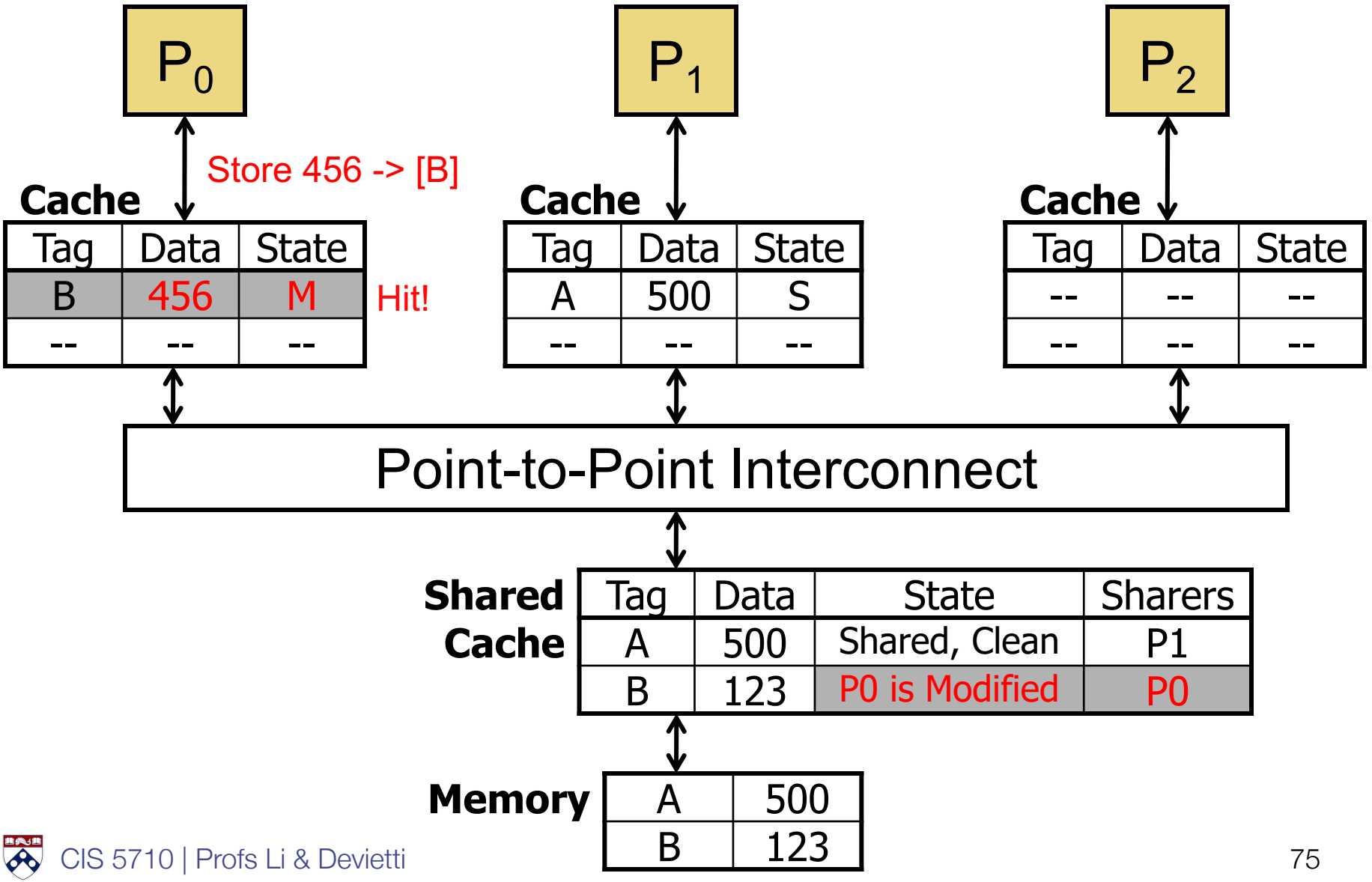
MESI Coherence Example: Step #5



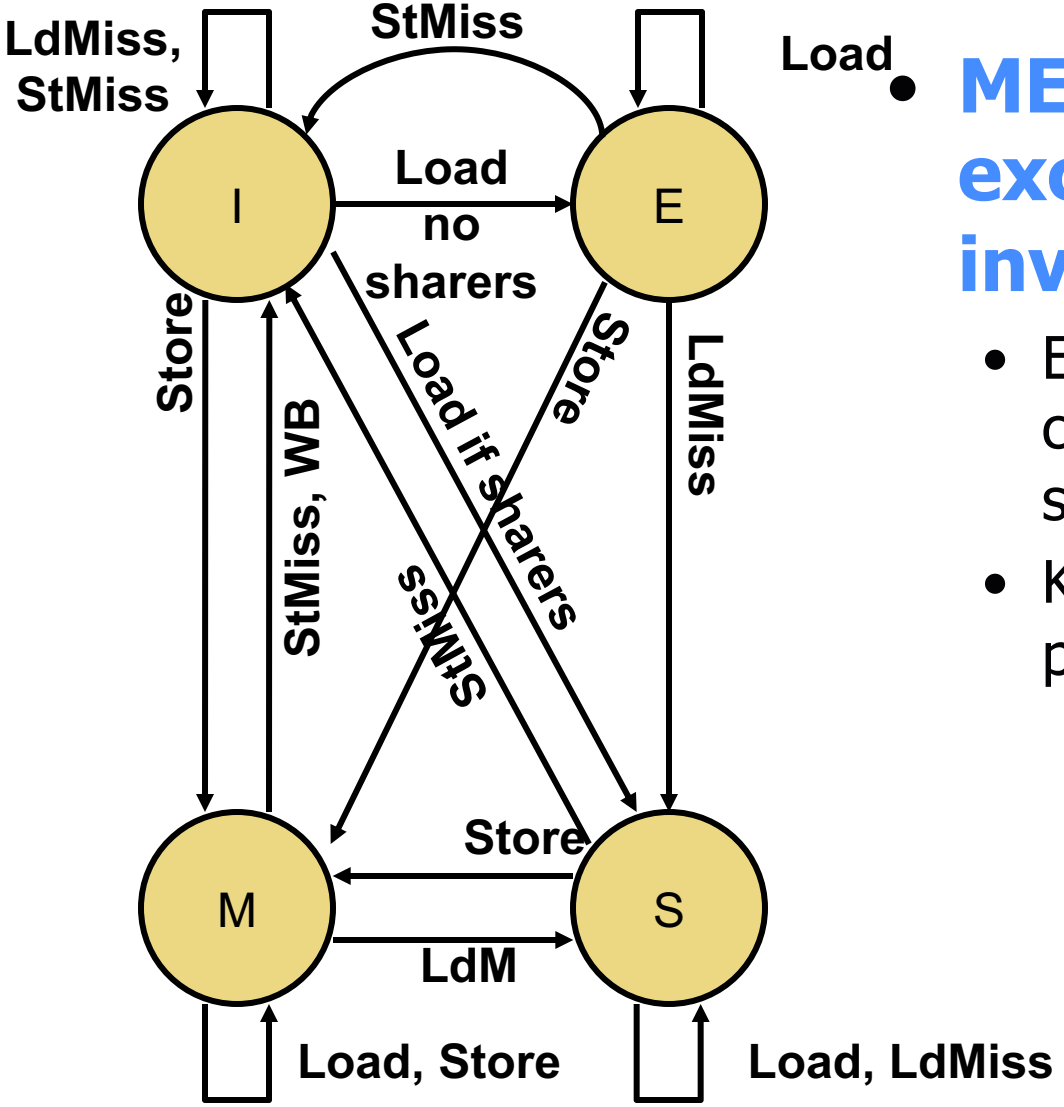
MESI Coherence Example: Step #6



MESI Coherence Example: Step #7



MSI → MESI



- **MESI (modified-exclusive-shared-invalid)**
  - Eliminates the cost of coherence when there's no sharing
  - Keeps single-threaded programs fast on multicores

MESI State Transition Table

State	*This Processor*		*Other Processor*	
	Load	Store	Load Miss	Store Miss
Invalid (I)	Miss → **S or E**	Miss → M	---	---
Shared (S)	Hit	Upg Miss → M	Send Data → S	Send Data → I
Exclusive (E)	Hit	**Hit → M**	Send Data → S	Send Data → I
Modified (M)	Hit	Hit	Send Data → S	Send Data → I

- Load misses lead to "E" if no other processors is caching the block

Coherence and writeback caches

- redundancy between dirty bit and coherence state
  - an Exclusive block is always clean, Modified always dirty
  - **a Shared block could be clean *or* dirty**
    - e.g., block was Modified, then downgraded to Shared
- When should we write back Shared blocks?
  - Never write back Shared blocks?
    - 👎previous stores to the Shared block are lost
  - Always write back Shared blocks?
    - 👎if k writebacks occur, k-1 writebacks are redundant
  - Track a separate dirty bit?
    - 👎wastes space
- Solution: integrate dirty bit into coherence state

MOESI coherence protocol

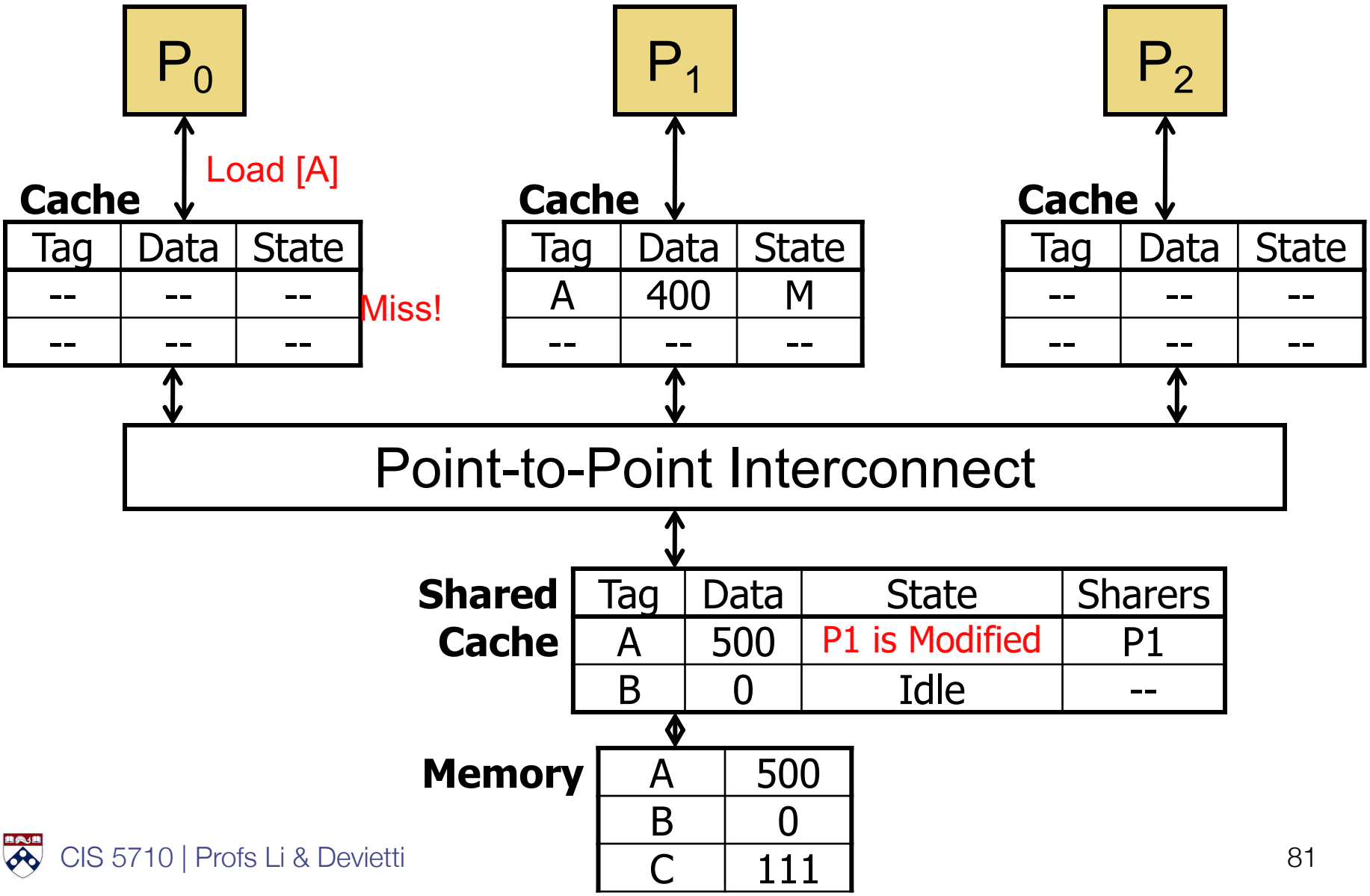
- split Shared state into **Shared** and **Owned** states
  - a Shared block is read-only and clean
  - **an Owned block is read-only and dirty**
    - on eviction, an Owned block must be written back
    - maintain just one Owned copy to avoid redundant writebacks
  - dirty bit is fully-redundant now, so scrap it
- this is the **MOESI** (”mow-zee”) coherence protocol
  - common in industrial designs

MOESI State Transition Table

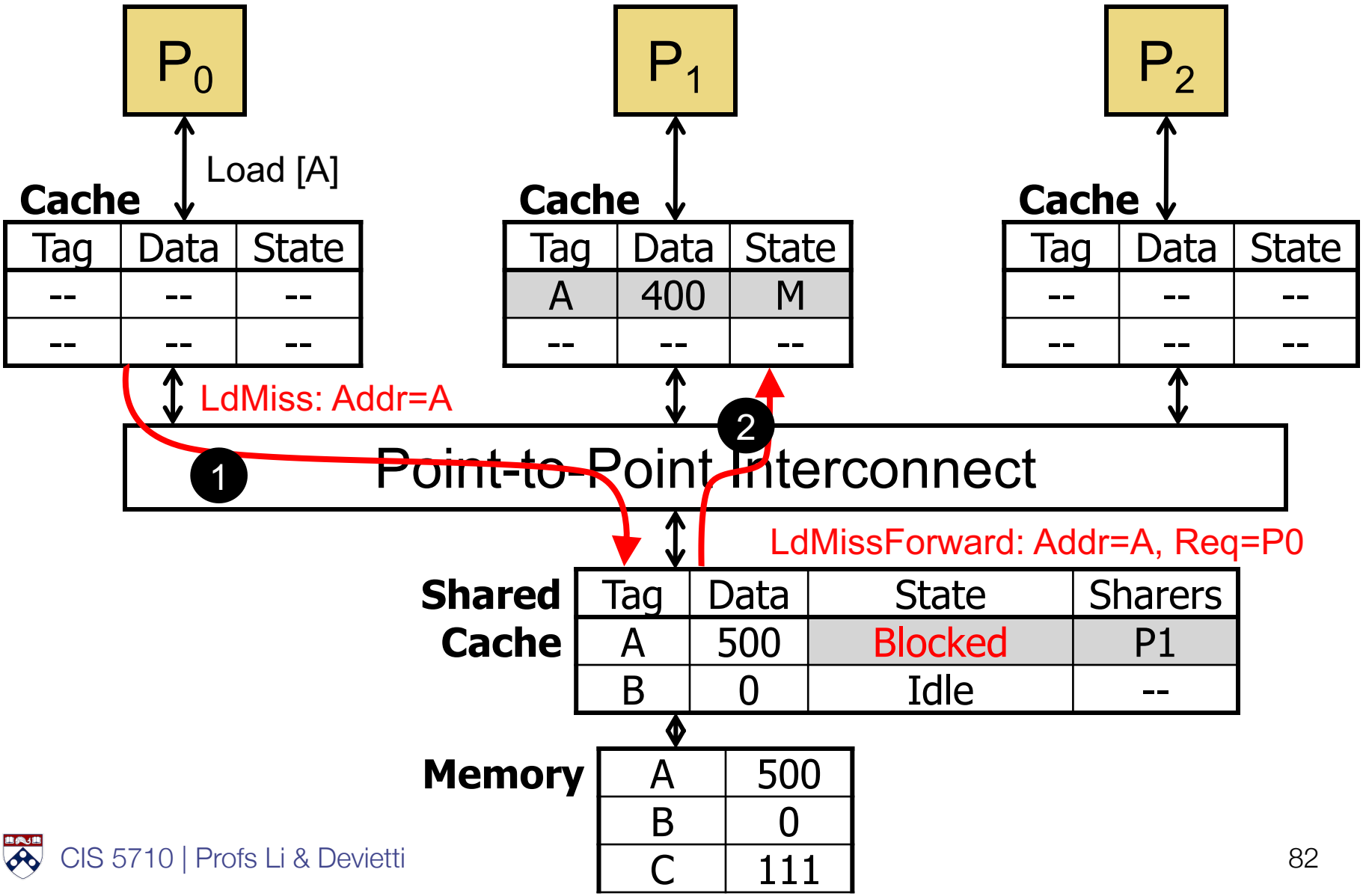
State	*This Processor*		*Other Processor*	
	Load	Store	Load Miss	Store Miss
Invalid (I)	Miss → S or E	Miss → M	---	---
Shared (S)	Hit	Upg Miss → M	Send Data → S	Send Data → I
Owned (O)	Hit	Upg Miss → M	Send Data → O	Send Data → I
Exclusive (E)	Hit	Hit → M	Send Data → S	Send Data → I
Modified (M)	Hit	Hit	Send Data → O	Send Data → I

- Load misses lead to "E" if no other processors is caching the block

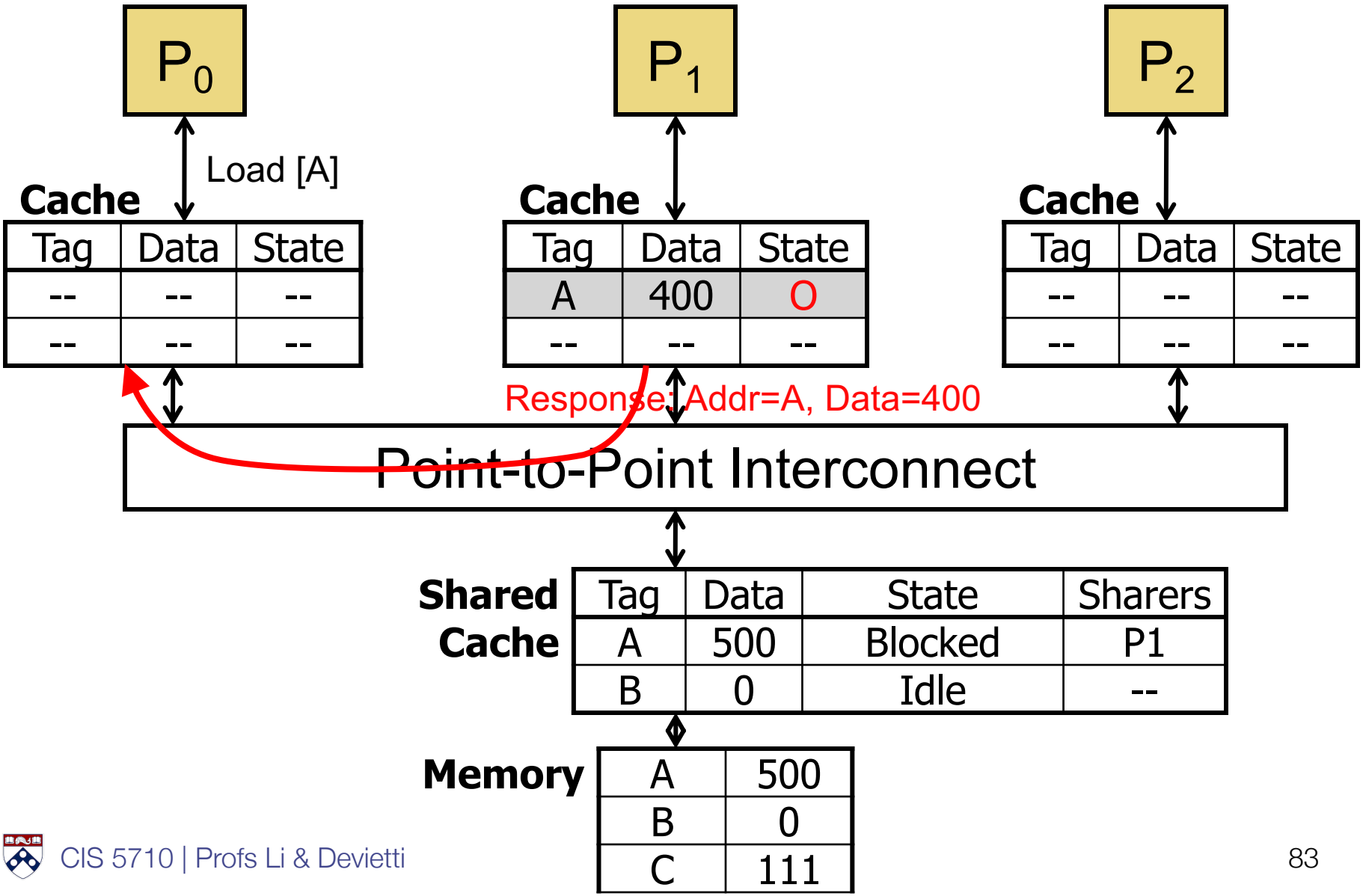
MOESI Coherence Example: Step #1



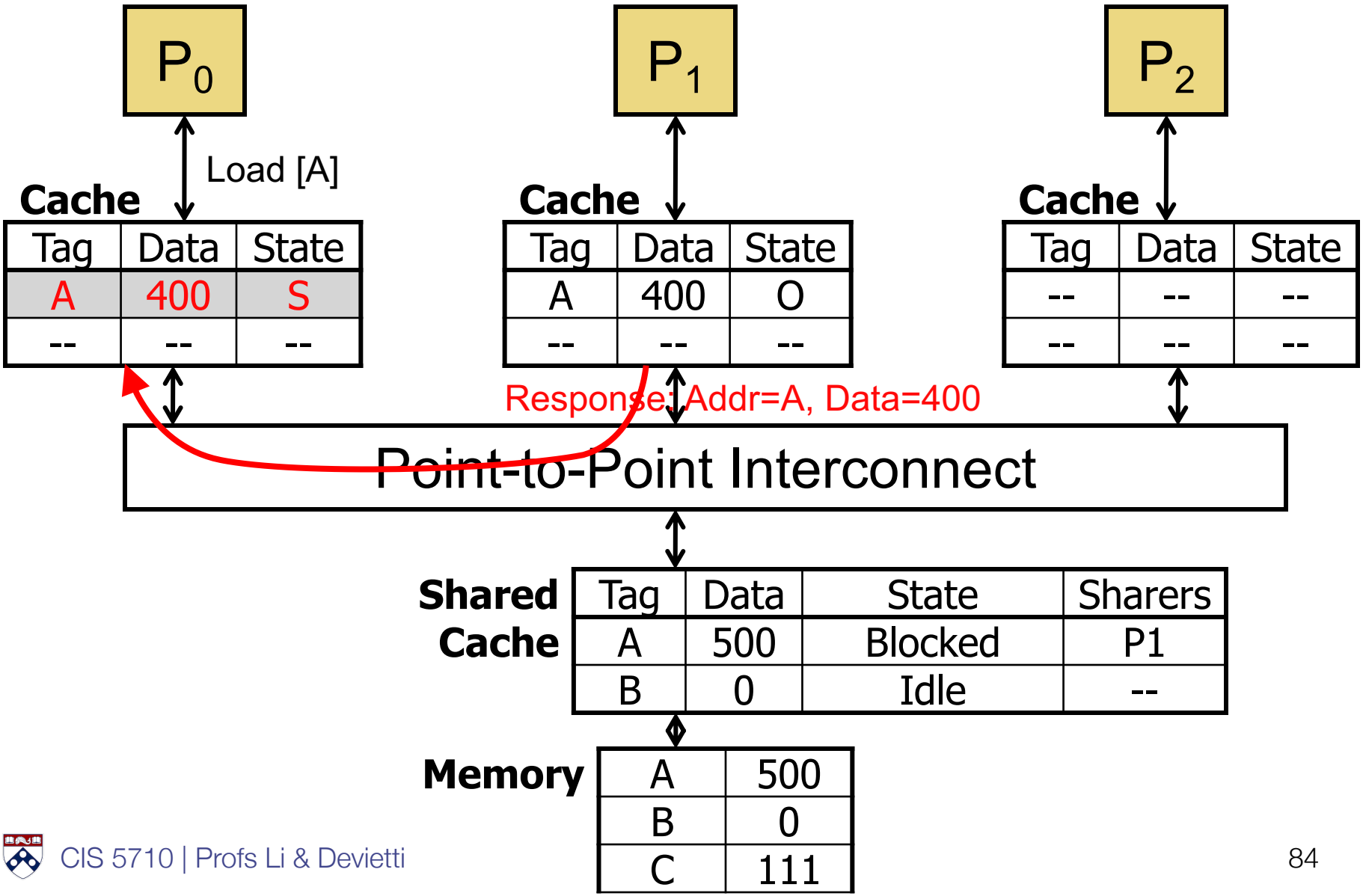
MOESI Coherence Example: Step #2



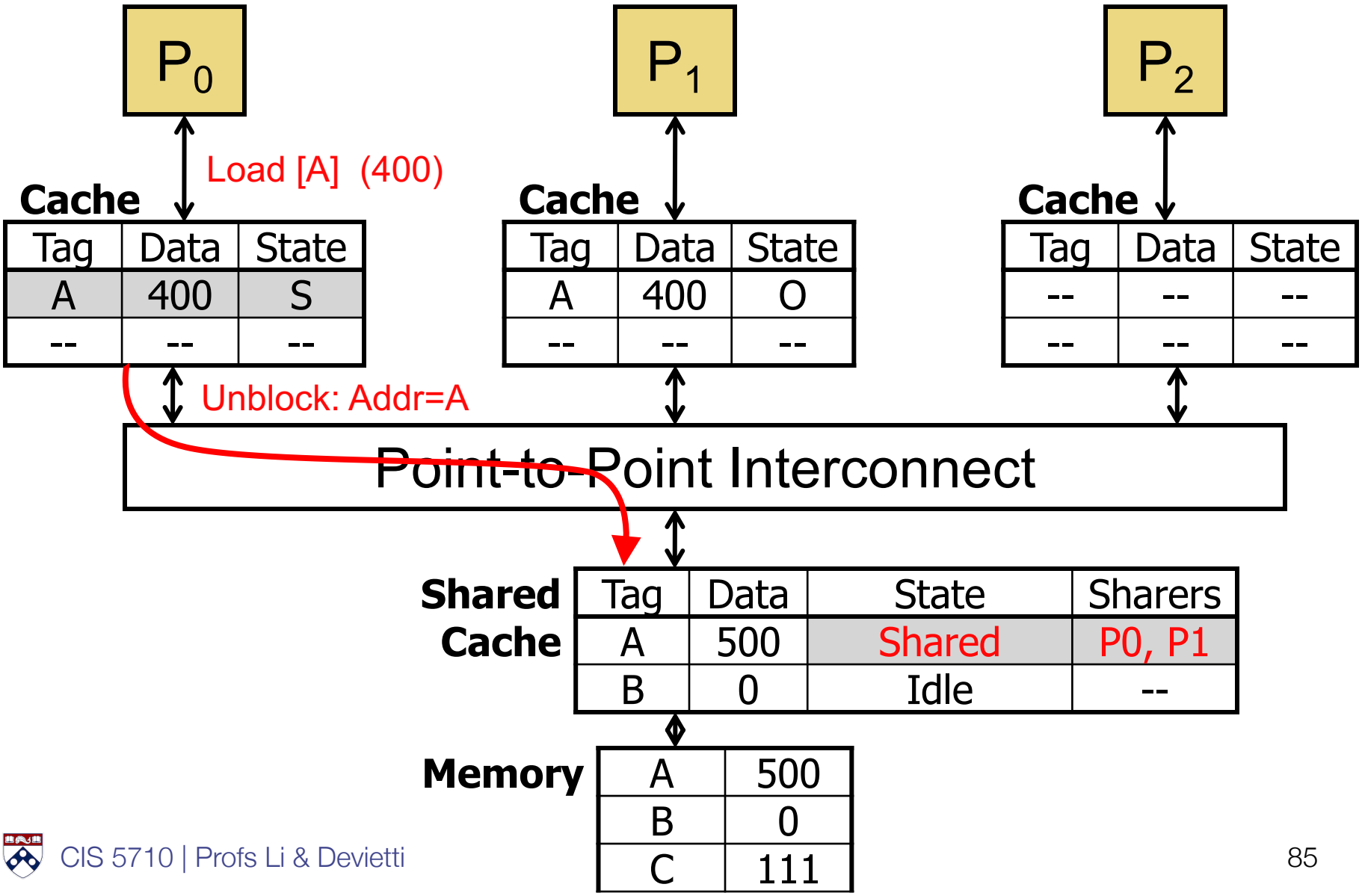
MOESI Coherence Example: Step #3



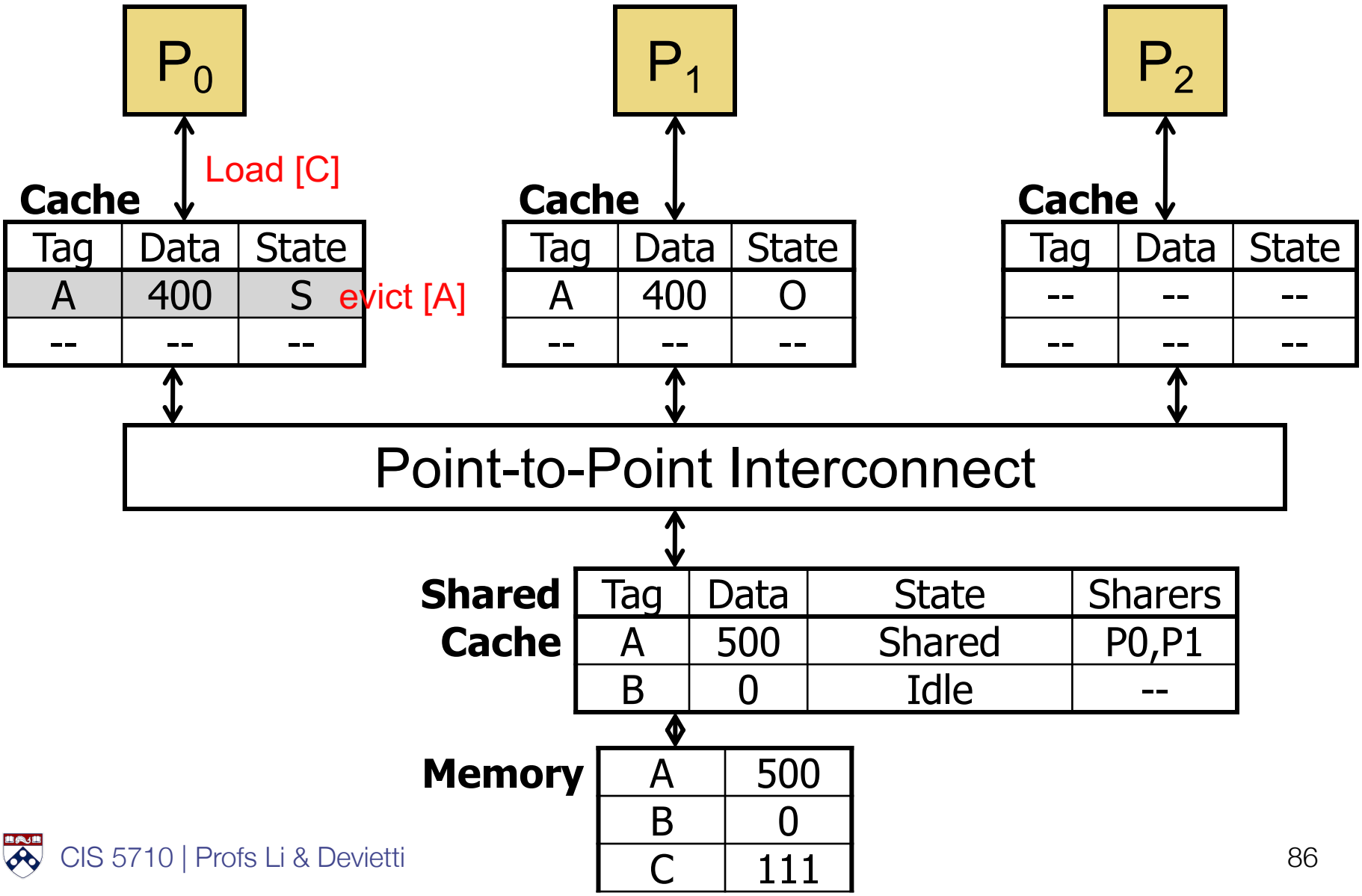
MOESI Coherence Example: Step #4



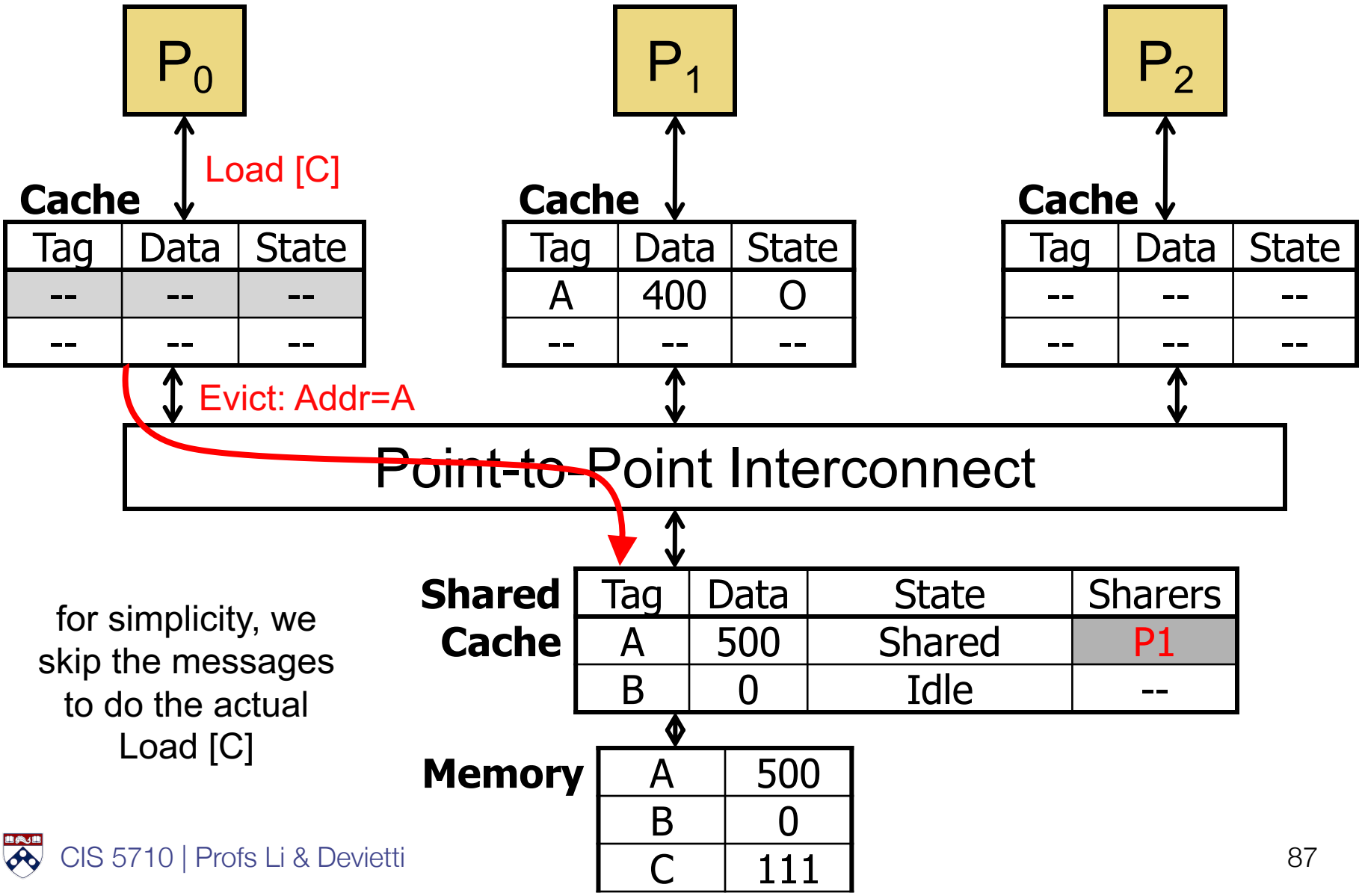
MOESI Coherence Example: Step #5



MOESI Coherence Example: Step #6

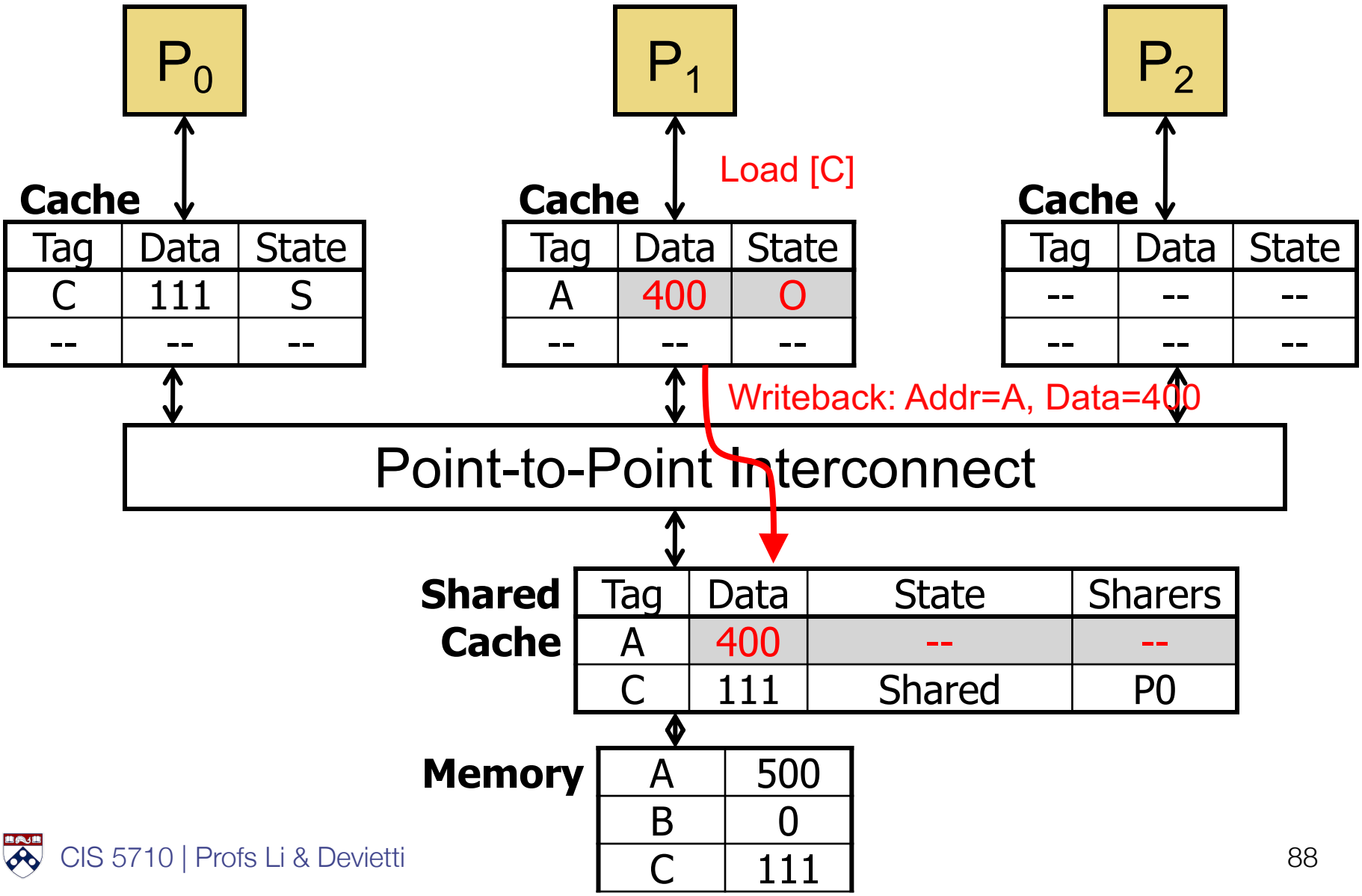


MOESI Coherence Example: Step #7



for simplicity, we skip the messages to do the actual Load [C]

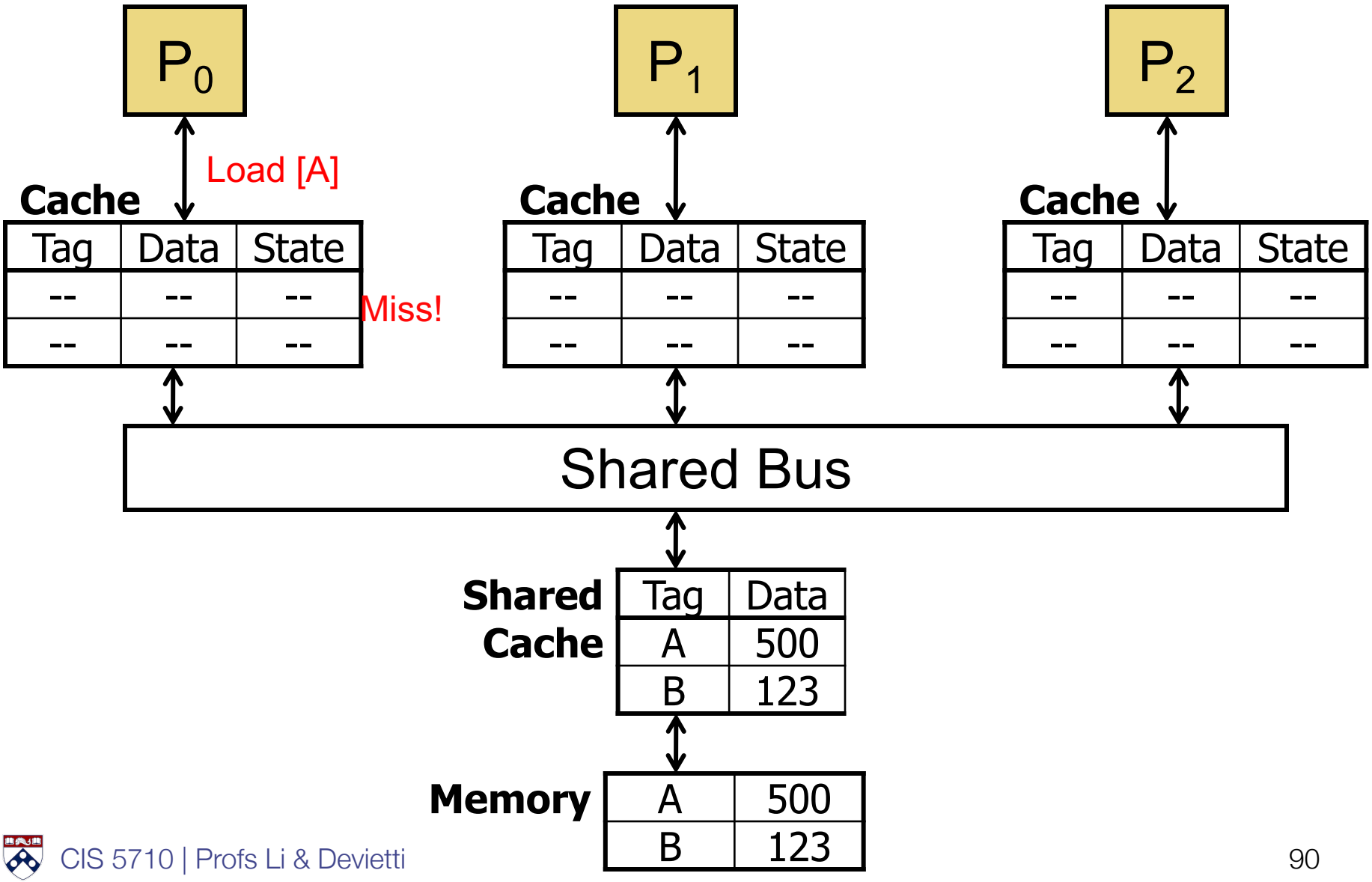
MOESI Coherence Example: Step #8



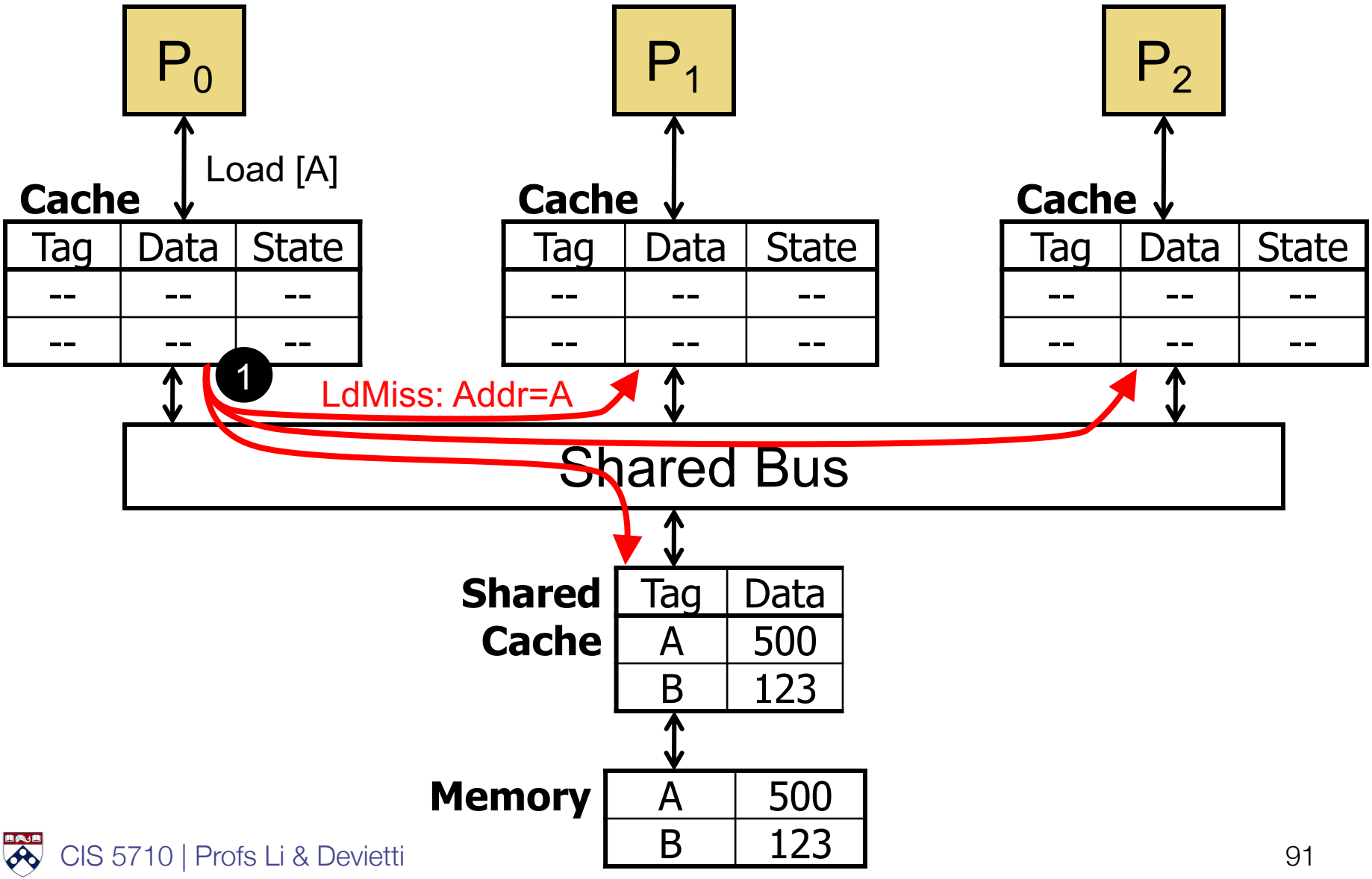
Cache Coherence Protocols

- Two general types
  - **Update-based cache coherence**
    - Write-through update to all caches
    - Too much traffic; used in the past, not common today
  - **Invalidation-based cache coherence** (examples shown)
- Of invalidation-based cache coherence, two types:
  - **Snooping/broadcast-based cache coherence** (example next)
    - No explicit state, but too much traffic for large systems
  - **Directory-based cache coherence** (examples shown)
- For directory-based cache coherence, two options:
  - Force LLC to be inclusive of all on-chip caches
    - **Encode sharers in LLC** (examples shown)
  - No inclusion? Track sharers in a directory/directory cache

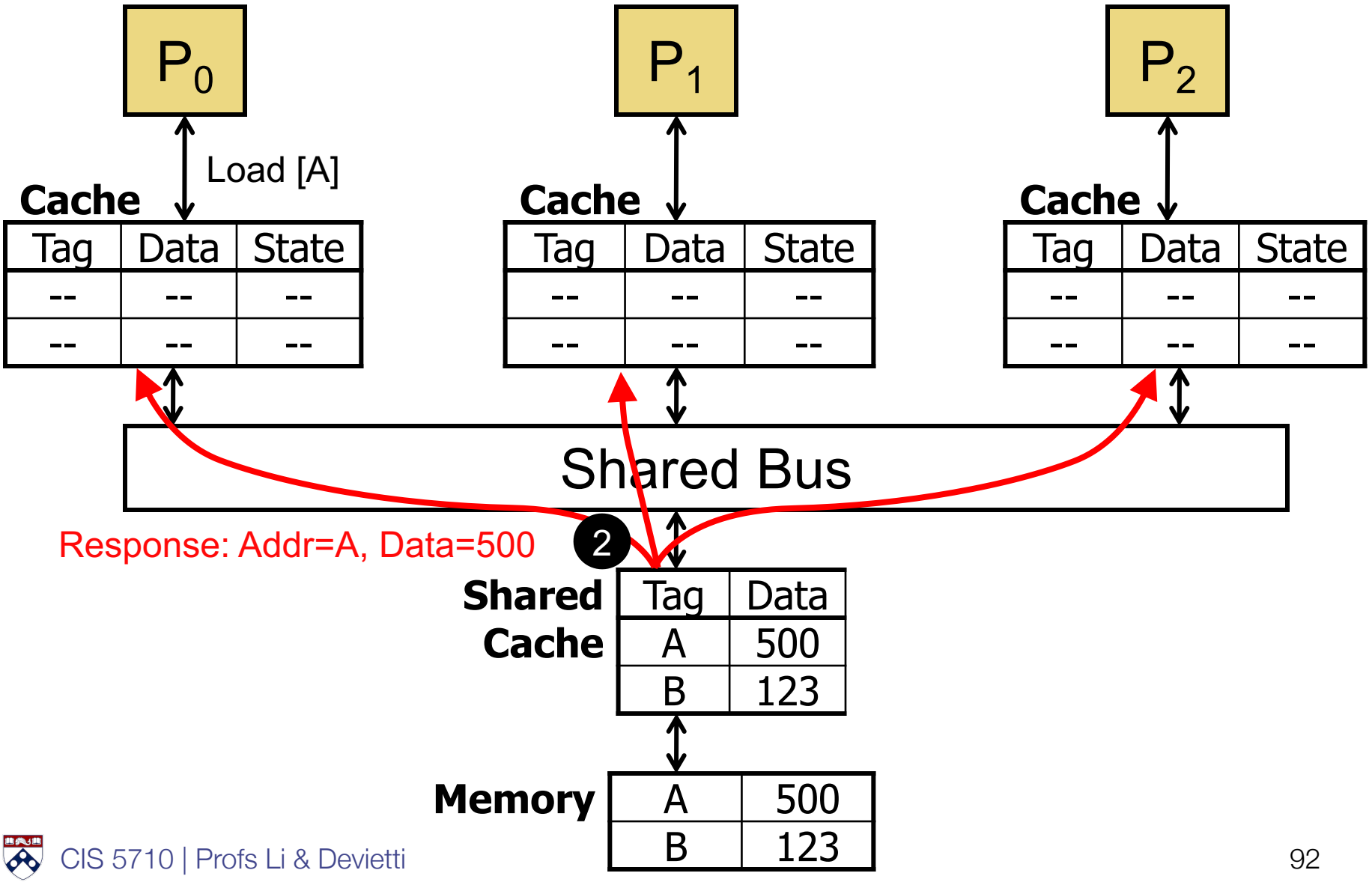
MESI Snooping Coherence: Step #1



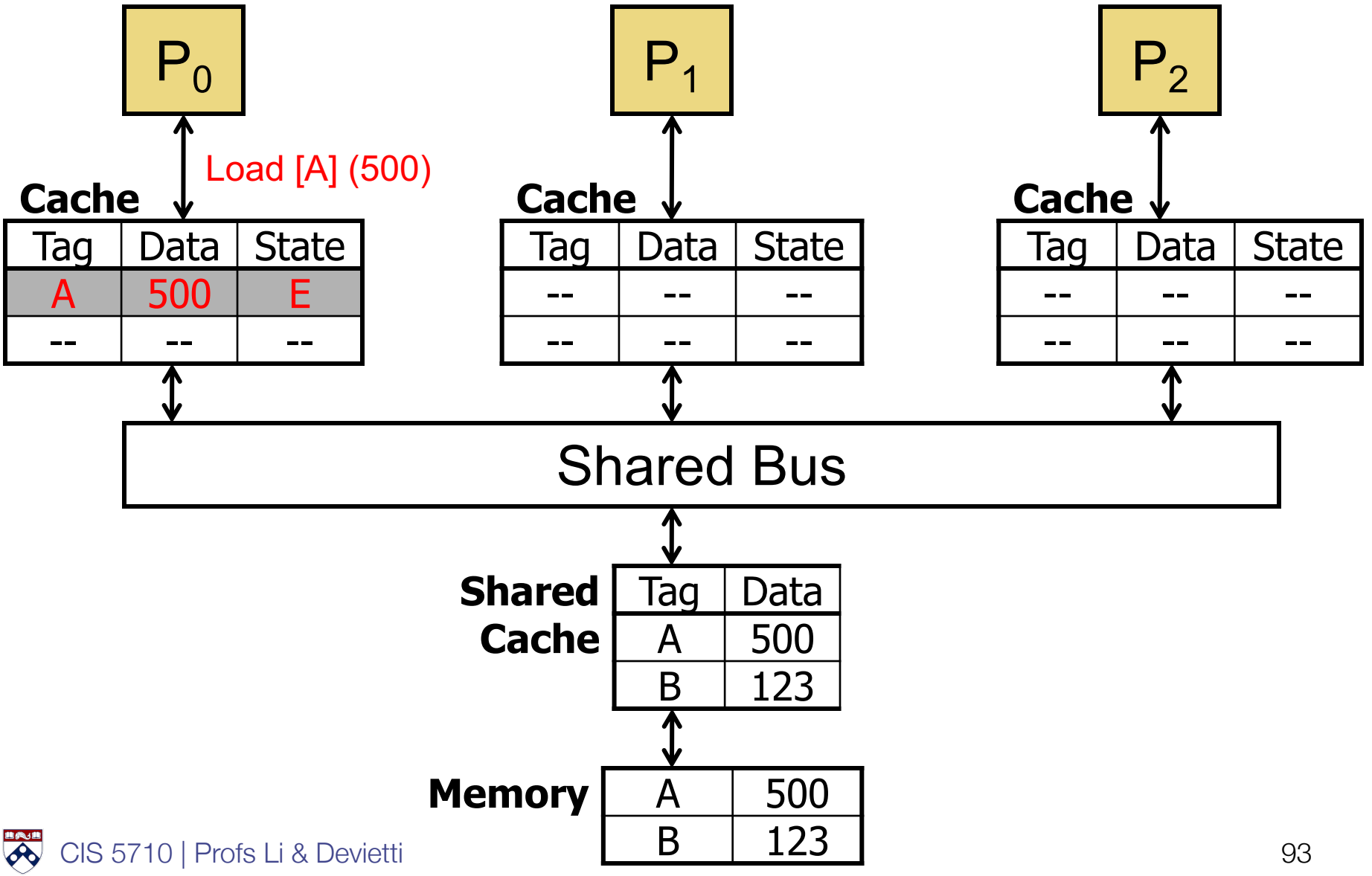
MESI Snooping Coherence: Step #2



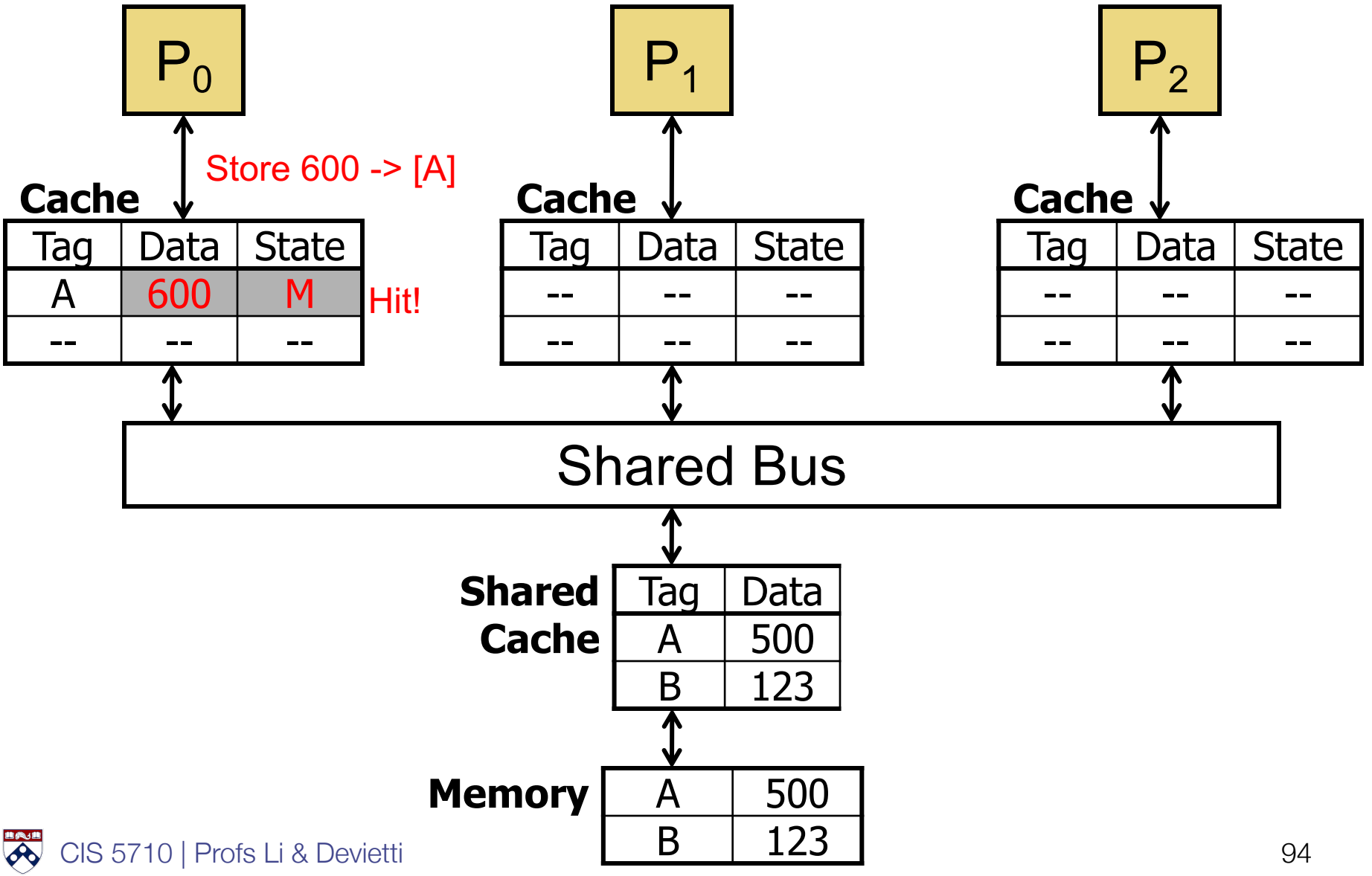
MESI Snooping Coherence: Step #3



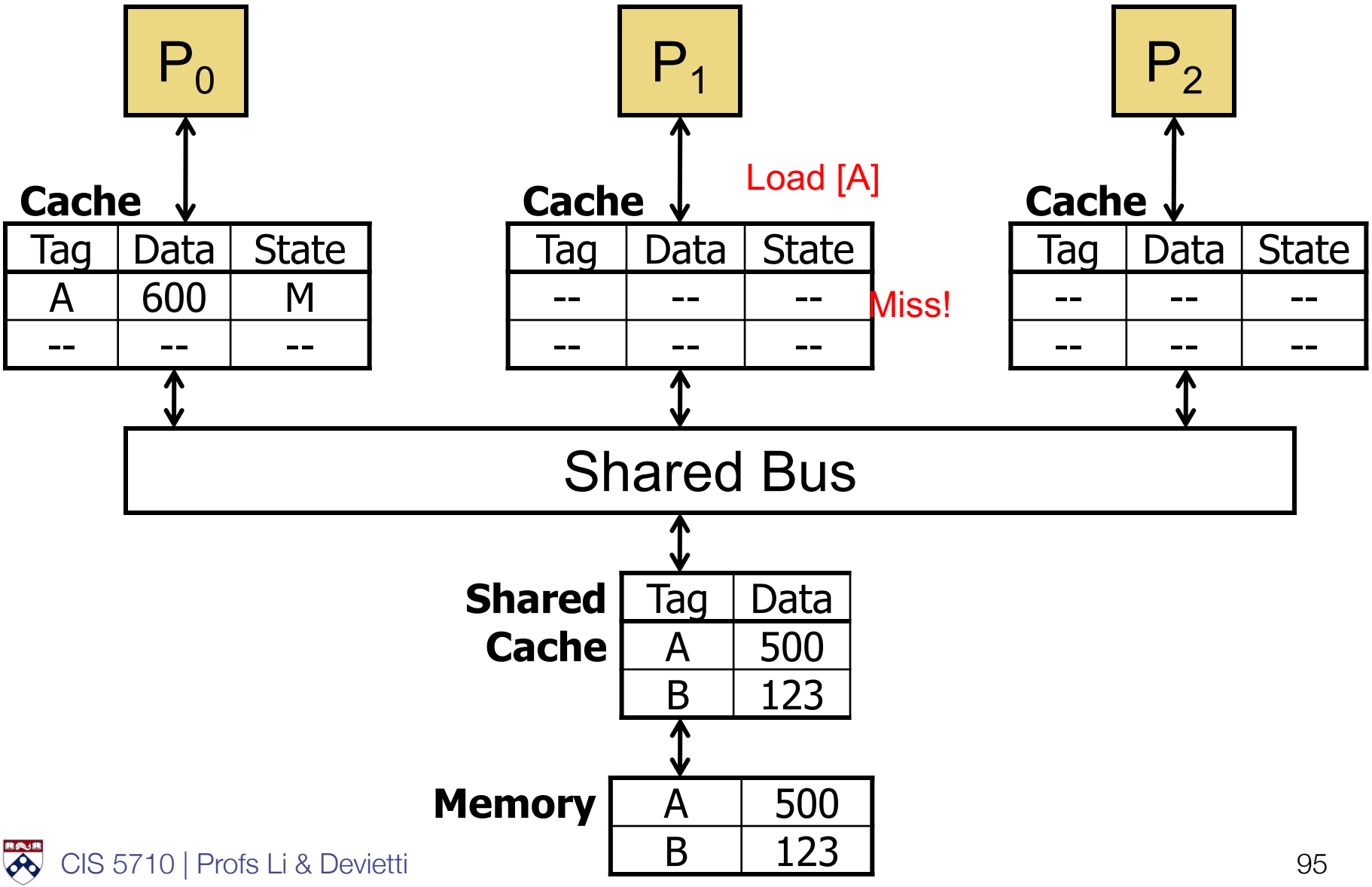
MESI Snooping Coherence: Step #4



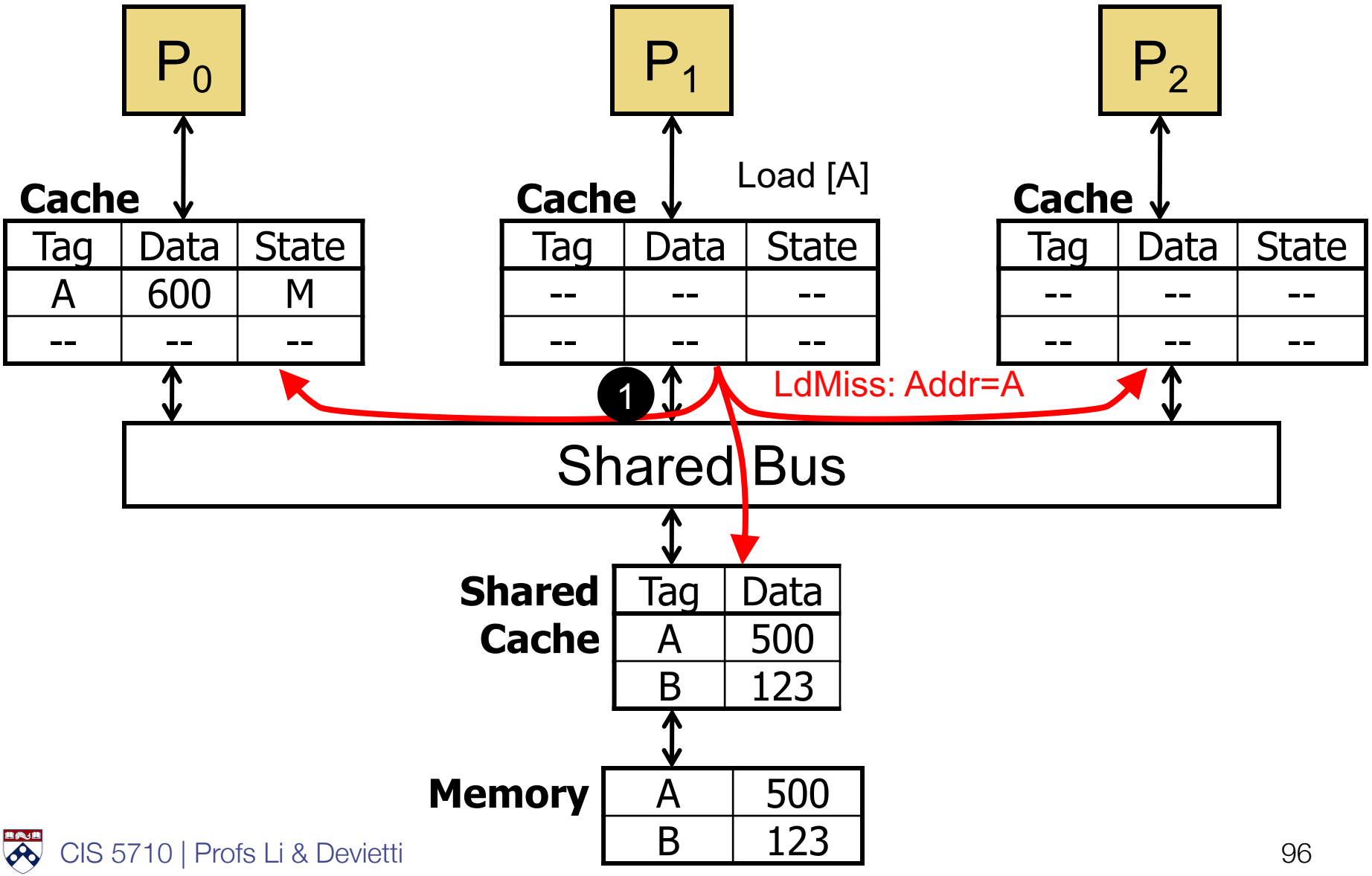
MESI Snooping Coherence: Step #5



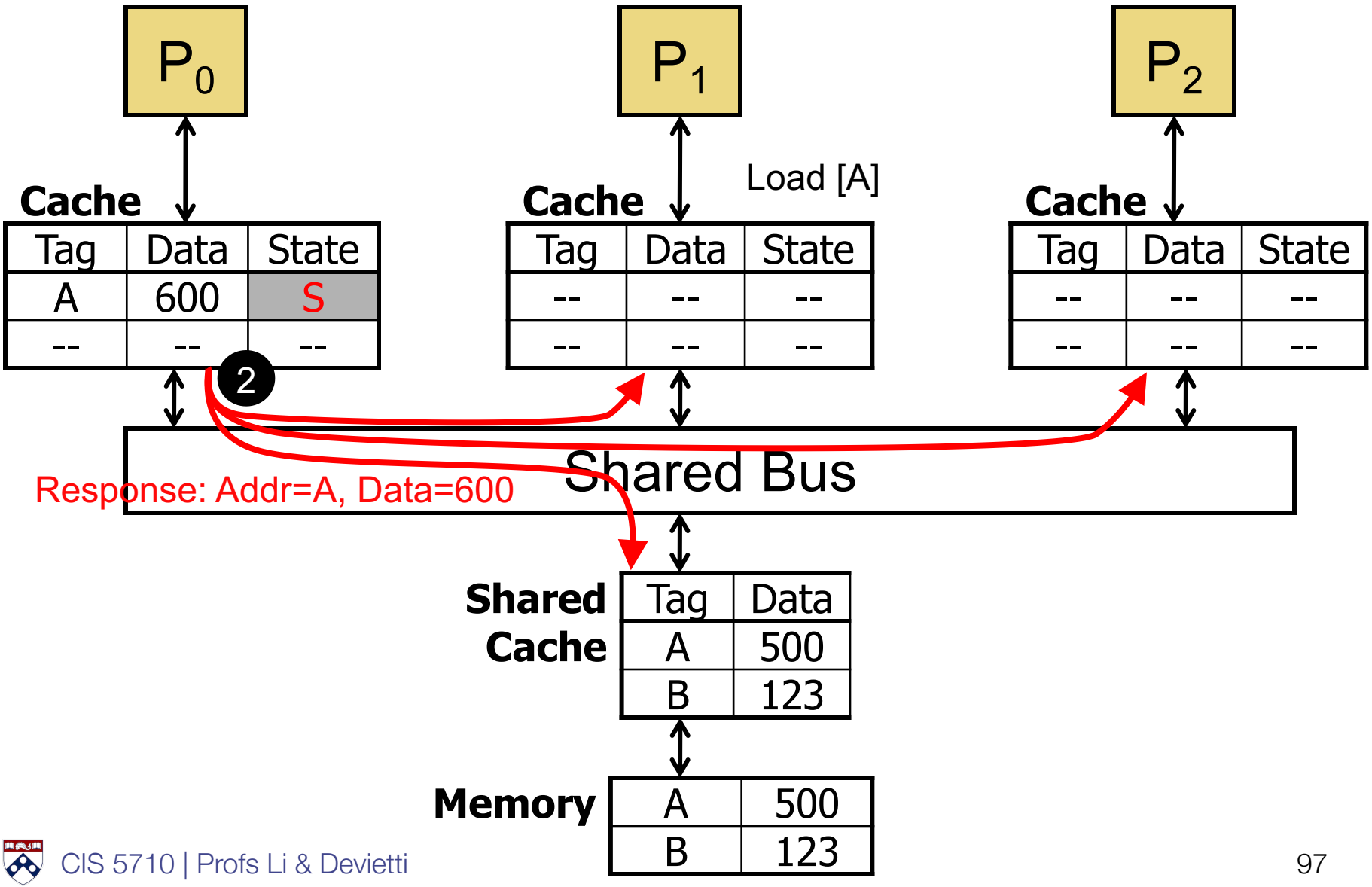
MESI Snooping Coherence: Step #6



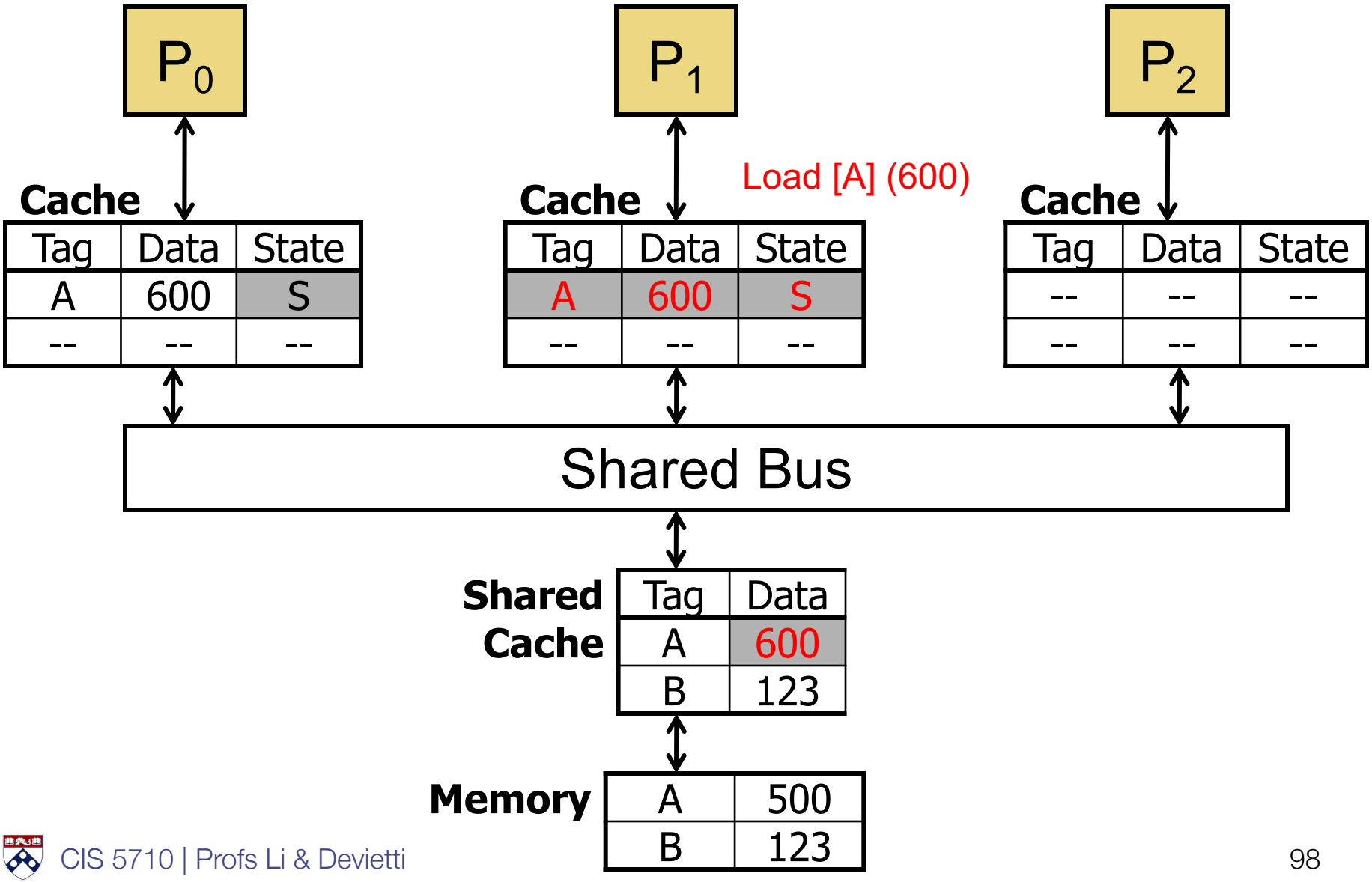
MESI Snooping Coherence: Step #7



MESI Snooping Coherence: Step #8

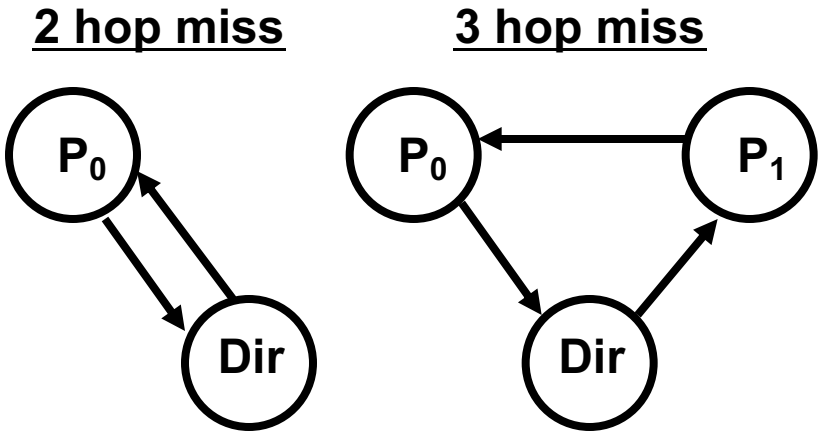


MESI Snooping Coherence: Step #9



Directory Downside: Latency

- Directory protocols
  - + Lower bandwidth consumption → more scalable
  - – Longer latencies
- Getting data from other processor (P1)
  - Assume cache-to-cache transfer optimization
  - Snooping: 2 hops (P0→P1→P0)
  - – Directory: **3 hops** (P0→memory→P1→P0)
  - Common, with many processors high probability someone has it



Scaling Cache Coherence

- **Scalable interconnect**
  - Build switched interconnect to communicate among cores
- **Scalable directory lookup bandwidth**
  - Address interleave (or "bank") the last-level cache
  - Low-order index bits select which cache bank to access
  - Coherence controller per bank
- **Scalable traffic**
  - Amortized analysis shows traffic overhead independent of core #
  - Each invalidation can be tied back to some earlier request
- **Scalable storage**
  - Sharers bit vector uses n-bits for n cores, scales to ~32 cores
  - Inexact & "coarse" encodings trade more traffic for less storage
- Hierarchical design can help all of the above, too
- See: "Why On-Chip Cache Coherence is Here to Stay", CACM, 2012

Coherence Recap & Alternatives

- Keeps caches "coherent"
  - And keep caches transparent to software
- Alternatives to cache coherence
  - #1: require software to explicitly manage cache (hard to use)
    - GPU caches work this way
  - #2: message passing (programming without shared memory)
    - Used in clusters of machines for high-performance computing
- However, directory-based coherence protocol scales well
  - >200-core systems commercially available

Synchronization

Example #1: Bank Accounts

- Consider

```
struct acct_t { int balance; … };
struct acct_t accounts[MAX_ACCT];     // current balances

struct trans_t { int id; int amount; };
struct trans_t transactions[MAX_TRANS];  // debit amounts

for (i = 0; i < MAX_TRANS; i++) {
  debit(transactions[i].id, transactions[i].amount);
}

void debit(int id, int amount) {
  if (accounts[id].balance >= amount) {
    accounts[id].balance -= amount;
  }
}
```

- Can we do "debit" operations in parallel?
  - Does the order matter?

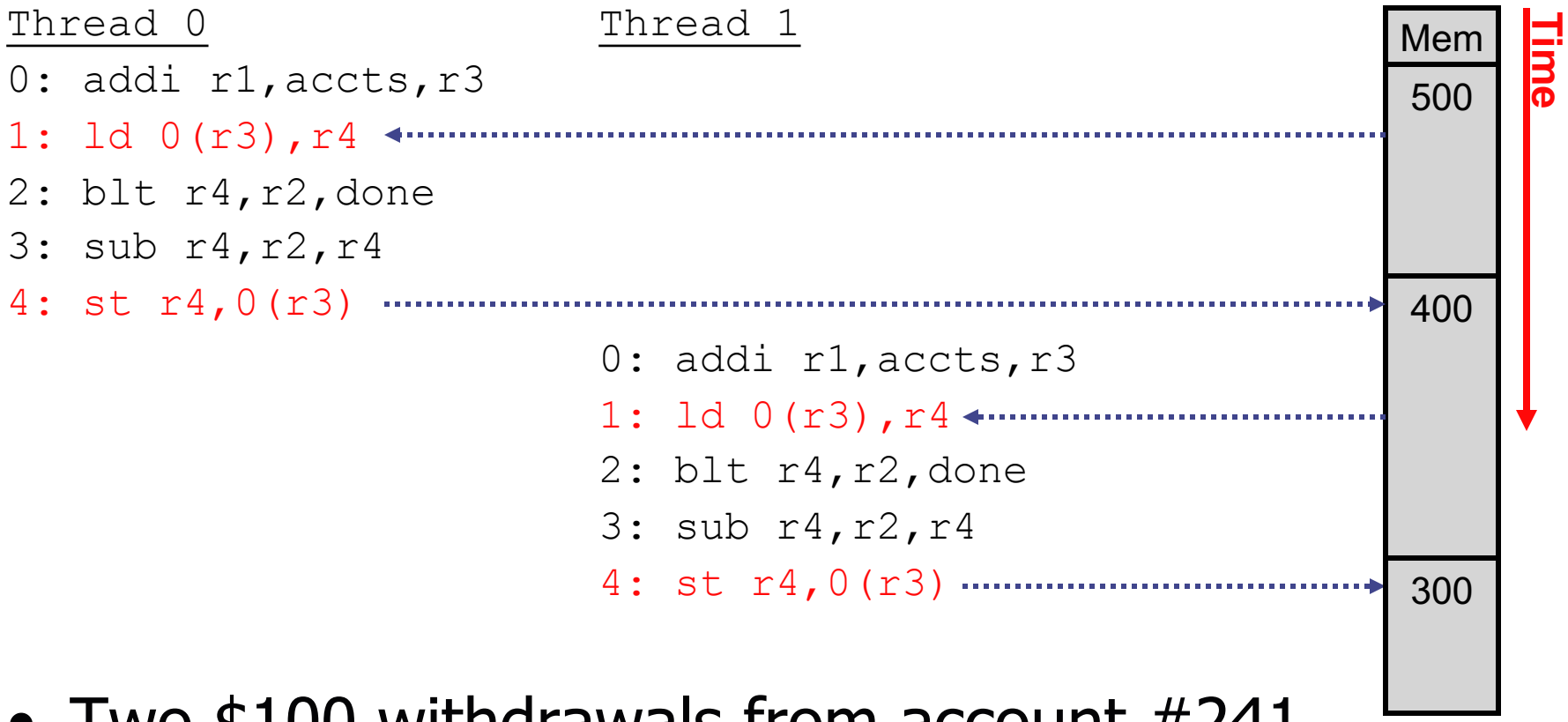
Example #1: Bank Accounts

```
struct acct_t { int bal; … };
shared struct acct_t accts[MAX_ACCT];
void debit(int id, int amt) {
  if (accts[id].bal >= amt)
  {
     accts[id].bal -= amt;
  }
}
```

```
0: addi r1,accts,r3
1: ld 0(r3),r4
2: blt r4,r2,done
3: sub r4,r2,r4
4: st r4,0(r3)
```

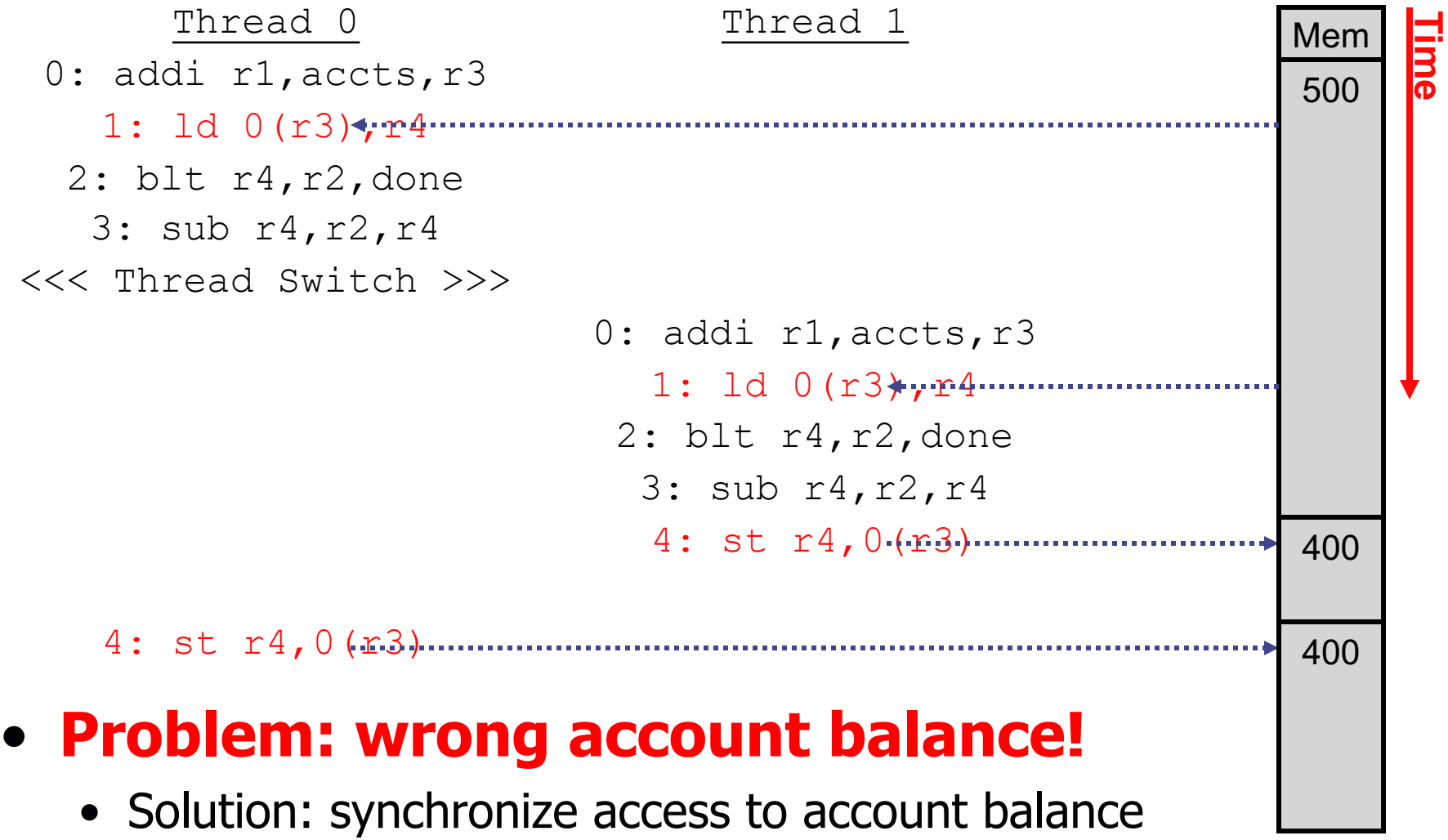
- Example of **Thread-level parallelism (TLP)**
- Process debits in parallel using multiple threads
  - **`accts`** is global and thus **shared**, can't register allocate
  - **`id`**, **`amt`** are private variables, allocated to registers **`r1`**, **`r2`**
- We'll use this as our running example

An Example Execution



- Two $100 withdrawals from account #241
  - Each transaction executed on different processor
  - Track **`accts[241].bal`** (address is in **`r3`**)

A Problem Execution



- **Problem: wrong account balance!**
  - Solution: synchronize access to account balance

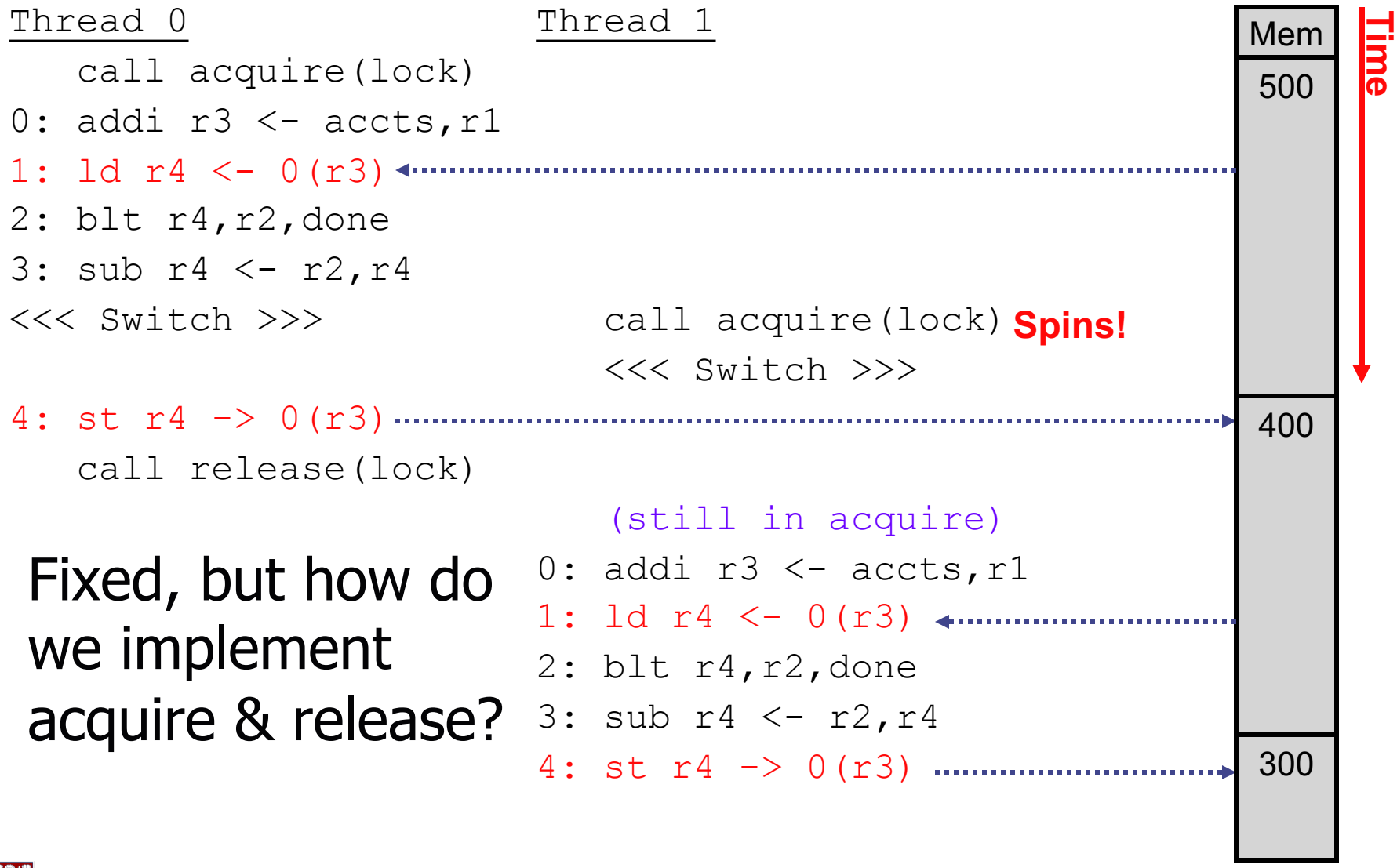
Synchronization

- **Synchronization**: a key issue for shared memory
- Regulate access to shared data (mutual exclusion)
- Low-level primitive: **lock** (higher-level: "semaphore")
  - Operations: `acquire(lock)` and `release(lock)`
  - Region between `acquire` and `release` is a **critical section**
  - Must interleave `acquire` and `release`
  - Interfering `acquire` will block
- Another option: **Barrier synchronization**
  - Blocks until all threads reach barrier, used at end of "parallel_for"

```
struct acct_t { int bal; … };
shared struct acct_t accts[MAX_ACCT];
shared int lock;
void debit(int id, int amt):
  acquire(lock);
  if (accts[id].bal >= amt) {
     accts[id].bal -= amt;
  }
  release(lock);
```

critical section

A Synchronized Execution



- Fixed, but how do we implement acquire & release?

First try at locking (incorrect!)

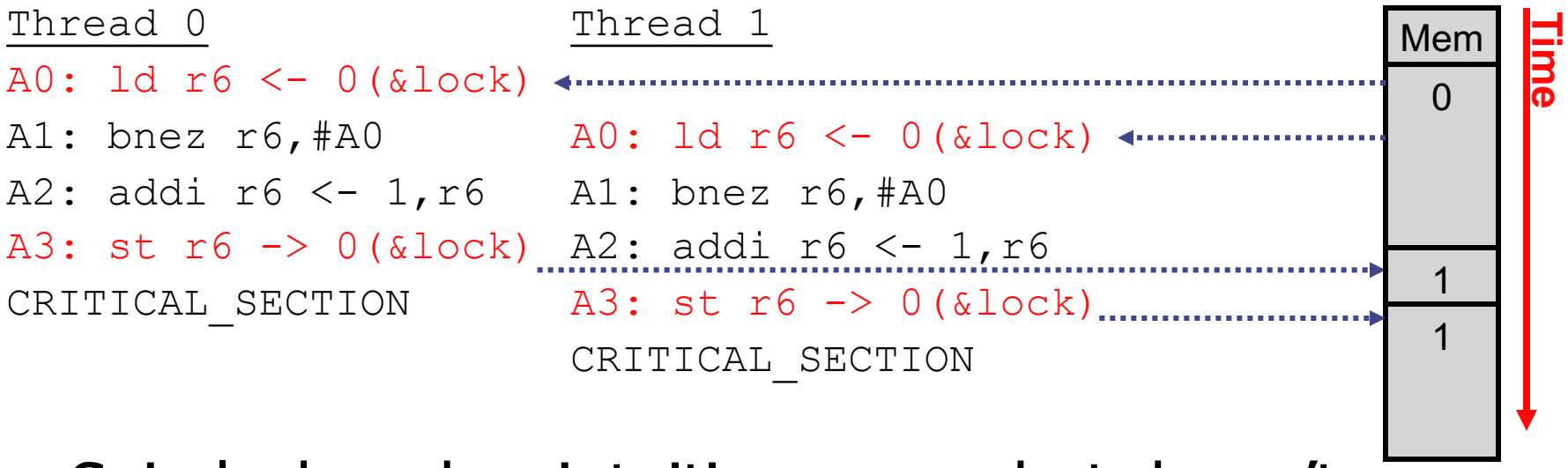
- **Spin lock**: software lock implementation
  - `acquire(lock): while (lock != 0) {} lock = 1;`
    - "Spin" while lock is 1, wait for it to turn 0

```
A0:  ld r6 <- 0(&lock)
A1:  bnez r6,A0
A2:  addi r6 <- 1,r6
A3:  st r6 -> 0(&lock)
```

  - `release(lock): lock = 0;`

```
R0:  st r0 -> 0(&lock)      // r0 holds 0
```

Incorrect Lock Implementation



- Spin lock makes intuitive sense, but doesn't actually work
  - Loads/stores of two **`acquire`** sequences can be interleaved
  - Lock **`acquire`** sequence also not atomic
  - **Same problem as before!**
- Note, **`release`** is trivially atomic

Correct Lock: Compare and Swap

- ISA provides an atomic lock acquisition instruction
  - Example: **atomic compare-and-swap (CAS)**

    `cas r3 <- r1,r2,0(&lock)`

    - Atomically executes:

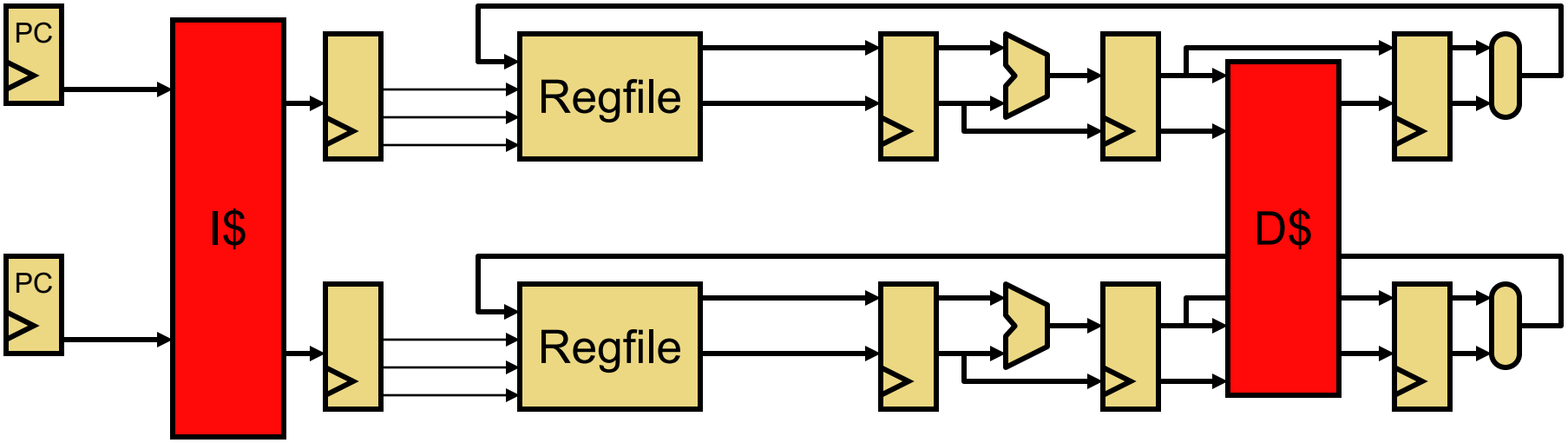
```
ld r3 <- 0(&lock)
if r3 == r2:
  st r1 -> 0(&lock)
```

- New acquire sequence

```
A0: cas r3 <- 1,0,0(&lock)
A1: bnez r3,A0
```

  - If lock was initially held (1), don't change it, **keep looping**
  - If lock was initially free (0), acquire it, break out of loop
- Ensures lock held by **at most one thread**
  - Other variants: exchange, compare-and-set, test-and-set (t&s), or fetch-and-add

CAS Implementation



- How is CAS implemented?
  - Need to ensure no intervening memory operations
  - Requires blocking access by other threads temporarily (yuck)
- How to pipeline it?
  - Both a load and a store – complicated!

Load-link/store-conditional

- CAS is not very "RISC"-like
  - Some ISAs use load-link/store-conditional instead

```
label:
    load-link r1 <- 0(&lock)
    // your code here
    store-conditional r2 -> 0(&lock)
    branch-not-zero label    // check for failure
```

  - On `load-link`, processor remembers address…
    - …And looks for writes by other processors
    - If write is detected, next `store-conditional` will fail
      - Sets failure condition
- LL/SC used by ARM, RV, PowerPC, MIPS

Lock Correctness

```
Thread 0                          Thread 1
A0: cas r1 <- 1,0,0(&lock)
A1: bnez r1,#A0                   A0: cas r1 <- 1,0,0(&lock)
CRITICAL_SECTION                  A1: bnez r1,#A0
                                  A0: cas r1 <- 1,0,0(&lock)
                                  A1: bnez r1,#A0
```

+ Lock actually works…
  - Thread 1 keeps spinning

- Sometimes called a “test-and-set lock”

Test-and-Set Lock Performance

```
Thread 0                          Thread 1
A0: cas r1 <- 1,0,0(&lock)
A1: bnez r1,#A0                   A0: cas r1 <- 1,0,0(&lock)
A0: cas r1 <- 1,0,0(&lock)        A1: bnez r1,#A0
A1: bnez r1,#A0                   A0: cas r1 <- 1,0,0(&lock)
                                  A1: bnez r1,#A0
```

- …but performs poorly
  - Consider 3 processors rather than 2
  - Processor 2 (not shown) has the lock and is in the critical section
  - But what are processors 0 and 1 doing in the meantime?
    - Loops of **`cas`**
    - each one gets the line in Modified state
    - Generating a ton of useless interconnect traffic

Test-and-Test-and-Set Locks

- Solution: **test-and-test-and-set locks**
  - New acquire sequence

```
A0: ld r1 <- 0(&lock)
A1: bnez r1,A0
A2: addi r1 <- 1,r1
A3: cas r1 <- r1,0,0(&lock)
A4: bnez r1,A0
```

  - Within each loop iteration, before doing a **`swap`**
    - Spin with a simple load to see if lock value has changed
    - Only do a **`cas`** if lock is actually free
  - Processors can spin on a busy lock locally (in their own cache)

    +Less unnecessary interconnect traffic
  - Note: no new insns, just better software!

Queue Locks

- Releasing a contended lock is tricky
  - Lock release by one processor creates "free-for-all" by others
  - – lots of `cas` requests, but only one will actually get the lock
- **Software queue lock**
  - Each waiting processor spins on a different location
  - When lock is released by one processor...
    - Only the next processors sees its location go "unlocked"
    - Others continue spinning locally
  - Effectively, pass lock from one processor to the next, in order

+ Greatly reduced network traffic (no mad rush for the lock)

+ Fairness (lock acquired in FIFO order)

– Extra complexity is only worth it if there's contention

– Poor performance if one thread is descheduled by OS

Coarse-grain locks: easy but slow

- **Coarse-grain locks**: e.g., one lock for entire database
  - \+ Easy to make correct: no chance for unintended interference
  - – Limits parallelism: no two critical sections can proceed in parallel

```
struct acct_t { int bal; … };
shared struct acct_t  accts[MAX_ACCT];
shared Lock_t lock;
void debit(int id, int amt) {
  acquire(lock);
  if (accts[id].bal >= amt) {
     accts[id].bal -= amt;
  }
  release(lock);
}
```

Fine-grain locks: parallel but hard

- **Fine-grain locks**: e.g., one lock per record
  - + Fast: critical sections (to different records) can proceed in parallel
  - – Easy to make mistakes
  - What about critical sections that require two locks?

```
struct acct_t { int bal, Lock_t lock; …  };
shared struct acct_t  accts[MAX_ACCT];

void debit(int id, int amt) {
  acquire(accts[id].lock);
  if (accts[id].bal >= amt) {
     accts[id].bal -= amt;
  }
  release(accts[id].lock);
}
```

Multiple Locks

- Consider acct-to-acct transfers
  - Must acquire both **`id_from`**, **`id_to`** locks
  - Running example with accts 241 and 37
  - Simultaneous transfers 241 → 37 and 37 → 241

```
struct acct_t { int bal, Lock_t lock; …};
shared struct acct_t  accts[MAX_ACCT];
void transfer(int id_from, int id_to, int amt) {
  acquire(accts[id_from].lock);
  acquire(accts[id_to].lock);
  if (accts[id_from].bal >= amt) {
     accts[id_from].bal -= amt;
     accts[id_to].bal += amt;
  }
  release(accts[id_to].lock);
  release(accts[id_from].lock);
}
```

Multiple Locks And Deadlock

Thread 0

```
id_from = 241;
id_to = 37;

acquire(accts[241].lock);
// wait to acquire lock 37
// waiting…
// still waiting…
```

Thread 1

```
id_from = 37;
id_to = 241;

acquire(accts[37].lock);
// wait to acquire lock 241
// waiting…
// …
```


FAILBlog

Multiple Locks And Deadlock

```
Thread 0                                Thread 1

id_from = 241;                          id_from = 37;
id_to = 37;                             id_to = 241;

acquire(accts[241].lock);               acquire(accts[37].lock);
// wait to acquire lock 37              // wait to acquire lock 241
// waiting…                             // waiting…
// still waiting…                       // …
```

- **Deadlock**: circular wait for shared resources
  - Thread 0 has lock 241 and waits for lock 37
  - Thread 1 has lock 37 and waits for lock 241
  - Obviously this is a problem
  - The solution is …

Coffman Conditions for Deadlock

- 4 necessary conditions
  - mutual exclusion
  - hold+wait
  - no preemption
  - circular waiting
- break any **one** of these conditions and your system is free of deadlocks
  - hard to accomplish in practice

Correct Multiple Lock Program

- **Always acquire multiple locks in same order**
  - Yet another thing to keep in mind when programming

```
struct acct_t { int bal, Lock_t lock; … };
shared struct acct_t  accts[MAX_ACCT];
void transfer(int id_from, int id_to, int amt) {
  int id_first = min(id_from, id_to);
  int id_second = max(id_from, id_to);

  acquire(accts[id_first].lock);
  acquire(accts[id_second].lock);
  if (accts[id_from].bal >= amt) {
     accts[id_from].bal -= amt;
     accts[id_to].bal += amt;
  }
  release(accts[id_second].lock);
  release(accts[id_first].lock);
}
```

Correct Multiple Lock Execution

Thread 0

```
id_from = 241;
id_to = 37;
id_first = min(241,37)=37;
id_second = max(37,241)=241;

acquire(accts[37].lock);
acquire(accts[241].lock);
// do stuff
release(accts[241].lock);
release(accts[37].lock);
```

Thread 1

```
id_from = 37;
id_to = 241;
id_first = min(37,241)=37;
id_second = max(37,241)=241;

// wait to acquire lock 37
// waiting…
// …
// …
// …
acquire(accts[37].lock);
```

- Great, are we done? Nope!

And To Make It Worse…

- **Acquiring locks is expensive…**
  - Requires slow atomic instructions
  - Ordering constraints (up next) make it even slower
- **…and 99% of the time un-necessary**
  - Most concurrent actions don't actually share data
- Fixing these problem is an area of active research
  - One proposed solution: **Transactional Memory**
  - Programmer uses construct: "atomic { … code … }"
    - Hardware & compiler execute the code atomically
    - Uses **speculation**, rolls back on conflicting accesses
  - Intel has Hardware TM support
    - has seen limited use, AMD/ARM considering TM as well

Memory Consistency Models

Shared Memory Example #1

- **Initially: all variables zero** (that is, x is 0, y is 0)

thread 1	thread 2
store 1 → y load x	store 1 → x load y

- What value pairs can be read by the two loads?

Shared Memory Example #1

- Initially: all variables zero (that is, x is 0, y is 0)

thread 1	thread 2
store 1 → y load x	store 1 → x load y

- What value pairs can be read by the two loads?

store 1 → y load x store 1 → x load y (x=0, y=1)	store 1 → y store 1 → x load x load y (x=1, y=1)	store 1 → y store 1 → x load y load x (x=1, y=1)
store 1 → x load y store 1 → y load x (x=1, y=0)	store 1 → x store 1 → y load y load x (x=1, y=1)	store 1 → x store 1 → y load x load y (x=1, y=1)

- What about (x=0, y=0)? Nope…*or can it?*

Shared Memory Example #2

- **Initially: all variables zero** ("flag" is 0, "a" is 0)

thread 1	thread 2
`store 1 → a` `store 1 → flag`	`loop: if (flag == 0) goto loop` `load a`

- What value can be read by load a?
- Can "load a" read the value zero?
  - Unfortunately, *yes it can!*

What is Going On?

- Reordering of memory operations to different addresses!
- **In the hardware**
  1. To tolerate write latency
     - Cores don't wait for writes to complete (via store buffers)
     - And why should they? No reason to wait with single-thread code
  2. To simplify out-of-order execution
- **In the compiler**
  3. Compilers are generally allowed to re-order memory operations to different addresses
     - Many compiler optimizations reorder memory operations.

Memory Consistency

- **Cache coherence**
  - Creates globally uniform (consistent) view of a single cache block
  - Not enough on its own:
    - What about accesses to different cache blocks?
    - Some optimizations skip coherence(!)
- **Memory consistency model**
  - Specifies the semantics of shared memory operations
  - i.e., what value(s) a load may return

Memory Consistency Models

- **Sequential consistency (SC)** (MIPS, PA-RISC)
  - **Typically what programmers expect**
  - 1. Processors see own loads and stores in program order
  - 2. Processors see others' loads and stores in program order
  - 3. All processors see same global load/store ordering
  - **Equivalent to multi-programmed uni-processor**
- **Total Store Order (TSO)** (x86, SPARC)
  - Allows an in-order (FIFO) store buffer
    - Stores can be deferred, but must go to cache in-order
- **Release consistency (RC)** (ARM, RV, Power)
  - Allows an un-ordered coalescing store buffer
    - Stores can be put into cache in any order
  - Loads re-ordered, too

Axiomatic vs Operational Semantics

- Two ways to understand consistency models
  - **Reorderings** allowed by the model (axiomatic)
  - **Hardware optimizations** allowed by the model (operational)

TABLE 3.4: SC Ordering Rules. An "X" Denotes an Enforced Ordering.

		Operation 2		
		Load	Store	RMW
Operation 1	Load	X	X	X
	Store	X	X	X
	RMW	X	X	X

equivalent

from "A Primer on Memory Consistency and Cache Coherence" by Sorin, Hill and Wood

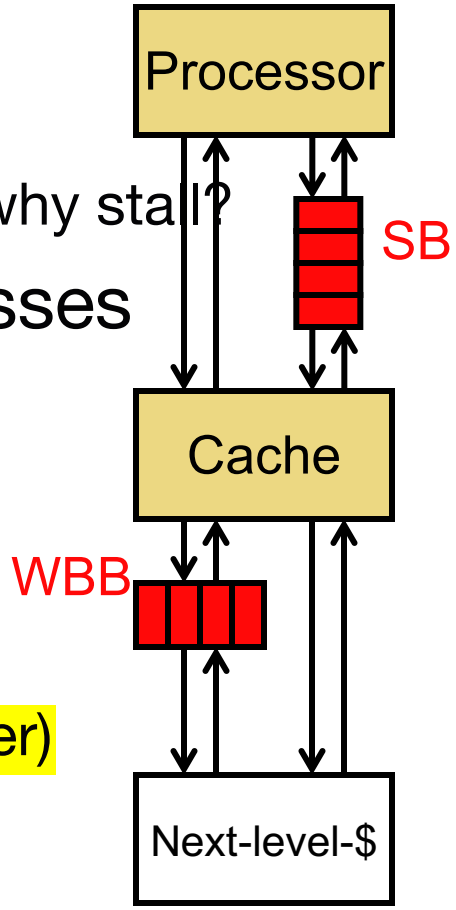
TSO (x86) Axiomatic Semantics

TABLE 4.4: TSO Ordering Rules. An "X" Denotes an Enforced Ordering. A "B" Denotes that Bypassing is Required if the Operations are to the Same Address. Entries that are Different from the SC Ordering Rules are Shaded and Shown in Bold.

		Operation 2			
		Load	Store	RMW	FENCE
Operation 1	Load	X	X	X	**X**
	Store	**B**	X	X	**X**
	RMW	X	X	X	**X**
	FENCE	**X**	**X**	**X**	**X**

Write Misses and Store Buffers

- Read miss?
  - Load can't go on without the data, it must stall
- Write miss?
  - Technically, no instruction is waiting for data, why stall?
- Store buffer: a small buffer for store misses
  - Stores put address/value into SB, keep going
  - SB writes to D$ in the background
  - Loads must search SB (in addition to D$)
  - (mostly) eliminates stalls on write misses
  - creates some problems in multiprocessors (later)



Why? To hide store miss latency

- Why allow such odd behavior?
  - Reason #1: hiding store miss latency
- Recall (back from caching unit)
  - Hide store miss latency with **store buffer**
- Unfortunately, it complicates multiprocessors
  - by allowing reordering of a store and a load (to different addresses)
- Example:
  - Both stores miss cache, are put in store buffers
  - Loads hit, receive value before stores complete, see "old" values

thread 1	thread 2
store 1 → y load x	store 1 → x load y

Shared Memory Example #1

- Initially: all variables zero (that is, x is 0, y is 0)

thread 1	thread 2
store 1 → y load x	store 1 → x load y

- What value pairs can be read by the two loads?

store 1 → y load x store 1 → x load y (x=0, y=1)	store 1 → y store 1 → x load x load y (x=1, y=1)	store 1 → y store 1 → x load y load x (x=1, y=1)
store 1 → x load y store 1 → y load x (x=1, y=0)	store 1 → x store 1 → y load y load x (x=1, y=1)	store 1 → x store 1 → y load x load y (x=1, y=1)

- What about (x=0,y=0)? Yes! (for x86, RV, ARM, Power)

RC Axiomatic Semantics

TABLE 5.5: XC Ordering Rules. An "X" Denotes an Enforced Ordering. An "A" Denotes an Ordering that is Enforced Only if the Operations are to the Same Address. A "B" Denotes that Bypassing is Required if the Operations are to the Same Address. Entries Different from TSO are Shaded and Indicated in Bold Font.

		Operation 2			
Operation 1		Load	Store	RMW	FENCE
	Load	**A**	**A**	**A**	X
	Store	B	**A**	**A**	X
	RMW	**A**	**A**	**A**	X
	FENCE	X	X	X	X

Why? Simplify OoO Execution

- One key benefit of out-of-order execution:
  - OoO execution of loads to (same or different) addresses

thread 1	thread 2
`store 1 → a` `store 1 → flag`	`loop: if (flag == 0) goto loop` `load a`

- Two options for hardware designers:
  - Option #1: change the spec to allow this reordering
  - Option #2: hw **detects & recovers** from such reorderings
    - Scan load queue (LQ) when cache block is invalidated
  - Note: some store buffers reorder stores by same thread to different addresses (as in thread 1 above)

Why? Allow Compiler Optimizations

- Compiler optimizations are important
  - Consider a case of loop-invariant code motion:
  - Optimized code is much faster, but **many** memory operations have been reordered

original code

```
for (i=0; i<10; i++)
  array[i] = array2[i] + x^2;
```

optimized code

```
tmp1 = x^2;
for (i=0; i<10; i++)
  array[i] = array2[i] + tmp1;
```

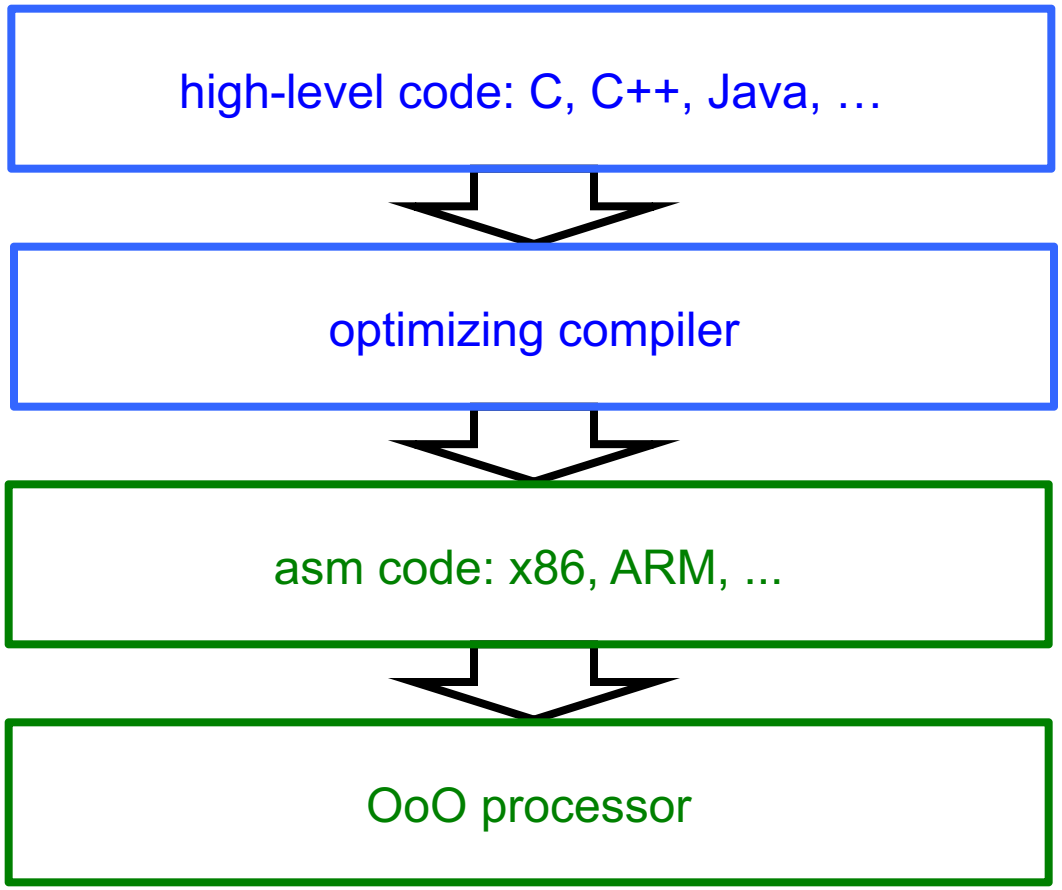
Shared Memory Example #2

- Initially: all variables zero (flag == a == 0)

thread 1	thread 2
`store 1 → a` `store 1 → flag`	`loop: if (flag == 0) goto loop` `load a`

- What value can be read by "load a"?
  - "load a" can see the value "1"
- Can "load a" read the value zero? (same as last slide)
  - **Yes! (for ARM, PowerPC, Itanium, and Alpha)**
  - **No! (for Intel/AMD x86, Sun SPARC, IBM 370)**
    - Assuming the compiler didn't reorder anything…

Consistency Models: A Layered Cake



- How do we prevent our code from getting screwed up?
  - by the compiler and/or the hw?
- We adhere to the **language's consistency model**
  - compiler writer ensures code is correct on each hw architecture

Restoring Order (Hardware)

- Sometimes we really need ordering
  - Prime example: ordering between "lock" and data
- How? insert **fences (memory barriers)**
  - Special ISA instructions
- Example
  - Ensure loads/stores don't cross synchronization operations

```
lock acquire
fence
critical section
fence
lock release
```

- How do fences work?
  - They stall execution until store buffers are empty
  - Makes lock acquisition and release even slower

Restoring Order (Software)

- How do we tell compiler to not reorder things?
- In Java:
  - The built-in **synchronized** construct informs the compiler to limit its optimization scope (prevent reorderings across synchronization)
  - Or use **volatile** keyword to explicitly mark variables
    - gives SC semantics for all locations marked as **volatile**
  - Java compiler inserts the hardware-level ordering instructions
- In C/C++:
  - C++11 has a new **atomic** keyword, similar to Java's **volatile**

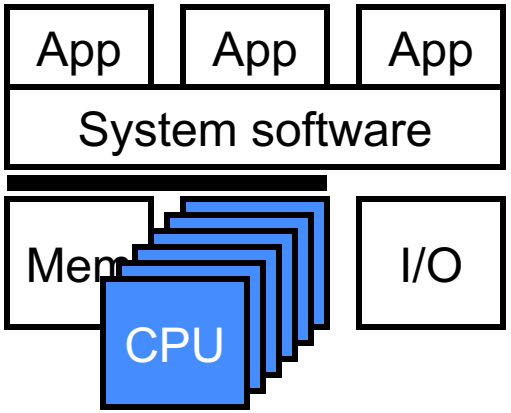
Use synchronization libraries

- Synchronization is very hard to get right
  - need solid understanding of language and hardware memory consistency models
- Use your language's built-in synchronization primitives
  - don't build your own unless you really need to!

SC for DRF programs

- If a program is **data-race-free**, all consistency models guarantee sequentially-consistent behavior
  - hw/compiler reorder operations, but we can't tell!
- a **data race** consists of:
  - two memory accesses
  - from different threads
  - to the same byte(s)
  - where at least one access is a write
  - without synchronization
- What if we do have a data race?
  - C/C++: anything can happen (just like buffer overflow)
  - Java: weird reorderings, but no out-of-thin-air reads

Summary



- Thread-level parallelism (TLP)
- Shared memory model
  - Multiplexed uniprocessor
  - Hardware multihreading
  - Multiprocessing
- Cache coherence
  - Valid/Invalid, MSI, MESI
- Parallel programming
- Synchronization
  - Lock implementation
  - Locking gotchas
- Memory consistency models